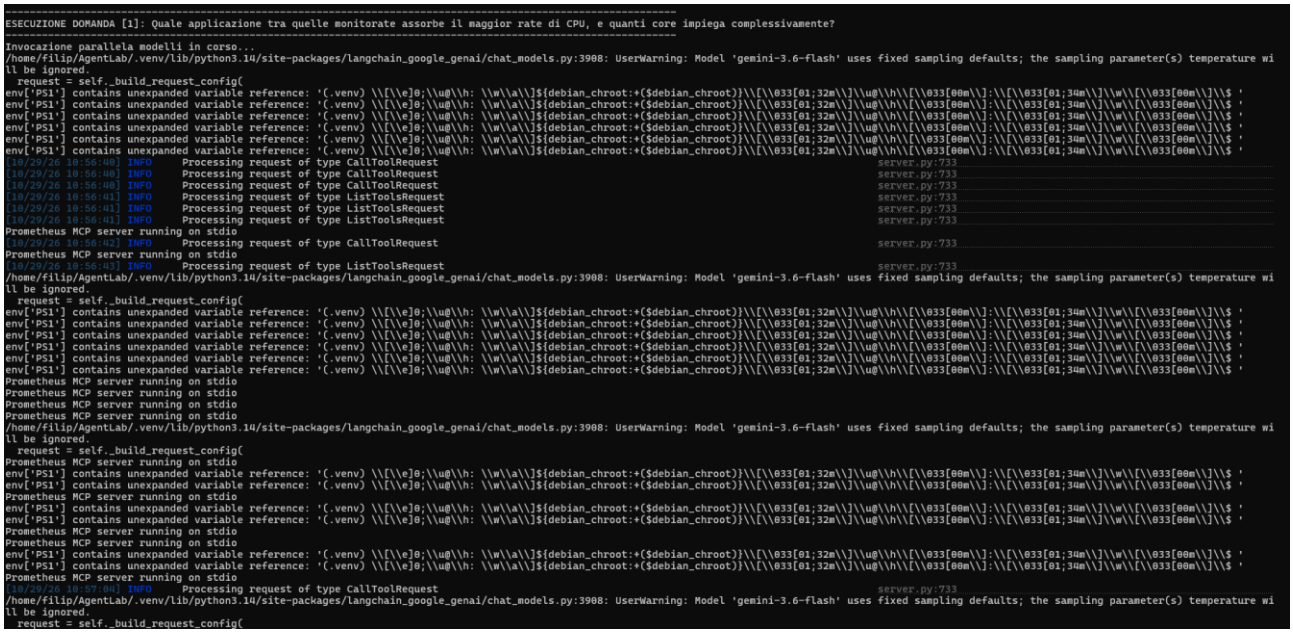
DOMANDA1



Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.14 ms	0	0	0	N/A	worker-app (0.5590 cores)
Gemini 2.5 Flash	46.42 s	128048	2629	130677	0.00	Analizzando i dati di utilizzo della CPU, l'applicazione che assorbe il maggior rate di CPU è `prometheus-prometheus-kube-prometheus-prometheus` con un consumo di circa **0.218 core**. Ecco il dettaglio delle applicazioni con il loro consumo di CPU (in core): * **prometheus-prometheus-kube-prometheus-prometheus**: 0.218 core * **alertmanager-prometheus-kube-prometheus-alertmanager**: 0.030 core * **prometheus-prometheus-node-exporter**: 0.018 core * **kube-proxy**: 0.017 core * **kindnet**: 0.012 core Si noti che i valori sono stati aggregati per `owner_name` (Deployment, StatefulSet o DaemonSet) per fornire una visione a livello di applicazione.

Gemini 3.6 Flash	37.30 s	7489	1528	9017	0.45	In base all'analisi dei carichi di lavoro e delle metriche di utilizzo CPU ricavate da Prometheus per le applicazioni monitorate nel cluster, ecco i dettagli:
						
						--- 	
						
						### **1. Mappatura Applicazioni e Utilizzo CPU (rate medio su 5 minuti)**
						
						* **`worker-app`** (2 pod attivi):
						  * `worker-app-84cf4dd446-ftlln`: ~**0,293 core**
						  * `worker-app-84cf4dd446-9kj75`: ~**0,294 core**
						  * **Totale per `worker-app`**: ~**0,587 core** (~587 millicore)
						
						* **`cache-app`** (1 pod attivo):
						  * `cache-app-5c6fd4f5cf-xxsz4`: ~**0,016 core** (~16 millicore)
						  * **Totale per `cache-app`**: ~**0,016 core**
						
						* **`demo-app`** (2 pod attivi):
						  * `demo-app-7958c64697-2qg9s`: **0,000 core**
						  * `demo-app-7958c64697-4rzjl`: **0,000 core**
						  * **Totale per `demo-app`**: **0,000 core**
						
						--- 	
						
						### **Conclusione**
						
						L'applicazione che assorbe il maggior rate di CPU è **`worker-app`**, impiegando complessivamente circa **0,587 core** (pari a circa **587 mCPU**).

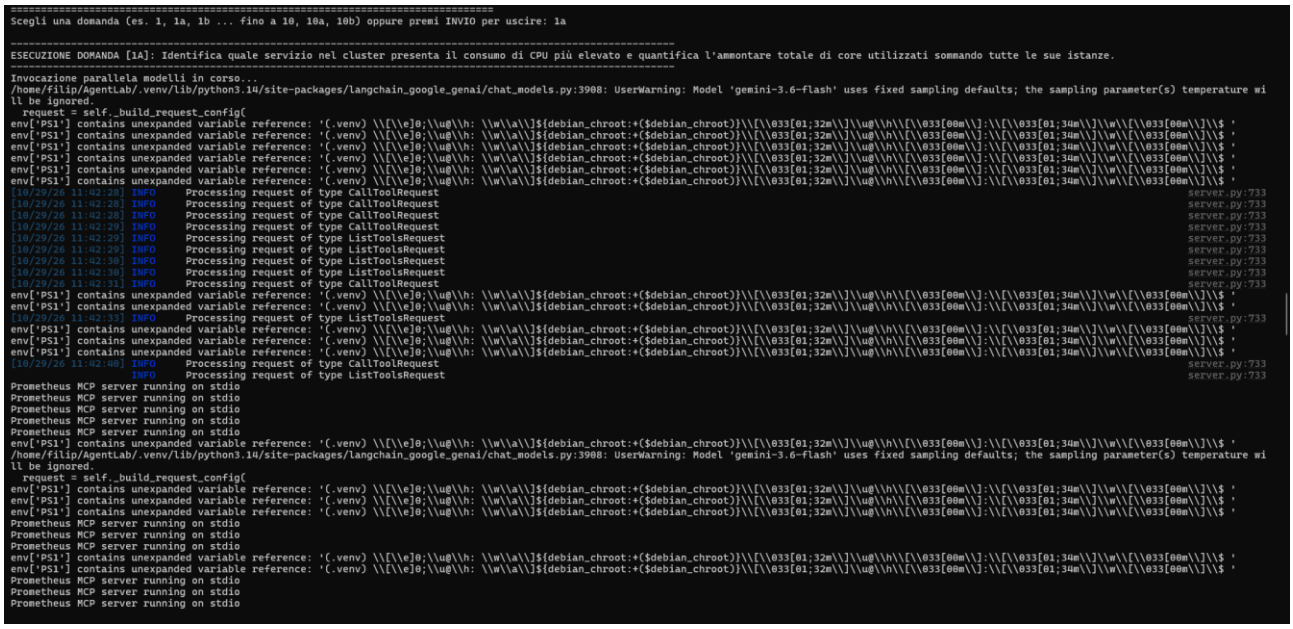
Claude Opus 5.5	32.53 s	4828	980	5808	0.45	L'applicazione che usa più CPU è **worker-app**: in totale impiega circa **0,578 core** (≈578 millicore). Il valore è il rate medio degli ultimi 5 minuti, misurato con `container_cpu_usage_seconds_total` nel namespace `default`. | Applicazione | Pod | CPU (core) | Totale app | |---|---|---|---| | **worker-app** | worker-app-84cf4dd446-ftlln | 0,2940 | **0,5783** | | | worker-app-84cf4dd446-9kj75 | 0,2843 | | | cache-app | cache-app-5c6fd4f5cf-xxsz4 | 0,0145 | 0,0145 | | demo-app | demo-app-7958c64697-2qg9s | 0 | 0 | | | demo-app-7958c64697-4rzjl | 0 | | **Cosa emerge:** - **worker-app** usa circa il 97% della CPU di tutte le applicazioni monitorate. Il carico è diviso in modo uniforme tra le due repliche (~0,29 core ciascuna), quindi il bilanciamento funziona. - **demo-app** risulta a 0 core su entrambi i pod: o è del tutto inattiva, o la metrica non viene raccolta per quei container. **Raccomandazioni:** 1. **Controllare requests e limits di worker-app.** Con ~0,29 core per pod, se il limite è vicino (ad esempio 300m o 500m) c'è rischio di throttling. Si può verificare con `rate(container_cpu_cfs_throttled_periods_total[5m])`. Se si usa l'HPA, conviene assicurarsi che la soglia di scaling tenga conto di questo consumo. 2. **Verificare demo-app.** Se 2 repliche sono sempre a 0 CPU, conviene ridurle (anche a 1), sempre che non debbano restare pronte per picchi di traffico. Prima va escluso un problema di raccolta della metrica. 3. **Guardare il trend.** Questo è un valore puntuale. Per capire se il consumo di worker-app sta crescendo, posso confrontarlo con lo storico salvato su MongoDB (`get_historical_metrics`) o con una range query su Prometheus.
Claude Sonnet 5.5	34.67 s	89472	876	90348	0.90	**L'applicazione che assorbe più CPU è `worker-app`, con circa 0,58 core in totale.** Ho calcolato il rate di CPU con `rate(container_cpu_usage_seconds_total[5m])` nel namespace `default`, sommando i container di ogni pod e poi i pod di ogni Deployment. | Applicazione | Pod | CPU per pod (core) | CPU totale (core) | |---|---|---|---| | **worker-app** | `worker-app-84cf4dd446-9kj75` | 0,2943 | **≈ 0,579** | | | `worker-app-84cf4dd446-ftlln` | 0,2848 | | | cache-app | `cache-app-5c6fd4f5cf-xxsz4` | 0,0149 | ≈ 0,015 | | demo-app | `demo-app-7958c64697-2qg9s` | 0 | 0 |

						| Applicazione | Pod | CPU per pod (core) | CPU totale (core) | |---|---|---|---| | **worker-app** | `worker-app-84cf4dd446-9kj75` | 0,2943 | **≈ 0,579** | | | `worker-app-84cf4dd446-ftlln` | 0,2848 | | | cache-app | `cache-app-5c6fd4f5cf-xxsz4` | 0,0149 | ≈ 0,015 | | demo-app | `demo-app-7958c64697-2qg9s` | 0 | 0 | | | `demo-app-7958c64697-4rzjl` | 0 | 0 | - **Distribuzione:** `worker-app` usa circa il 97% della CPU delle tre applicazioni. Le due repliche sono bilanciate, con circa 0,29 core ciascuna. - **`demo-app` a zero:** i suoi due pod risultano a 0 core. Può essere un'applicazione davvero inattiva, ma conviene controllare che il pod sia sano e che la metrica venga raccolta correttamente. - **Limiti:** i dati sono un'istantanea sugli ultimi 5 minuti. Non ho confrontato il valore con `requests` e `limits` dei pod né con lo storico. Se vuoi, posso verificare `requests` e `limits` di `worker-app` e guardarne il trend su MongoDB. Così capiamo se 0,58 core è un carico stabile o un picco, e se le repliche sono dimensionate bene.

GPT-4o Mini	35.87 s	3073	374	3447	0.43	Dall'analisi dei dati, ecco il consumo di CPU per ciascuna applicazione monitorata:
						
						1. **cache-app**
						   - Pod: `cache-app-5c6fd4f5cf-xxsz4`
						   - Rate di CPU: **0.0308 core**
						
						2. **demo-app**
						   - Pod: `demo-app-7958c64697-2qg9s`: **0 core**
						   - Pod: `demo-app-7958c64697-4rzjl`: **0 core**
						   - Rate di CPU totale: **0 core**
						
						3. **worker-app**
						   - Pod: `worker-app-84cf4dd446-9kj75`: **0.5876 core**
						   - Pod: `worker-app-84cf4dd446-ftlln`: **0.5765 core**
						   - Rate di CPU totale: **1.1641 core**
						
						### Risultato finale:
						L'applicazione che assorbe il maggior rate di CPU è **worker-app**, con un totale di **1.1641 core** utilizzati.

GPT-4o	34.78 s	69156	103	69259	0.18	L'applicazione che assorbe il maggior rate di CPU è "worker-app", con un totale di circa 1.176 core (0.5877 + 0.5887) impiegati complessivamente dai suoi pod.

DOMANDA 1 A



Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	5.48 ms	0	0	0	N/A	worker-app (0.2258 cores)
Gemini 2.5 Flash	71.15 s	4810	4051	8861	0.18	Il servizio che presenta il consumo di CPU più elevato è **worker-app**, con un utilizzo totale di **1.1647532205000000 core**.
Gemini 3.6 Flash	30.76 s	3653	1257	4910	0.43	Il servizio che presenta il consumo di CPU più elevato nel cluster è **`worker-app`**. ### Dettaglio del consumo di CPU per istanza (Pod): * **`worker-app-84cf4dd446-9kj75`**: ~0.285 core (284.91 mCPU) * **`worker-app-84cf4dd446-ftlln`**: ~0.280 core (279.55 mCPU) --- ### Consumo Totale: Sommando tutte le istanze attive, **`worker-app`** consuma un totale di circa **0.564 core** (ovvero ***564.46 mCPU**). --- ### Confronto con gli altri servizi dell'applicazione (`namespace: default`): * **`cache-app`**: ~0.005 core (5.10 mCPU) * **`demo-app`**: 0.000 core
Claude Opus 5.5	44.19 s	8675	1269	9944	0.43	Il servizio che usa più CPU è **`worker-app`** (namespace `default`). Le sue 2 istanze usano in tutto circa **0,56–0,59 core**. ### Consumo per istanza Media degli ultimi 5 minuti, misurata con `rate(container_cpu_usage_seconds_total[5m])`: | Pod | CPU (core) | |---|---| | worker-app-84cf4dd446-ftlln | ~0,280 – 0,294 | | worker-app-84cf4dd446-9kj75 | ~0,285 – 0,294 | | **Totale worker-app** | **~0,565 – 0,588 core** (circa 565–590 millicore) | Ho fatto due rilevazioni a pochi secondi di distanza: 0,5645 core nella prima e 0,5884 nella seconda. Il carico è stabile e diviso quasi in parti uguali tra i due pod. ### Confronto con gli altri servizi - **demo-app** (2 pod): 0 core, cioè di fatto inattivo. - **cache-app** (1 pod): circa 0,005 core. - Fuori dal namespace `default`, il pod che consuma di più è `kube-apiserver`, con circa 0,108 core. Ogni pod di `worker-app` da solo consuma quasi tre volte tanto.

Claude Opus 5.5	44.19 s	8675	1269	9944	0.43	Il servizio che usa più CPU è **`worker-app`** (namespace `default`). Le sue 2 istanze usano in tutto circa **0,56–0,59 core**. ### Consumo per istanza Media degli ultimi 5 minuti, misurata con `rate(container_cpu_usage_seconds_total[5m])`: | Pod | CPU (core) | |---|---| | worker-app-84cf4dd446-ftlln | ~0,280 – 0,294 | | worker-app-84cf4dd446-9kj75 | ~0,285 – 0,294 | | **Totale worker-app** | **~0,565 – 0,588 core** (circa 565–590 millicore) | Ho fatto due rilevazioni a pochi secondi di distanza: 0,5645 core nella prima e 0,5884 nella seconda. Il carico è stabile e diviso quasi in parti uguali tra i due pod. ### Confronto con gli altri servizi - **demo-app** (2 pod): 0 core, cioè di fatto inattivo. - **cache-app** (1 pod): circa 0,005 core. - Fuori dal namespace `default`, il pod che consuma di più è `kube-apiserver`, con circa 0,108 core. Ogni pod di `worker-app` da solo consuma quasi tre volte tanto. `worker-app` usa da solo circa il **99% della CPU delle applicazioni** in `default`. Rispetto a tutti i pod del cluster resta di gran lunga il primo. ### Raccomandazioni 1. **Controllare requests e limits** di `worker-app`. Con circa 0,29 core per pod, se il limit fosse 300m o 500m il servizio sarebbe vicino al throttling. Si può verificare con `container_cpu_cfs_throttled_periods_total`. 2. **Valutare l'autoscaling orizzontale (HPA)** se il carico cresce. Il consumo è già diviso in modo uniforme, quindi aggiungere repliche funzionerebbe bene. 3. **Ridurre le risorse di `demo-app`**: con 2 pod a consumo zero, si potrebbe passare a 1 replica o abbassare le requests per liberare capacità schedulabile. 4. **Guardare l'andamento nel tempo**: questi valori sono una fotografia degli ultimi 5 minuti. Prima di decidere sul dimensionamento, conviene confrontarli con lo storico su MongoDB (`get_historical_metrics`) o con una query su un intervallo di ore o giorni (`prom_range`). Se serve, posso salvare questa rilevazione su MongoDB per gli storici.
Claude Sonnet 5.5	31.40 s	6003	790	6793	0.45	**Il servizio con il maggior consumo di CPU è `worker-app`, con circa 0,564 core in totale.** Ho misurato il consumo come `rate(container_cpu_usage_seconds_total[5m])`, cioè la media degli ultimi 5 minuti. | Pod | CPU (core) | |---|---| | `worker-app-84cf4dd446-9kj75` | 0,2849 | | `worker-app-84cf4dd446-ftlln` | 0,2795 | | **Totale `worker-app` (2 repliche)** | **≈ 0,5645** | Gli altri servizi consumano molto meno: - **Applicativi:** `cache-app` usa circa 0,005 core con 1 pod. `demo-app` è a 0 core con 2 pod. - **Componenti di sistema:** il più alto è `kube-apiserver`, con circa 0,116 core. Seguono Grafana (0,053) e Prometheus (0,053). `worker-app` usa circa 5 volte la CPU del componente di sistema più pesante e circa 100 volte quella di `cache-app`. I due pod di `worker-app` sono bilanciati, perché consumano quasi lo stesso. Il carico quindi non dipende da una singola replica anomala. Prima di cambiare le risorse, ti consiglio di confrontare questi valori con i `requests` e i `limits` CPU del Deployment. Conviene anche guardare lo storico delle metriche per vedere se il consumo è stabile o ha picchi. Se è stabile vicino ai limiti, valuta un HPA o un aumento dei limiti.

GPT-4o Mini	37.44 s	3098	367	3465	0.43	Ecco i risultati del consumo di CPU per ciascun servizio nel cluster:
						
						1. **cache-app**
						   - Pod: `cache-app-5c6fd4f5cf-xxsz4`
						   - Consumo totale: **0.0103 core**
						
						2. **demo-app**
						   - Pod: `demo-app-7958c64697-2qg9s`: **0 core**
						   - Pod: `demo-app-7958c64697-4rzjl`: **0 core**
						   - Consumo totale: **0 core**
						
						3. **worker-app**
						   - Pod: `worker-app-84cf4dd446-9kj75`: **0.5720 core**
						   - Pod: `worker-app-84cf4dd446-ftlln`: **0.5783 core**
						   - Consumo totale: **1.1503 core**
						
						### Risultato finale
						Il servizio con il consumo di CPU più elevato è **worker-app**, con un totale di **1.1503 core** utilizzati da tutte le sue istanze.

						al limiti, valuta un HPA o un aumento dei limiti.

GPT-4o Mini	37.44 s	3098	367	3465	0.43	Ecco i risultati del consumo di CPU per ciascun servizio nel cluster:
						
						1. **cache-app**
						   - Pod: `cache-app-5c6fd4f5cf-xxsz4`
						   - Consumo totale: **0.0103 core**
						
						2. **demo-app**
						   - Pod: `demo-app-7958c64697-2qg9s`: **0 core**
						   - Pod: `demo-app-7958c64697-4rzjl`: **0 core**
						   - Consumo totale: **0 core**
						
						3. **worker-app**
						   - Pod: `worker-app-84cf4dd446-9kj75`: **0.5720 core**
						   - Pod: `worker-app-84cf4dd446-ftlln`: **0.5783 core**
						   - Consumo totale: **1.1503 core**
						
						### Risultato finale
						Il servizio con il consumo di CPU più elevato è **worker-app**, con un totale di **1.1503 core** utilizzati da tutte le sue istanze.

GPT-4o	35.93 s	3022	297	3319	0.30	Dai dati raccolti, il servizio con il consumo di CPU più elevato è il "worker-app", con un totale di circa 1.15 core utilizzati sommando tutte le sue istanze. Ecco il dettaglio del consumo di CPU per ciascun servizio: - **cache-app**: 0.0103 core - **demo-app**: 0 core - **worker-app**: 1.15 core Se hai bisogno di ulteriori analisi o dettagli, fammi sapere!

DOMANDA 1 B

```
Scegli una domanda (es. 1, 1a, 1b ... fino a 10, 10a, 10b) oppure premi INVIO per uscire: 1b

--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [1B]: Dall'analisi delle metriche di utilizzo del processore, quale applicazione risulta essere il top consumer di cicli CPU e qual è il suo tasso di assorbimento complessivo in core?
--------------------------------------------------------------------------------------------------------------
Invocazione parallela modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
[10/29/26 11:45:08] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/29/26 11:45:08] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/29/26 11:45:08] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/29/26 11:45:08] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
[10/29/26 11:45:29] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
```

DOMANDA2

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [2]: Qual è l'uso effettivo di memoria Working Set (in MB) dell'applicazione con il maggior numero di pod o repliche?
--------------------------------------------------------------------------------------------------------------
Invocazione parallela modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/29/26 16:25:06] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/29/26 16:25:06] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/29/26 16:25:06] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/29/26 16:25:06] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/29/26 16:25:07] INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
[10/29/26 16:25:07] INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
[10/29/26 16:25:07] INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
[10/29/26 16:25:07] INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
[10/29/26 16:25:07] INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
[10/29/26 16:25:11] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/29/26 16:25:12] INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.08 ms	0	0	0	N/A	demo-app: 17.31 MB
Gemini 2.5 Flash	50.24 s	5097	1832	6929	0.28	Le applicazioni con il maggior numero di pod (2 repliche ciascuna) sono `demo-app` e `worker-app`. L'uso effettivo di memoria Working Set per `demo-app` è di circa 10.03 MB. L'uso effettivo di memoria Working Set per `worker-app` è di circa 3.84 MB. Considerando l'applicazione con il maggior numero di pod e il maggiore utilizzo di memoria Working Set tra queste, l'applicazione è `demo-app` con un uso effettivo di memoria Working Set di circa 10.03 MB.

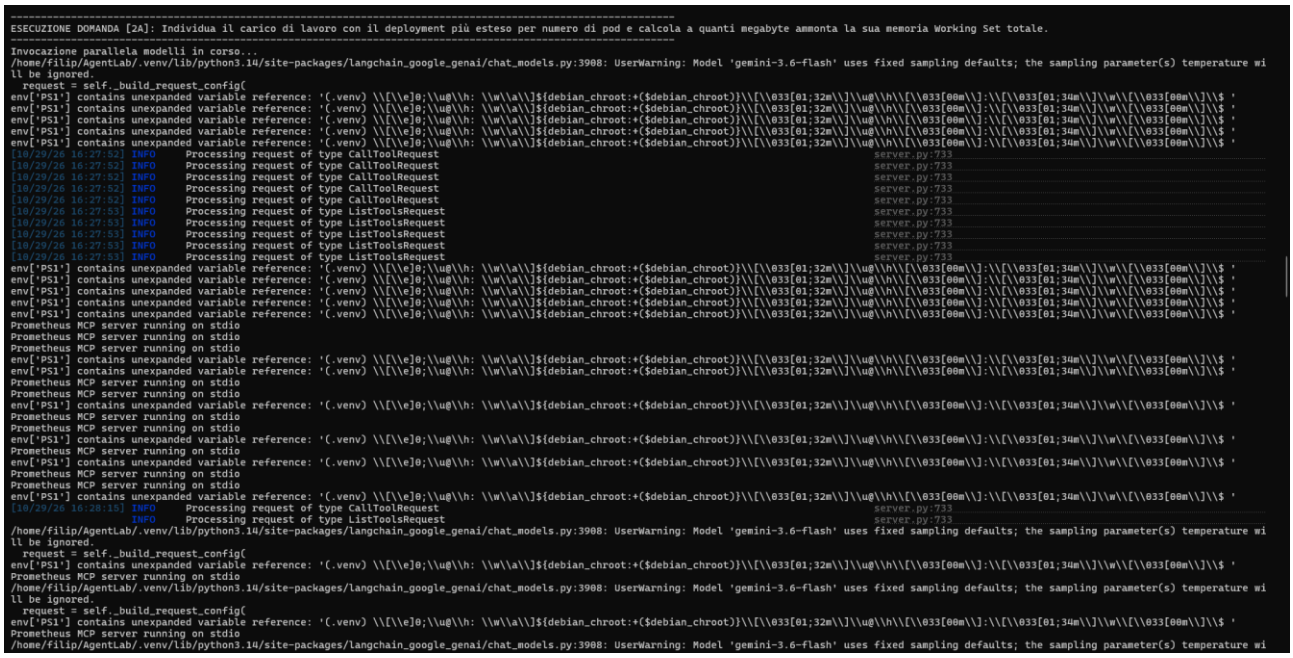
Gemini 3.6 Flash	71.45 s	47582	3379	50961	0.72	Dall'analisi della topologia dei carichi di lavoro del cluster nel namespace `default`, le applicazioni con il maggior numero di pod/repliche attive sono **`demo-app`** e **`worker-app`**, entrambe con **2 repliche/pod attivi**.
						
						Di seguito è riportato l'uso effettivo di memoria **Working Set** (`container_memory_working_set_bytes`) per ciascuna di queste due applicazioni:
						
						--- 	
						
						### 1. `demo-app` (2 Repliche)
						
						* **Pod `demo-app-7958c64697-2qg9s`**:
						  * **Bytes**: `7.847.936` bytes
						  * **In MiB (base 1024)**: ~**7,48 MiB**
						  * **In MB (base 1000)**: ~**7,85 MB**
						* **Pod `demo-app-7958c64697-4rzjl`**:
						  * **Bytes**: `8.052.736` bytes
						  * **In MiB (base 1024)**: ~**7,68 MiB**
						  * **In MB (base 1000)**: ~**8,05 MB**
						
						* **Totale per `demo-app`**:
						  * **Bytes complessivi**: `15.900.672` bytes
						  * **Totale in MiB**: **15,16 MiB**
						  * **Totale in MB**: **15,90 MB**
						
						--- 	
						
						### 2. `worker-app` (2 Repliche)
						
						* **Pod `worker-app-84cf4dd446-9kj75`**:
						  * **Bytes**: `4.300.800` bytes
						  * **In MiB (base 1024)**: ~**4,10 MiB**
						  * **In MB (base 1000)**: ~**4,30 MB**
						* **Pod `worker-app-84cf4dd446-ftlln`**:
						  * **Bytes**: `1.273.856` bytes

						### 2. `worker-app` (2 Repliche)
						
						* **Pod `worker-app-84cf4dd446-9kj75`**:
						  * **Bytes**: `4.300.800` bytes
						  * **In MiB (base 1024)**: ~**4,10 MiB**
						  * **In MB (base 1000)**: ~**4,30 MB**
						* **Pod `worker-app-84cf4dd446-ftlln`**:
						  * **Bytes**: `1.273.856` bytes
						  * **In MiB (base 1024)**: ~**1,21 MiB**
						  * **In MB (base 1000)**: ~**1,27 MB**
						
						* **Totale per `worker-app`**:
						  * **Bytes complessivi**: `5.574.656` bytes
						  * **Totale in MiB**: **5,32 MiB**
						  * **Totale in MB**: **5,57 MB**
						
						--- 	
						
						### Summary
						* L'applicazione con il consumo di memoria Working Set più elevato tra quelle a maggior numero di repliche è **`demo-app`**, con un totale di **15,90 MB** (15,16 MiB) complessivi su 2 pod.

Claude Opus 5.5	49.05 s	6985	1065	8050	0.36	**Non c'è un'unica applicazione con più pod: due sono a pari merito.** demo-app e worker-app hanno 2 pod ciascuna, cache-app ne ha 1. Ecco il Working Set attuale di entrambe (dati Prometheus, `container_memory_working_set_bytes`, 1 MB = 1024² byte): | Applicazione | Pod | Working Set | |---|---|---| | **demo-app** | demo-app-7958c64697-2qg9s | 7,23 MB | | | demo-app-7958c64697-4rzjl | 7,32 MB | | | **Totale** | **≈ 14,55 MB** | | **worker-app** | worker-app-84cf4dd446-9kj75 | 3,99 MB | | | worker-app-84cf4dd446-ftlln | 0,97 MB | | | **Totale** | **≈ 4,96 MB** | **Osservazioni:** - **demo-app** usa la memoria in modo simile su entrambi i pod, quindi il carico sembra ben distribuito. - **worker-app** è sbilanciata: un pod usa circa 4 volte più memoria dell'altro. Può dipendere dal modo in cui il lavoro viene assegnato ai worker (alcuni lavorano più di altri) oppure dal fatto che un pod è fermo o sta appena partendo. Conviene controllare i log e l'età del pod `worker-app-84cf4dd446-ftlln`. - I consumi sono comunque molto bassi (meno di 15 MB per app). Se le richieste di memoria (`requests`) di questi deployment sono molto più alte, si potrebbero abbassare (rightsizing). Prima però conviene guardare lo storico dei consumi, non solo il valore di adesso. Se mi dici con quale criterio scegliere tra le due (per esempio quella che consuma di più, cioè demo-app), posso salvare le metriche su MongoDB o analizzarne l'andamento nel tempo.
Claude Sonnet 5.5	46.65 s	6984	1065	8049	0.68	**Non c'è un'unica applicazione con più repliche.** `demo-app` e `worker-app` hanno entrambe 2 pod, mentre `cache-app` ne ha 1. Ti riporto il working set di tutte e due. Il dato viene da `container_memory_working_set_bytes` (namespace `default`, escluso il container pause), letto in un'unica interrogazione istantanea. | App | Pod | Working Set per pod | Totale app | |---|---|---|---| | demo-app | `demo-app-7958c64697-2qg9s` | 7.581.696 B ≈ 7,23 MiB | 15.253.504 B ≈ 14,55 MiB (≈ 15,25 MB decimali) | | demo-app | `demo-app-7958c64697-4rzjl` | 7.671.808 B ≈ 7,32 MiB | | | worker-app | `worker-app-84cf4dd446-9kj75` | 4.186.112 B ≈ 3,99 MiB | 5.206.016 B ≈ 4,96 MiB (≈ 5,21 MB decimali) | | worker-app | `worker-app-84cf4dd446-ftlln` | 1.019.904 B ≈ 0,97 MiB | | - **`demo-app`:** usa circa 14,55 MiB in totale, con i due pod molto simili tra loro (circa 7,3 MiB ciascuno). - **`worker-app`:** usa circa 4,96 MiB in totale. I due pod sono sbilanciati: uno ha circa 4 MiB e l'altro circa 1 MiB. - **Quale scegliere:** se ti serve una sola risposta, `demo-app` è la più significativa. Ha il working set più alto tra le app a parità di repliche, circa **14,55 MB** (MiB). I valori sono una singola misura istantanea, non una media nel tempo. Se vuoi, posso guardare lo storico su MongoDB o un intervallo Prometheus per vedere il trend.
GPT-4o Mini	41.61 s	3774	180	3954	0.28	L'applicazione con il maggior numero di pod è la **demo-app**, che ha 2 pod attivi. Ecco l'uso effettivo di memoria Working Set per ciascun pod della demo-app: - **demo-app-7958c64697-2qg9s**: 15,634,432 bytes (circa 14.92 MB) - **demo-app-7958c64697-4rzjl**: 15,925,248 bytes (circa 15.19 MB) Pertanto, l'uso effettivo di memoria Working Set totale per la **demo-app** è di circa **30.11 MB**.
GPT-4o	47.05 s	2958	194	3152	0.20	L'applicazione con il maggior numero di pod è la "demo-app", che ha 2 pod attivi. L'uso effettivo di memoria Working Set per questa applicazione è di circa 30.10 MB.

[BILLING MONITOR] Spesa cumulata OpenAI: $0.4097 (mini: $0.0112 | 4o: $0.3985)

DOMANDA 2A



Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	3.06 ms	0	0	0	N/A	demo-app: 17.31 MB
Gemini 2.5 Flash	28.79 s	5438	1286	6724	0.63	Il carico di lavoro con il deployment più esteso per numero di pod è `demo-app`, con 2 pod. La memoria Working Set totale per il deployment `demo-app` è di circa 14.99 MB. (Pod `demo-app-7958c64697-2qg9s`: 7.46 MB, Pod `demo-app-7958c64697-4rzjl`: 7.53 MB)

Gemini 3.6 Flash	65.33 s	47606	4186	51792	0.00	Dall'analisi dello stato del cluster e delle metriche di Prometheus, è emersa un'**ambiguità nei dati**: non esiste un singolo Deployment che sia strettamente il "più esteso" per numero di pod, poiché vi è una parità al primo posto tra più workload.
						
						### Analisi dei Deployment per numero di Pod
						
						1. **`demo-app`** (namespace: `default`): **2 pod**
						  - `demo-app-7958c64697-2qg9s`
						  - `demo-app-7958c64697-4rzjl`
						2. **`worker-app`** (namespace: `default`): **2 pod**
						  - `worker-app-84cf4dd446-9kj75`
						  - `worker-app-84cf4dd446-ftlln`
						3. **`coredns`** (namespace: `kube-system`): **2 pod**
						  - `coredns-76f75df574-4vpgj`
						  - `coredns-76f75df574-hct7b`
						4. **`cache-app`** (namespace: `default`): **1 pod**
						
						--- 	
						
						### Calcolo della memoria Working Set totale per i workload candidati
						
						Poiché tre Deployment condividono il numero massimo di pod (2 pod ciascuno), di seguito riporto il calcolo dettagliato della memoria Working Set totale per ciascuno di essi:
						
						* **`demo-app`**:
						  * Pod `demo-app-7958c64697-2qg9s`: 7.823.360 byte (~7,82 MB)
						  * Pod `demo-app-7958c64697-4rzjl`: 7.892.992 byte (~7,89 MB)
						  * **Totale Working Set**: **15.716.352 byte ≈ 15,72 MB** (15,00 MiB)
						
						* **`worker-app`**:
						  * Pod `worker-app-84cf4dd446-9kj75`: 4.440.064 byte (~4,44 MB)
						  * Pod `worker-app-84cf4dd446-ftlln`: 1.253.376 byte (~1,25 MB)
						  * **Totale Working Set**: **5.693.440 byte ≈ 5,69 MB** (5,43 MiB)
						
						* **`coredns`** (workload di sistema):
						  * Pod `coredns-76f75df574-4vpgj`: 33.296.384 byte (~33,30 MB)

						### Raccomandazione Data la presenza di una **parità (tie)** tra più Deployment con 2 pod ciascuno (`demo-app`, `worker-app` e `coredns`), non è possibile determinare in maniera univoca ed esclusiva un singolo carico di lavoro senza un criterio di disambiguazione aggiuntivo (ad es. restringere ai soli applicativi applicativi utente o selezionare quello con consumo di risorse maggiore/minore). Si consiglia esplicitamente all'operatore di **sottomettere la richiesta a un modello più avanzato** specificando eventuali criteri di preferenza o di disambiguazione.
Claude Opus 5.5	32.13 s	7012	1182	8194	0.86	Nessun deployment ha più pod degli altri: **demo-app** e **worker-app** ne hanno 2 ciascuno, **cache-app** solo 1. Riporto quindi la memoria Working Set di entrambi. **Pod per deployment (namespace `default`)** | Deployment | Pod attivi | |---|---| | demo-app | 2 | | worker-app | 2 | | cache-app | 1 | **Memoria Working Set totale** (somma dei container, esclusi i container `POD` di sistema) | Deployment | Pod | Working Set (byte) | |---|---|---| | **demo-app** | demo-app-7958c64697-2qg9s | 7.577.600 | | | demo-app-7958c64697-4rzjl | 7.655.424 | | | **Totale** | **15.233.024 B ≈ 14,53 MiB (15,23 MB)** | | **worker-app** | worker-app-84cf4dd446-9kj75 | 4.485.120 | | | worker-app-84cf4dd446-ftlln | 1.236.992 | | | **Totale** | **5.722.112 B ≈ 5,46 MiB (5,72 MB)** | Unità: MiB = byte / 1.048.576; MB = byte / 1.000.000. **Osservazioni e suggerimenti** - Con lo stesso numero di pod, **demo-app** usa circa 2,7 volte la memoria di worker-app. Se serve un solo vincitore, conviene scegliere demo-app come carico più pesante. - In **worker-app** un pod usa molta più memoria dell'altro (circa 4,3 MiB contro 1,2 MiB). Potrebbe dipendere da una distribuzione disomogenea del lavoro o da uno stato interno diverso tra i pod. Vale la pena controllare come vengono distribuite le richieste o i job. - I consumi sono molto bassi in valore assoluto. Conviene confrontarli con le `requests`/`limits` di memoria dei deployment per capire se le risorse riservate sono troppe e si possono ridurre. Se vuoi, posso salvare queste misure su MongoDB (`save_metrics_to_mongodb`) per seguirne l'andamento nel tempo.

						ridurre. Se vuoi, posso salvare queste misure su MongoDB (`save_metrics_to_mongodb`) per seguirne l'andamento nel tempo.
Claude Sonnet 5.5	29.42 s	7035	1140	8175	0.86	**Il deployment più esteso non è uno solo: c'è un pareggio.** `demo-app` e `worker-app` hanno entrambi 2 pod attivi, mentre `cache-app` ne ha 1. Ti riporto la memoria Working Set (`container_memory_working_set_bytes`) di entrambi. | Deployment | Pod | Working Set per pod | Totale | |---|---|---|---| | **demo-app** | `demo-app-7958c64697-2qg9s` | 7.577.600 B | **15.233.024 B ≈ 14,53 MiB** (≈ 15,23 MB decimali) | | | `demo-app-7958c64697-4rzjl` | 7.655.424 B | | | **worker-app** | `worker-app-84cf4dd446-9kj75` | 4.485.120 B | **5.722.112 B ≈ 5,46 MiB** (≈ 5,72 MB decimali) | | | `worker-app-84cf4dd446-ftlln` | 1.236.992 B | | Per confronto, `cache-app` (1 pod) usa 1.269.760 B, cioè circa 1,21 MiB. - **Se serve un solo carico di lavoro:** `demo-app` è quello con la memoria totale maggiore tra i due a pari pod, con circa 14,53 MiB (15,23 MB decimali). Questa scelta è un criterio di spareggio mio, perché la richiesta non ne indicava uno. - **Unità di misura:** i valori in MiB dividono per 1.048.576, quelli in MB decimali dividono per 10⁶. - **Come ho calcolato:** ho sommato i Working Set dei container di ciascun pod, escludendo il container `POD` (pause). Il valore è un'istantanea del momento della query. - **Squilibrio in `worker-app`:** i suoi due pod usano memoria molto diversa, 4,3 MiB contro 1,2 MiB. Può essere normale, ma conviene controllare come viene distribuito il carico tra le repliche.
GPT-4o Mini	30.30 s	2934	242	3176	0.28	Ho identificato i carichi di lavoro e calcolato la memoria Working Set totale per ciascun deployment: 1. **cache-app**: 2,813,952 bytes (circa 2.68 MB) 2. **demo-app**: 31,678,464 bytes (circa 30.19 MB) 3. **worker-app**: 12,075,008 bytes (circa 11.52 MB) Il deployment con il numero maggiore di pod è **demo-app**, che ha 2 pod attivi. La sua memoria Working Set totale è di circa **30.19 MB**.
GPT-4o	29.57 s	2997	231	3228	0.15	Il deployment con il maggior numero di pod è "demo-app", che ha 2 pod attivi. La memoria Working Set totale per questo deployment ammonta a 31,350,784 byte, che corrispondono a circa 29.9 MB.

[BILLING MONITOR] Spesa cumulata OpenAI: $0.4200 (mini: $0.0118 | 4o: $0.4083)

DOMANDA 2B

```
--------------------------------------------------------------------------------
ESECUZIONE DOMANDA [2B]: Considerando le applicazioni che contano il maggior numero di repliche attive, a quanto corrisponde il loro consumo aggregato di Working Set espresso in MB?
--------------------------------------------------------------------------------
Invocazione parallela modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/29/26 16:30:14] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/29/26 16:30:14] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/29/26 16:30:16] INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
[10/29/26 16:30:16] INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
[10/29/26 16:30:16] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/29/26 16:30:17] INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
[10/29/26 16:30:17] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/29/26 16:30:17] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/29/26 16:30:18] INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
[10/29/26 16:30:18] INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
[10/29/26 16:30:19] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/29/26 16:30:20] INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
Prometheus MCP server running on stdio
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.43 ms	0	0	0	N/A	demo-app: 17.31 MB
Gemini 2.5 Flash	22.96 s	3470	665	4135	0.00	Il consumo aggregato di Working Set per le applicazioni con il maggior numero di repliche attive (`demo-app` e `worker-app`) è di 39.82 MB.

Gemini 3.6 Flash	38.65 s	24263	3496	27759	0.00	Sulla base delle verifiche effettuate tramite la mappa dei Pod e le metriche restituite da Prometheus per il Working Set (`container_memory_working_set_bytes`), di seguito si riporta l'analisi dettagliata.
						
						--- 	
						
						### 1. Identificazione delle applicazioni con il maggior numero di repliche attive
						
						Dall'analisi della mappa dei Deployment/Pod attivi:
						* **`demo-app`**: **2 repliche** (`demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl`)
						* **`worker-app`**: **2 repliche** (`worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln`)
						* **`cache-app`**: **1 replica** (`cache-app-5c6fd4f5cf-xxsz4`)
						
						Il numero massimo di repliche attive è **2**. Le applicazioni a pari merito con il maggior numero di repliche sono **`demo-app`** e **`worker-app`**.
						
						--- 	
						
						### 2. Consumo di Working Set per singolo Pod
						
						#### **A. Applicazione `demo-app`**
						* **`demo-app-7958c64697-2qg9s`**: `7.823.360 byte`
						* **`demo-app-7958c64697-4rzjl`**: `7.892.992 byte`
						* **Totale `demo-app`**: **15.716.352 byte**
						  * *In MiB (base 1024)*: **~14,99 MiB**
						  * *In MB (base 1000)*: **~15,72 MB**
						
						#### **B. Applicazione `worker-app`**
						* **`worker-app-84cf4dd446-9kj75`**: `3.997.696 byte`
						* **`worker-app-84cf4dd446-ftlln`**: `1.245.184 byte`
						* **Totale `worker-app`**: **5.242.880 byte**
						  * *In MiB (base 1024)*: **5,00 MiB**
						  * *In MB (base 1000)*: **~5,24 MB**
						
						--- 	
						
						### 3. Consumo Aggregato Complessivo
						
						Sommando i consumi di tutte le repliche di `demo-app` e `worker-app`:
						
						$$\text{Totale aggregato} = 15.716.352 \text{ byte} + 5.242.880

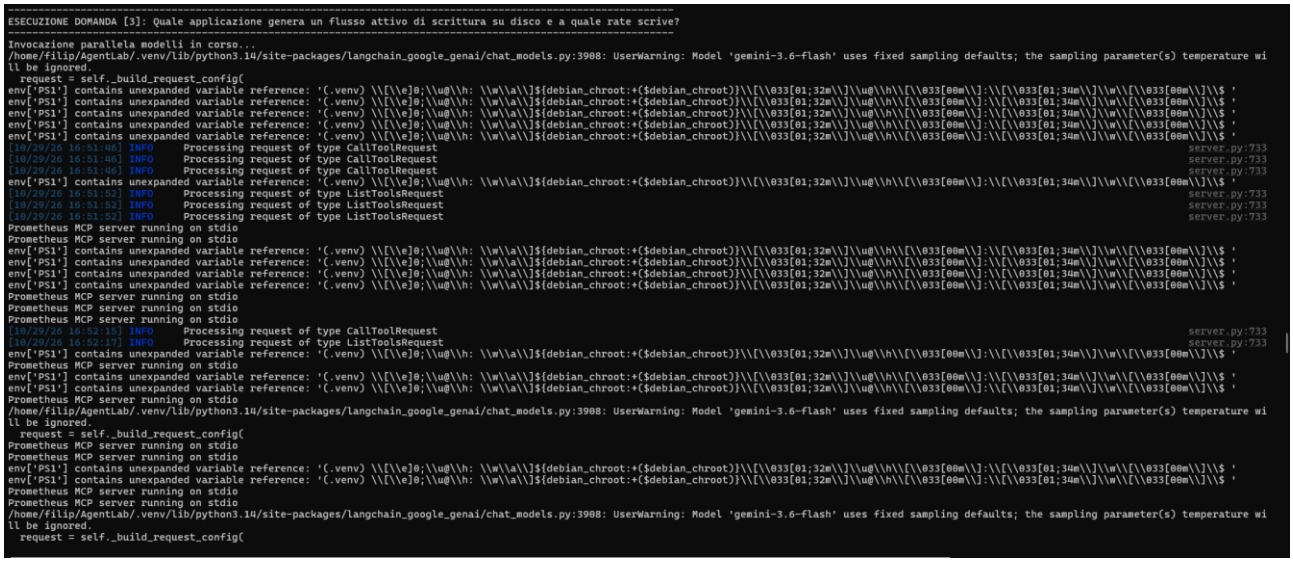
						--- ### 3. Consumo Aggregato Complessivo Sommando i consumi di tutte le repliche di `demo-app` e `worker-app`: $$\text{Totale aggregato} = 15.716.352 \text{ byte} + 5.242.880 \text{ byte} = \mathbf{20.959.232 \text{ byte}}$$ * **In Megabyte (MB, base 1000)**: **20,96 MB** * **In Mebibyte (MiB, base 1024)**: **19,99 MiB** (circa **20 MiB**)
Claude Opus 5.5	31.45 s	6967	1174	8141	0.00	**Risposta: le applicazioni con più repliche (`demo-app` e `worker-app`) consumano insieme circa 19,05 MB di Working Set** (19.976.192 byte; sono circa 19,98 MB se si usa 1 MB = 1.000.000 byte). **Repliche per applicazione (namespace `default`)** | Deployment | Repliche attive | |---|---| | demo-app | 2 | | worker-app | 2 | | cache-app | 1 | `demo-app` e `worker-app` hanno lo stesso numero di repliche, il più alto, quindi le ho considerate entrambe. **Working Set misurato in questo momento** (metrica `container_memory_working_set_bytes`, escludendo il container "pause"): | Applicazione | Pod | Working Set | |---|---|---| | demo-app | demo-app-7958c64697-2qg9s | 7,23 MB | | demo-app | demo-app-7958c64697-4rzjl | 7,29 MB | | **demo-app (totale)** | | **14,52 MB** | | worker-app | worker-app-84cf4dd446-9kj75 | 3,59 MB | | worker-app | worker-app-84cf4dd446-ftlln | 0,94 MB | | **worker-app (totale)** | | **4,53 MB** | | **Totale** | | **19,05 MB** | **Osservazioni** - I consumi di memoria sono molto bassi. Se le richieste di memoria dei pod sono molto più alte, conviene ridurle: oggi stanno solo occupando capacità del cluster. - Le due repliche di `worker-app` usano quantità di memoria diverse (3,6 MB contro 0,9 MB). Il carico potrebbe essere ripartito male tra le due repliche, oppure una delle due potrebbe essere quasi ferma. Vale la pena controllare. - Questi valori sono una misura istantanea. Prima di cambiare le impostazioni dei pod conviene guardare l'andamento storico, ad esempio con `get_historical_metrics` sulle ultime 24 ore.
Claude Sonnet 5.5	28.49 s	6982	1088	8070	0.00	Le applicazioni con più repliche attive sono **demo-app** e **worker-app**, con 2 repliche ciascuna. cache-app ne ha 1. Il loro consumo aggregato di Working Set è di circa **19,05 MiB**, cioè circa 19,98 MB decimali. Ho letto `container_memory_working_set_bytes` per ogni pod, escludendo i container `POD` e quelli senza nome. | Applicazione | Pod | Working Set (byte) | |---|---|---| | demo-app | demo-app-7958c64697-2qg9s | 7.577.600 | | demo-app | demo-app-7958c64697-4rzjl | 7.647.232 | | worker-app | worker-app-84cf4dd446-9kj75 | 3.764.224 | | worker-app | worker-app-84cf4dd446-ftlln | 987.136 | **Totali per applicazione** - demo-app: 15.224.832 byte, cioè circa 14,52 MiB (15,22 MB decimali). - worker-app: 4.751.360 byte, cioè circa 4,53 MiB (4,75 MB decimali). **Totale aggregato** - 19.976.192 byte, cioè circa 19,05 MiB (19,98 MB decimali). Il valore in MB cambia a seconda dell'unità: Kubernetes usa 1 MiB = 1.048.576 byte, quindi per i confronti con requests e limits conviene il valore in MiB. Nel worker-app i due pod sono molto sbilanciati: uno usa circa 3,6 MiB e l'altro circa 0,94 MiB. Può dipendere da una distribuzione del carico non uniforme tra le repliche, ma questi dati non bastano per dirlo. Se vuoi, posso guardare lo storico delle metriche per verificarlo.
GPT-4o Mini	29.96 s	2908	312	3220	0.00	Non sono riuscito a recuperare informazioni sul consumo di memoria aggregato (Working Set) per le applicazioni attive nel cluster. Le query per ottenere i dati sulle richieste di memoria per i pod delle applicazioni "cache-app", "demo-app" e "worker-app" non hanno restituito risultati. Questo potrebbe essere dovuto a diverse ragioni, come la mancanza di metriche disponibili per i pod o la configurazione delle risorse. Ti consiglio di verificare se i pod hanno effettivamente richieste di memoria definite e se il sistema di monitoraggio sta raccogliendo correttamente le metriche. Se il problema persiste, potrebbe essere utile sottomettere la richiesta a un modello più avanzato per un'analisi più approfondita.

						decimali). **Totale aggregato** - 19.976.192 byte, cioè circa 19,05 MiB (19,98 MB decimali). Il valore in MB cambia a seconda dell'unità: Kubernetes usa 1 MiB = 1.048.576 byte, quindi per i confronti con requests e limits conviene il valore in MiB. Nel worker-app i due pod sono molto sbilanciati: uno usa circa 3,6 MiB e l'altro circa 0,94 MiB. Può dipendere da una distribuzione del carico non uniforme tra le repliche, ma questi dati non bastano per dirlo. Se vuoi, posso guardare lo storico delle metriche per verificarlo.
GPT-4o Mini	29.96 s	2908	312	3220	0.00	Non sono riuscito a recuperare informazioni sul consumo di memoria aggregato (Working Set) per le applicazioni attive nel cluster. Le query per ottenere i dati sulle richieste di memoria per i pod delle applicazioni "cache-app", "demo-app" e "worker-app" non hanno restituito risultati. Questo potrebbe essere dovuto a diverse ragioni, come la mancanza di metriche disponibili per i pod o la configurazione delle risorse. Ti consiglio di verificare se i pod hanno effettivamente richieste di memoria definite e se il sistema di monitoraggio sta raccogliendo correttamente le metriche. Se il problema persiste, potrebbe essere utile sottomettere la richiesta a un modello più avanzato per un'analisi più approfondita.

GPT-4o	23.87 s	3099	340	3439	0.00	Le applicazioni con il maggior numero di repliche attive sono "demo-app" e "worker-app", entrambe con 2 repliche ciascuna. Il consumo aggregato di Working Set per queste applicazioni, espresso in MB, è il seguente:
						
						- **demo-app**:
						  - Pod "demo-app-7958c64697-2qg9s": 15.6 MB
						  - Pod "demo-app-7958c64697-4rzjl": 15.7 MB
						  - Totale demo-app: 31.3 MB
						
						- **worker-app**:
						  - Pod "worker-app-84cf4dd446-9kj75": 7.9 MB
						  - Pod "worker-app-84cf4dd446-ftlln": 2.4 MB
						  - Totale worker-app: 10.3 MB
						
						Il consumo aggregato totale di Working Set per entrambe le applicazioni è di 41.6 MB.

[BILLING MONITOR] Spesa cumulata OpenAI: $0.4318 (mini: $0.0124 | 4o: $0.4194)

DOMANDA 3



Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	6.01 ms	0	0	0	N/A	cache-app (494183.96 B/s)
Gemini 2.5 Flash	35.52 s	17557	539	18096	0.63	Analizzando i dati di scrittura su disco, ho identificato le seguenti applicazioni con un flusso attivo di scrittura: * **cache-app**: 513283.28 B/s * **prometheus-grafana**: 709.92 B/s * **etcd**: 33787.00 B/s * **prometheus-prometheus-kube-prometheus-prometheus**: 7418.96 B/s L'applicazione `cache-app` è quella che genera il flusso di scrittura su disco più elevato, con un rate di circa 513 KB/s.
Gemini 3.6 Flash	70.59 s	75374	1058	76432	0.72	Dall'analisi delle metriche di I/O del filesystem ricavate dal cluster (tramite la query PromQL `rate(container_fs_writes_bytes_total[2m])`), risulta quanto segue: ### Applicazione con flusso attivo di scrittura su disco: * **Applicazione / Deployment:** `cache-app` * **Pod:** `cache-app-5c6fd4f5cf-xxsz4` * **Container:** `cache-service` ### Rate di scrittura: * **A livello di Container (`cache-service`):** **~421.6 KB/s** (*421.635,55 B/s*) * **A livello complessivo del Pod:** **~435.8 KB/s** (*435.808,29 B/s*) --- *Nota: Le altre applicazioni utente nel namespace `default` (`demo-app`, `worker-app`) presentano attualmente un rate di scrittura pari a 0 B/s.*
Claude Opus 5.5	76.16 s	8947	1528	10475	0.87	**Risposta: l'unica applicazione che scrive su disco in modo continuo è `cache-app`, a circa 480 KB/s (≈ 0,47 MiB/s).** **Rate di scrittura (media mobile su 5 minuti, `rate(container_fs_writes_bytes_total[5m])`)** | Applicazione | Pod | Scrittura su disco | |---|---|---| | **cache-app** | `cache-app-5c6fd4f5cf-xxsz4` (container `cache-service`, immagine busybox, device `/dev/sde`) | **≈ 486 KB/s** | | demo-app | 2 pod | 0 B/s | | worker-app | 2 pod | 0 B/s | **Nota sul calcolo:** sommando per pod risultano circa 960 KB/s, ma il valore è raddoppiato. cAdvisor espone due serie per lo

| Claude Opus 5.5 | 76.16 s | 8947 | 1528 | 10475 | 0.87 | **Risposta: l'unica applicazione che scrive su disco in modo continuo è `cache-app`, a circa 480 KB/s (≈ 0,47 MiB/s).**

Rate di scrittura (media mobile su 5 minuti, `rate(container_fs_writes_bytes_total[5m])`)

\| Applicazione \| Pod \| Scrittura su disco \|
\|---\|---\|---\|
\| **cache-app** \| `cache-app-5c6fd4f5cf-xxsz4` (container `cache-service`, immagine busybox, device `/dev/sde`) \| **≈ 486 KB/s** \|
\| demo-app \| 2 pod \| 0 B/s \|
\| worker-app \| 2 pod \| 0 B/s \|

Nota sul calcolo: sommando per pod risultano circa 960 KB/s, ma il valore è raddoppiato. cAdvisor espone due serie per lo stesso pod: una a livello di pod (465 KB/s) e una a livello di container (486 KB/s), e la somma le conta entrambe. Filtrando con `container!=""` resta il valore corretto, circa 480 KB/s.

Andamento nell'ultima ora: ci sono dati solo per gli ultimi ~30 minuti circa, forse perché il pod è ripartito di recente. In quel periodo la scrittura è continua e stabile:
- 378 KB/s, poi 501, 446, 471, 491, 481 e 484 KB/s.
- Media di circa 465 KB/s, senza picchi.
- Al ritmo attuale corrisponde a circa 1,7 GB/ora e circa 40 GB/giorno.

Cosa consiglio di fare
1. **Capire dove scrive `cache-service`.** Se scrive nel filesystem del container o in un `emptyDir` senza `sizeLimit`, ~40 GB/giorno possono riempire il disco del nodo `thesis-cluster-control-plane`. In quel caso il pod verrebbe espulso (eviction) per mancanza di spazio.
2. **Fissare dei limiti.** Valuta un `resources.limits.ephemeral-storage` per il container, oppure un PVC con dimensione definita se i dati devono restare.
3. **Controllare il livello di log.** Con un'immagine busybox, molte scritture potrebbero essere log o file di debug: conviene verificarlo.
4. **Mettere un allarme.** Ad esempio un alert Prometheus su `rate(container_fs_writes_bytes_total{container!=""}[5m]) > 1e6` (oltre 1 MB/s per container) e sullo spazio libero del nodo.

Se vuoi, posso salvare questa misura su MongoDB per seguirne l'andamento nel tempo. |

Claude Sonnet 5.5	69.55 s	89715	874	90589	0.87	**L'applicazione che scrive attivamente su disco è `cache-app`, a circa 486 KB/s (≈ 0,46 MiB/s).** Ho calcolato il rate medio su 5 minuti di `container_fs_writes_bytes_total` per i pod del namespace `default`: | Applicazione | Pod | Scrittura su disco | |---|---|---| | **cache-app** | cache-app-5c6fd4f5cf-xxsz4 | **≈ 486.339 B/s (~475 KiB/s)** | | worker-app | worker-app-84cf4dd446-9kj75 | 0 B/s | | worker-app | worker-app-84cf4dd446-ftlln | 0 B/s | | demo-app | demo-app-7958c64697-2qg9s | 0 B/s | | demo-app | demo-app-7958c64697-4rzjl | 0 B/s | - Il valore è una misura puntuale sugli ultimi 5 minuti. Non ho verificato se il flusso sia costante nel tempo. - `cache-app` ha una sola replica, quindi quel rate corrisponde all'intero deployment. **Raccomandazioni** 1. Una cache che scrive in modo continuo su disco è insolita, quindi conviene capire cosa scrive. Può trattarsi di persistenza (RDB/AOF se è Redis), di log verbosi o di spill su disco per memoria insufficiente. 2. Se la scrittura non è voluta, rivedi la configurazione di persistenza o il livello di log. 3. Se è voluta, controlla che il volume abbia spazio e IOPS adeguati. A 486 KB/s si scrivono circa 40 GB al giorno. 4. Posso recuperare lo storico da MongoDB per vedere se il rate è in crescita o stabile. Posso anche controllare l'uso del filesystem del nodo.
GPT-4o Mini	54.05 s	4280	255	4535	0.00	Dall'analisi delle metriche di scrittura su disco, sembra che ci siano flussi attivi di scrittura. Ecco i dettagli: - **Applicazione**: Non è possibile identificare direttamente quale applicazione stia generando il flusso di scrittura, poiché le metriche non sono state segmentate per applicazione. Tuttavia, posso dirti che ci sono scritture attive su disco. - **Rate di scrittura**: Le scritture su disco avvengono a diverse rate, con valori che variano da 0 a circa 595494.22 byte al secondo in alcuni momenti. Per avere un'analisi più dettagliata e identificare quale applicazione specifica sta generando il flusso di scrittura, ti consiglio di eseguire una query più mirata che segmenti le metriche per applicazione o pod. Potresti considerare di sottomettere questa richiesta a un modello più avanzato per ottenere informazioni più specifiche.

						metriche per applicazione o pod. Potresti considerare di sottomettere questa richiesta a un modello più avanzato per ottenere informazioni più specifiche.
GPT-4o	54.33 s	59471	69	59540	0.15	L'applicazione che genera il flusso di scrittura su disco più significativo è "cache-app", con un totale di 1,001,394,176 byte scritti.

```
[BILLING MONITOR] Spesa cumulata OpenAI: $0.5820 (mini: $0.0132 | 4o: $0.5688)
```

DOMANDA 3A

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [3A]: Rileva quale carico di lavoro sta eseguendo operazioni continue di I/O in scrittura su filesystem e specifica la velocità di scrittura registrata.
--------------------------------------------------------------------------------------------------------------
Invocazione parallela modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/29/26 16:54:17] INFO     Processing request of type CallToolRequest                                                                                                                server.py:733
[10/29/26 16:54:17] INFO     Processing request of type CallToolRequest                                                                                                                server.py:733
Prometheus MCP server running on stdio
[10/29/26 16:54:19] INFO     Processing request of type CallToolRequest                                                                                                                server.py:733
[10/29/26 16:54:20] INFO     Processing request of type ListToolsRequest                                                                                                               server.py:733
[10/29/26 16:54:21] INFO     Processing request of type ListToolsRequest                                                                                                               server.py:733
Prometheus MCP server running on stdio
[10/29/26 16:54:24] INFO     Processing request of type ListToolsRequest                                                                                                               server.py:733
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/29/26 16:54:41] INFO     Processing request of type CallToolRequest                                                                                                                server.py:733
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
[10/29/26 16:54:42] INFO     Processing request of type ListToolsRequest                                                                                                               server.py:733
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	2.06 ms	0	0	0	N/A	cache-app (494183.96 B/s)
Gemini 2.5 Flash	25.69 s	4920	1480	6400	0.63	Il carico di lavoro che sta eseguendo operazioni continue di I/O in scrittura su filesystem è `cache-app` nel namespace `default`. La velocità di scrittura registrata per il pod `cache-app-5c6fd4f5cf-xxsz4` è di circa **0.45 MB/s**.
Gemini 3.6 Flash	53.93 s	99701	666	100367	0.00	[Risposta priva di testo]
Claude Opus 5.5	51.64 s	16534	1734	18268	0.90	Il carico che scrive in modo continuo su filesystem è il Deployment **`cache-app`** (namespace `default`, un solo pod: `cache-app-5c6fd4f5cf-xxsz4`). Scrive circa **476 KB/s** in modo costante. ## Velocità di scrittura misurata | Rilevazione | Valore | |---|---| | Valore attuale (media sugli ultimi 5 minuti) | **475.996 B/s ≈ 476 KB/s ≈ 0,45 MiB/s** | | Ultimi 30 minuti circa (Prometheus, una misura ogni 5 min) | sempre tra **474 e 529 KB/s** | | Storico delle ultime 6 ore (database delle metriche) | in genere **430–557 KB/s** | A questo ritmo il pod scrive circa **1,7 GB all'ora (circa 41 GB al giorno)**. È una stima che parte dal valore attuale e presuppone che il ritmo resti costante. ## Perché la scrittura risulta continua - **Andamento piatto:** nella serie temporale la scrittura resta stabile, con oscillazioni di circa ±10%. Non ci sono picchi isolati: è un flusso costante. - **Si ripete nel tempo:** lo storico mostra lo stesso comportamento nelle rilevazioni tra le 09:56 e le 10:56 e tra le 15:20 e le 15:51. Fanno eccezione tre letture: una pari a zero alle 10:21, forse un riavvio o una lettura mancata; un valore di circa 98 KB/s nel primo punto della serie, probabilmente lo stesso tipo di evento; e un calo isolato a 161 KB/s alle 10:37. - **Il resto è uso leggero:** gli altri processi che scrivono sono quelli di sistema, con volumi molto più bassi. etcd scrive circa 34 KB/s, Prometheus circa 8,7 KB/s e Grafana circa 0,6 KB/s. - **Nessuna scrittura dalle altre app:** `demo-app` e `worker-app` (2 pod ciascuna) sono a **0 B/s**. - **Il consumo di risorse è basso:** CPU circa 0,005–0,02 core e memoria circa 1–2 MB. Il pod sembra fare soprattutto scrittura sequenziale continua, senza elaborazioni pesanti. ## Cosa consiglio 1. **Capire cosa scrive:** controllare se il pod scrive sul layer del container (cioè sul disco del nodo) o su un volume. Se scrive

						## Velocità di scrittura misurata | Rilevazione | Valore | |---|---| | Valore attuale (media sugli ultimi 5 minuti) | **475.996 B/s ≈ 476 KB/s ≈ 0,45 MiB/s** | | Ultimi 30 minuti circa (Prometheus, una misura ogni 5 min) | sempre tra **474 e 529 KB/s** | | Storico delle ultime 6 ore (database delle metriche) | in genere **430–557 KB/s** | A questo ritmo il pod scrive circa **1,7 GB all'ora (circa 41 GB al giorno)**. È una stima che parte dal valore attuale e presuppone che il ritmo resti costante. ## Perché la scrittura risulta continua - **Andamento piatto:** nella serie temporale la scrittura resta stabile, con oscillazioni di circa ±10%. Non ci sono picchi isolati: è un flusso costante. - **Si ripete nel tempo:** lo storico mostra lo stesso comportamento nelle rilevazioni tra le 09:56 e le 10:56 e tra le 15:20 e le 15:51. Fanno eccezione tre letture: una pari a zero alle 10:21, forse un riavvio o una lettura mancata; un valore di circa 98 KB/s nel primo punto della serie, probabilmente lo stesso tipo di evento; e un calo isolato a 161 KB/s alle 10:37. - **Il resto è uso leggero:** gli altri processi che scrivono sono quelli di sistema, con volumi molto più bassi. etcd scrive circa 34 KB/s, Prometheus circa 8,7 KB/s e Grafana circa 0,6 KB/s. - **Nessuna scrittura dalle altre app:** `demo-app` e `worker-app` (2 pod ciascuna) sono a **0 B/s**. - **Il consumo di risorse è basso:** CPU circa 0,005–0,02 core e memoria circa 1–2 MB. Il pod sembra fare soprattutto scrittura sequenziale continua, senza elaborazioni pesanti. ## Cosa consiglio 1. **Capire cosa scrive:** controllare se il pod scrive sul layer del container (cioè sul disco del nodo) o su un volume. Se scrive sul layer del container o su un `emptyDir` senza limite, il disco del nodo può riempirsi e il pod può essere sfrattato (eviction). Comandi: `kubectl describe pod cache-app-5c6fd4f5cf-xxsz4` e `kubectl exec ... -- du -sh /percorso`. 2. **Limitare lo spazio usato:** impostare `resources.requests/limits.ephemeral-storage` e, se usa un `emptyDir`, un `sizeLimit`. Se i dati devono restare anche dopo un riavvio, spostarli su un PVC dedicato. 3. **Controllare la logica dell'app:** per una "cache" circa 41 GB al giorno sono molti. Verificare se c'è una politica che elimina o sostituisce i dati vecchi (eviction o rotazione). Se manca, lo spazio occupato cresce senza limite. 4. **Aggiungere un allarme:** creare un alert su `rate(container_fs_writes_bytes_total{pod=~"cache-app.*"}[5m])` e sullo spazio libero del nodo (`node_filesystem_avail_bytes`), così da accorgersi in anticipo della saturazione.

Claude Sonnet 5.5	48.81 s	134354	1273	135627	0.38	**Il carico di lavoro che scrive in continuo sul filesystem è `cache-app`.** Il pod è `cache-app-5c6fd4f5cf-xxsz4`, nel namespace `default`. **Velocità di scrittura** (metrica `container_fs_writes_bytes_total` di cAdvisor): - Media su 5 minuti: circa 940.000 B/s, cioè circa **0,9 MB/s** (≈ 0,90 MiB/s). - Media su 1 minuto: circa 915.000 B/s, cioè circa 0,87 MiB/s. Il valore è stabile. **Confronto con gli altri workload** (stessa query, namespace `default`): | Workload | Pod | Scrittura FS (B/s) | |---|---|---| | **cache-app** | cache-app-5c6fd4f5cf-xxsz4 | **≈ 940.218** | | demo-app | 2qg9s, 4rzjl | 0 | | worker-app | 9kj75, ftlln | 0 | Le operazioni di scrittura sono circa 0,36 al secondo (`container_fs_writes_total`). Con circa 0,9 MB/s totali, ogni operazione scrive quindi in media 2,5 MB. Sono quindi poche scritture molto grandi, e questo fa pensare a flush o snapshot periodici. Un esempio tipico è la persistenza su disco di una cache (dump RDB/AOF o simili). È solo un'ipotesi: i dati non permettono di confermarla. **Raccomandazioni** 1. Controlla cosa scrive `cache-app`. Verifica se la persistenza è voluta e come è configurata (frequenza degli snapshot, AOF rewrite). 2. Verifica dove finiscono i dati. Con 0,9 MB/s si scrivono circa 75 GB al giorno. Se la destinazione è l'`emptyDir` o il layer scrivibile del container, c'è il rischio di esaurire lo spazio sul nodo o di avere eviction per ephemeral-storage. Conviene usare un PVC dedicato e impostare un `ephemeral-storage` limit. 3. Se la scrittura non serve, riduci o disattiva la persistenza. Se serve, aumenta l'intervallo tra i flush. 4. Imposta un alert su `rate(container_fs_writes_bytes_total[5m])` per questo workload, così ti accorgi di eventuali aumenti. Non ho verificato i volumi montati né la configurazione del container, perché gli strumenti a disposizione non lo permettono.
GPT-4o Mini	32.96 s	5523	356	5879	0.20	Dall'analisi delle metriche di scrittura su filesystem, ho rilevato che ci sono operazioni di I/O in scrittura attive nel cluster. Ecco i dettagli: ### Velocità di Scrittura Registrata - **Velocità di scrittura massima**: 7595.11 bytes/s - **Velocità di scrittura media**: 357361.38 bytes/s - **Velocità di scrittura minima**: 0 bytes/s (indicando periodi senza scritture)

GPT-4o Mini	32.96 s	5523	356	5879	0.20	Dall'analisi delle metriche di scrittura su filesystem, ho rilevato che ci sono operazioni di I/O in scrittura attive nel cluster. Ecco i dettagli: ### Velocità di Scrittura Registrata - **Velocità di scrittura massima**: 7595.11 bytes/s - **Velocità di scrittura media**: 357361.38 bytes/s - **Velocità di scrittura minima**: 0 bytes/s (indicando periodi senza scritture)

						### Carichi di Lavoro Attivi I carichi di lavoro attivi che stanno eseguendo operazioni di scrittura includono: - **cache-app**: 0 bytes/s - **demo-app**: 0 bytes/s - **worker-app**: 0 bytes/s ### Conclusione Sebbene ci siano stati picchi di scrittura, attualmente non ci sono carichi di lavoro attivi che stiano eseguendo operazioni di scrittura continua. Ti consiglio di monitorare ulteriormente le metriche nel tempo per identificare eventuali pattern o carichi di lavoro che potrebbero attivarsi in momenti diversi. Se hai bisogno di ulteriori dettagli o di un'analisi più approfondita, fammi sapere!
GPT-4o	29.65 s	59507	106	59613	0.15	Il carico di lavoro che sta eseguendo operazioni continue di I/O in scrittura su filesystem con la velocità di scrittura più alta è il pod `cache-app-5c6fd4f5cf-xxsz4` nel namespace `default`. La velocità di scrittura registrata è di 1,059,065,856 byte.

[BILLING MONITOR] Spesa cumulata OpenAI: $0.7329 (mini: $0.0142 | 4o: $0.7186)

DOMANDA 3B

```
================================================================================================================
ESECUZIONE DOMANDA [3B]: C'è un'applicazione che presenta throughput di scrittura su disco diverso da zero? Se presente, qual è e a che frequenza/tasso (rate) trasferisce dati verso lo storage?
----------------------------------------------------------------------------------------------------------------
Invocazione parallela modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/29/26 16:56:25] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 16:56:25] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
[10/29/26 16:56:26] INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/29/26 16:56:28] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 16:56:30] INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.71 ms	0	0	0	N/A	cache-app (494183.96 B/s)

Gemini 2.5 Flash	43.84 s	56340	1239	57579	0.64	Le seguenti applicazioni presentano un throughput di scrittura su disco diverso da zero:
						
						* **Applicazione:** `prometheus-grafana` (nel namespace `monitoring`), pod `prometheus-grafana-6d8f777649-5xgj7`
						    * **Tasso di scrittura su disco:** 700.27 B/s (generale per il pod)
						    * **Tasso di scrittura su disco (container `grafana`):** 593.50 B/s
						
						* **Applicazione:** `prometheus-prometheus-kube-prometheus-prometheus` (nel namespace `monitoring`), pod `prometheus-prometheus-kube-prometheus-prometheus-0`
						    * **Tasso di scrittura su disco:** 8286.23 B/s (generale per il pod)
						    * **Tasso di scrittura su disco (container `prometheus`):** 8533.73 B/s
						
						* **Applicazione:** `cache-app` (nel namespace `default`), pod `cache-app-5c6fd4f5cf-xxsz4`
						    * **Tasso di scrittura su disco:** 418570.56 B/s (generale per il pod)
						    * **Tasso di scrittura su disco (container `cache-service`):** 457521.30 B/s
						
						* **Applicazione:** `etcd-thesis-cluster-control-plane` (nel namespace `kube-system`), pod `etcd-thesis-cluster-control-plane`
						    * **Tasso di scrittura su disco:** 31570.80 B/s (generale per il pod)
						    * **Tasso di scrittura su disco (container `etcd`):** 32010.06 B/s
						
						Questi tassi sono stati calcolati come media negli ultimi 5 minuti.

Gemini 3.6 Flash	60.05 s	121041	1761	122802	0.72	Sì, nel cluster è presente un'applicazione con throughput di scrittura su disco maggiore di zero.
						
						### Dettagli dell'applicazione e delle metriche riscontrate:
						
						* **Applicazione / Deployment**: `cache-app`
						* **Pod interessato**: `cache-app-5c6fd4f5cf-xxsz4` (contenitore `cache-service`)
						* **Tasso / Rate di scrittura (Disk Write Rate)**:
						  * **~458.5 KB/s** (ovvero **~458.537 B/s** a livello di singolo contenitore `cache-service`).
						  * Il totale a livello di pod si attesta attorno a **~470 - 490 KB/s**.
						
						--- 	

Gemini 3.6 Flash	60.05 s	121041	1761	122802	0.72	Sì, nel cluster è presente un'applicazione con throughput di scrittura su disco maggiore di zero.
						
						### Dettagli dell'applicazione e delle metriche riscontrate:
						
						* **Applicazione / Deployment**: `cache-app`
						* **Pod interessato**: `cache-app-5c6fd4f5cf-xxsz4` (contenitore `cache-service`)
						* **Tasso / Rate di scrittura (Disk Write Rate)**:
						  * **~458.5 KB/s** (ovvero **~458.537 B/s** a livello di singolo contenitore `cache-service`).
						  * Il totale a livello di pod si attesta attorno a **~470 - 490 KB/s**.
						
						--- 	
						
						### Verifiche eseguite sulle altre applicazioni del carico di lavoro:
						* **`demo-app`** (Pod: `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl`): **0 B/s** (nessuna scrittura su disco).
						* **`worker-app`** (Pod: `worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln`): **0 B/s** (nessuna scrittura su disco).

Claude Opus 5.5	57.16 s	13593	2003	15596	0.86	Sì, una sola applicazione scrive su disco: **`cache-app`**. ### Risultato | Applicazione | Pod | Container | Device | Throughput di scrittura | Operazioni di scrittura | |---|---|---|---|---|---| | **cache-app** | `cache-app-5c6fd4f5cf-xxsz4` | `cache-service` (busybox) | `/dev/sde` | **~471.500 B/s ≈ 460 KiB/s ≈ 0,45 MiB/s** | **~0,16 operazioni/s** | | demo-app (2 pod) | – | – | – | 0 B/s | – | | worker-app (2 pod) | – | – | – | 0 B/s | – | Il dato viene da `rate(container_fs_writes_bytes_total[5m])` sul namespace `default`, nodo `thesis-cluster-control-plane`. ### Cosa significano i numeri - **Tasso:** circa **0,45 MiB/s** di media negli ultimi 5 minuti, cioè circa 1,6–1,7 GiB all'ora se il ritmo resta costante. - **Frequenza:** circa 0,16 operazioni di scrittura al secondo, quindi **una scrittura ogni 6 secondi circa**. - **Dimensione media di una scrittura:** 471.528 / 0,1617 ≈ **2,9 MB**. Sembra un processo che scrive blocchi grandi a intervalli regolari, non un flusso continuo di piccole scritture. È una stima basata su medie, non un'osservazione diretta. ### Nota sul calcolo Con la prima query avevo ottenuto ~877 KB/s, ma il valore era sbagliato: sommava due volte gli stessi dati, una come pod e una come container. Il valore corretto è quello del container, **~471 kB/s**. Se costruite dashboard o alert su questa metrica, filtrate con `container!=""` per evitare lo stesso errore. ### Raccomandazioni 1. **Controllate dove scrive il container.** Se scrive sul filesystem interno del container (writable layer) e non su un volume, usa il disco del nodo e può farlo riempire, con il rischio che Kubernetes sfratti i pod per pressione sul disco. In quel caso conviene un volume dedicato (`emptyDir` con `sizeLimit`, oppure un PVC). 2. **Impostate dei limiti di spazio disco** sul container (`resources.requests/limits.ephemeral-storage`), così una crescita eccessiva resta confinata a questo pod. 3. **Aggiungete un alert** su questa metrica (con il filtro sopra), con una soglia di partenza intorno a 1 MiB/s mantenuto per 10 minuti. 4. **Verificate se i file vengono ruotati o cancellati.** A circa 1,6 GiB/h, senza pulizia, lo spazio si esaurisce in fretta. **Limite dei dati:** la query sull'ultima ora ha restituito un solo punto, quindi Prometheus non ha storico sufficiente per dire se questo ritmo è stabile o è un picco. I valori sono medie degli ultimi 5 minuti. Per capire la tendenza ripetete la misura più avanti o salvate periodicamente le metriche su MongoDB.
Claude Sonnet 5.5	42.80 s	90403	1255	91658	0.29	Sì, c'è un'applicazione con throughput di scrittura su disco diverso da zero: **`cache-app`**. **Risultati per il namespace `default`** (rate su finestra di 5 minuti, metrica cAdvisor `container_fs_writes_bytes_total`): | Applicazione | Pod | Scrittura su disco | |---|---|---| | **cache-app** | `cache-app-5c6fd4f5cf-xxsz4` | **≈ 942.671 B/s** (≈ 0,94 MB/s, ≈ 0,90 MiB/s) | | demo-app | `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl` | 0 B/s | | worker-app | `worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln` | 0 B/s | **Dettaglio su `cache-app`:** - Scrive circa **0,94 MB/s**, cioè circa 56 MB al minuto o 3,3 GB all'ora se il tasso resta costante. - Le operazioni di scrittura (`container_fs_writes_total`) sono circa **0,34 al secondo**. Questo vale circa 2,8 MB per operazione. Le scritture sono quindi poche e di grandi dimensioni, tipiche di flush o snapshot periodici (per esempio una persistenza RDB/AOF) e non di scritture piccole e continue. **Raccomandazioni:** 1. Controlla che la scrittura sia voluta. Per una cache in memoria può indicare persistenza attiva, log verbosi o scritture nel filesystem scrivibile del container. 2. Se i dati devono sopravvivere al riavvio del pod, usa un volume persistente (PVC) dimensionato per ≈ 3 GB/h di scritture. Se la persistenza non serve, disattivala per ridurre l'I/O. 3. Se la scrittura è inattesa, controlla la configurazione dell'applicazione (persistenza, log) e imposta `ephemeral-storage` requests/limits per evitare che riempia il disco del nodo. 4. Ho misurato solo un istante, con rate su 5 minuti. Per distinguere un tasso costante da picchi periodici, guarda l'andamento su più ore (`get_historical_metrics` per `cache-app` o una `prom_range`).
GPT-4o Mini	21.40 s	1720	91	1811	0.00	Non ci sono applicazioni che presentano un throughput di scrittura su disco diverso da zero, poiché la query ha restituito risultati vuoti. Pertanto, non ci sono dati trasferiti verso lo storage in questo momento. Se hai bisogno di ulteriori informazioni o di un'analisi più approfondita, fammi sapere!

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	1.11 ms	0	0	0	N/A	worker-app (0.2258 cores)
Gemini 2.5 Flash	34.38 s	3714	1089	4803	0.17	Dall'analisi delle metriche di utilizzo del processore, l'applicazione che risulta essere il top consumer di cicli CPU è **worker-app**, con un tasso di assorbimento complessivo di **1.149 core**.
Gemini 3.6 Flash	30.57 s	4716	1651	6367	0.43	Dall'analisi delle metriche di utilizzo CPU aggregate dai Pod attivi del cluster, l'applicazione risultante come **top consumer** è **`worker-app`**. --- ### **Dettaglio del consumo per applicazione** 1. **`worker-app`** (Top Consumer) - **Consumo complessivo:** **`~0.575 core`** (circa **575.4 millicore**) - *Dettaglio dei Pod:* - `worker-app-84cf4dd446-9kj75`: ~0.295 core (294.7 m CPU) - `worker-app-84cf4dd446-ftlln`: ~0.281 core (280.7 m CPU) 2. **`cache-app`** - **Consumo complessivo:** **`~0.005 core`** (circa **5.2 millicore**) - *Dettaglio dei Pod:* - `cache-app-5c6fd4f5cf-xxsz4`: ~0.005 core 3. **`demo-app`** - **Consumo complessivo:** **`0.000 core`** (inattiva / consumo trascurabile) - *Dettaglio dei Pod:* - `demo-app-7958c64697-2qg9s`: 0 core - `demo-app-7958c64697-4rzjl`: 0 core --- ### **Sintesi** L'applicazione **`worker-app`** assorbe da sola la quasi totalità delle risorse CPU allocate ai carichi applicativi, con un tasso di assorbimento complessivo pari a **0.575 core CPU**.
Claude Opus 5.5	50.75 s	11507	1513	13020	0.45	**worker-app** è l'applicazione che consuma più CPU, con circa **0,575 core** in totale (media degli ultimi 5 minuti, solo namespace `default`). ### Consumo per applicazione | Applicazione | Pod | CPU (core) | |---|---|---| | **worker-app** | worker-app-84cf4dd446-9kj75 | 0,2947 | | | worker-app-84cf4dd446-ftlln | 0,2808 | | | **Totale** | **~0,575** |

| Claude Opus 5.5 | 50.75 s | 11507 | 1513 | 13020 | 0.45 | **worker-app** è l'applicazione che consuma più CPU, con circa **0,575 core** in totale (media degli ultimi 5 minuti, solo namespace `default`).

Consumo per applicazione

\| Applicazione \| Pod \| CPU (core) \|
\|---\|---\|---\|
\| **worker-app** \| worker-app-84cf4dd446-9kj75 \| 0,2947 \|
\| \| worker-app-84cf4dd446-ftlln \| 0,2808 \|
\| \| **Totale** \| **~0,575** \|
\| cache-app \| cache-app-5c6fd4f5cf-xxsz4 \| ~0,005 \|
\| demo-app \| 2 pod \| 0 \|

worker-app da sola usa circa il 99% della CPU consumata nel namespace.

Problema: worker-app è frenata dal limite di CPU
- **Limite:** ogni pod ha un limite di 0,3 core e ne usa circa 0,28–0,29, cioè il 94–98% di quanto gli è permesso.
- **Rallentamento forzato (throttling):** in circa il **95%** dei periodi di controllo della CPU il pod è stato fermato perché aveva raggiunto il limite. Questo vale per entrambi i pod.
- **Conseguenza:** il consumo reale è "bloccato" a quel valore. Se il limite fosse più alto, l'applicazione userebbe probabilmente più CPU. La latenza e la velocità di elaborazione del worker sono quasi certamente peggiorate.

Cosa consiglio
1. **Alzare il limite di CPU** di worker-app ad almeno 0,6–1 core per pod. Poi controllare il nuovo consumo reale e il throttling con `container_cpu_cfs_throttled_periods_total`.
2. **Controllare le richieste di CPU.** La mia query non le ha distinte dai limiti, quindi vanno verificate. Dovrebbero essere vicine al consumo reale (~0,3 core per pod), così lo scheduler sa quanta CPU riservare.
3. **Valutare uno scaling orizzontale (HPA)** se il lavoro del worker si può dividere tra più pod. Ha senso dopo aver sistemato i limiti, perché ora il throttling falsa il dato di CPU su cui l'HPA si basa.
4. **demo-app non usa CPU**, nonostante abbia 2 repliche. Verificare se è inattiva per scelta o se ha un problema, e nel caso ridurre le repliche. |

Claude Sonnet 5.5	39.75 s	89909	920	90829	0.45	**Il top consumer di CPU è `worker-app`, con circa 0,575 core in totale.** Ho calcolato `rate(container_cpu_usage_seconds_total[5m])` per i pod del namespace `default` e li ho sommati per Deployment. | Applicazione | Pod | CPU per pod (core) | CPU totale (core) | |---|---|---|---| | **worker-app** | `worker-app-84cf4dd446-ftlln` | 0,2948 | **≈ 0,575** | | | `worker-app-84cf4dd446-9kj75` | 0,2799 | | | cache-app | `cache-app-5c6fd4f5cf-xxsz4` | 0,0055 | ≈ 0,005 | | demo-app | `demo-app-7958c64697-2qg9s` | 0 | 0 | | | `demo-app-7958c64697-4rzjl` | 0 | 0 | - **Carico:** `worker-app` consuma circa 100 volte più di `cache-app`. I suoi due pod sono bilanciati, intorno a 0,28–0,29 core ciascuno. - **demo-app:** i suoi pod risultano a 0 core. Potrebbero essere inattivi, ma potrebbe anche mancare la raccolta delle metriche. Non l'ho verificato. - **Limiti di CPU:** non ho confrontato il consumo con i limiti dei pod. Non so quindi se `worker-app` sia vicino al throttling. - **Raccomandazione:** controlla requests e limits di CPU di `worker-app`, ad esempio con `kube_pod_container_resource_limits` e `container_cpu_cfs_throttled_periods_total`. Così verifichi che siano dimensionati su circa 0,3 core per pod, con un margine adeguato per i picchi.

GPT-4o Mini	26.49 s	46150	207	46357	0.43	Dall'analisi delle metriche di utilizzo del processore, l'applicazione che risulta essere il top consumer di cicli CPU è **"worker-app"**, con un tasso di assorbimento complessivo di **1.164 core** (sommando i valori dei pod associati).
						
						### Dettagli:
						- **Pod**:
						  - `worker-app-84cf4dd446-9kj75`: 0.589 core
						  - `worker-app-84cf4dd446-ftlln`: 0.575 core
						
						### Raccomandazione:
						Ti consiglio di monitorare attentamente l'utilizzo della CPU di questa applicazione e considerare eventuali ottimizzazioni o scalabilità per gestire il carico.

GPT-4o	32.51 s	69500	158	69658	0.43	L'applicazione che risulta essere il top consumer di cicli CPU è "worker-app". I suoi pod stanno consumando complessivamente: - `worker-app-84cf4dd446-9kj75`: 0.5893 core - `worker-app-84cf4dd446-ftlln`: 0.5754 core Il tasso di assorbimento complessivo in core per l'applicazione "worker-app" è quindi di circa 1.1647 core.

						o una `prom_range`).
GPT-4o Mini	21.40 s	1720	91	1811	0.00	Non ci sono applicazioni che presentano un throughput di scrittura su disco diverso da zero, poiché la query ha restituito risultati vuoti. Pertanto, non ci sono dati trasferiti verso lo storage in questo momento. Se hai bisogno di ulteriori informazioni o di un'analisi più approfondita, fammi sapere!

GPT-4o	32.05 s	59522	343	59865	0.28	Sì, ci sono diverse applicazioni che presentano un throughput di scrittura su disco diverso da zero. Ecco alcune delle applicazioni e i loro tassi di scrittura:
						
						1. **cache-app**:
						  - Pod: `cache-app-5c6fd4f5cf-xxsz4`
						  - Throughput di scrittura: 1,110,446,080 bytes
						
						2. **prometheus-prometheus-kube-prometheus-prometheus**:
						  - Pod: `prometheus-prometheus-kube-prometheus-prometheus-0`
						  - Throughput di scrittura: 107,913,216 bytes
						
						3. **etcd**:
						  - Pod: `etcd-thesis-cluster-control-plane`
						  - Throughput di scrittura: 84,508,672 bytes
						
						4. **prometheus-grafana**:
						  - Pod: `prometheus-grafana-6d8f777649-5xgj7`
						  - Throughput di scrittura: 2,195,456 bytes
						
						5. **demo-app**:
						  - Pod: `demo-app-7958c64697-2qg9s` e `demo-app-7958c64697-4rzjl`
						  - Throughput di scrittura: 16,384 bytes ciascuno
						
						Questi valori rappresentano il totale dei byte scritti su disco dai rispettivi pod. Se hai bisogno di ulteriori dettagli o di altre applicazioni, fammi sapere!

[BILLING MONITOR] Spesa cumulata OpenAI: $0.8854 (mini: $0.0145 | 4o: $0.8709)

DOMANDA 4

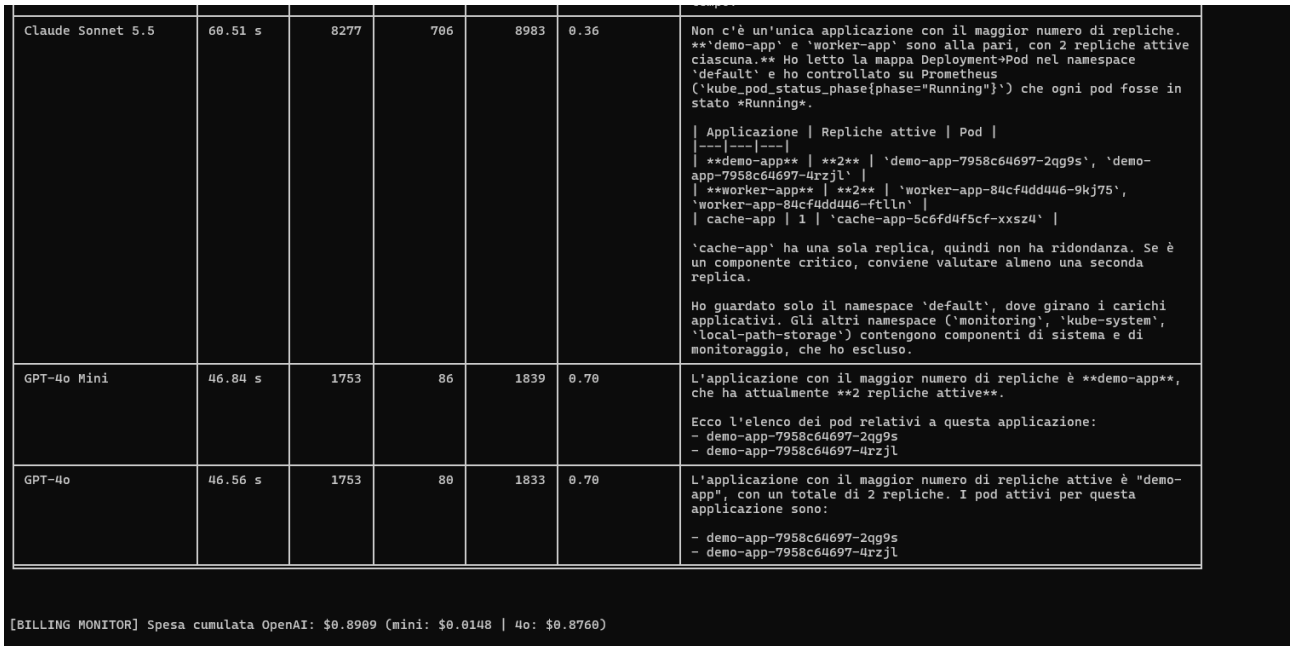
```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [4]: Identifica l'applicazione con il maggior numero di repliche: elenca i relativi pod e indica quante repliche risultano attive.
--------------------------------------------------------------------------------------------------------------
Invocazione parallela modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/29/26 17:11:55] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 17:11:55] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 17:11:55] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 17:11:55] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 17:11:55] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 17:11:55] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 17:12:01] INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
[10/29/26 17:12:01] INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
[10/29/26 17:12:02] INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
[10/29/26 17:12:02] INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
[10/29/26 17:12:02] INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.12 ms	0	0	0	N/A	App: demo-app | Repliche: 2 | Pods: ['demo-app-7958c64697-2qg9s', 'demo-app-7958c64697-4rzjl']
Gemini 2.5 Flash	48.37 s	2194	649	2843	0.70	L'applicazione con il maggior numero di repliche attive è `demo-app`, con 2 repliche. I pod relativi sono: `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl`.

Gemini 3.6 Flash	139.64 s	8088	1210	9298	0.32	In base all'analisi dei carichi di lavoro presenti nel cluster, **due applicazioni a pari merito** registrano il maggior numero di repliche: **`demo-app`** e **`worker-app`**, ciascuna con **2 repliche attive** su 2 desiderate.
						
						Di seguito i dettagli per ciascuna applicazione:
						
						--- 	
						
						### 1. `demo-app`
						* **Repliche attive (available):** 2
						* **Pod associati:**
						  * `demo-app-7958c64697-2qg9s`
						  * `demo-app-7958c64697-4rzjl`
						
						--- 	
						
						### 2. `worker-app`
						* **Repliche attive (available):** 2
						* **Pod associati:**
						  * `worker-app-84cf4dd446-9kj75`
						  * `worker-app-84cf4dd446-ftlln`
						
						--- 	
						
						### Riepilogo complessivo delle altre applicazioni nel namespace `default`:
						* **`cache-app`**: 1 replica attiva (Pod: `cache-app-5c6fd4f5cf-xxsz4`)

Claude Opus 5.5	68.07 s	10880	918	11798	0.00	Nessuna applicazione ha più repliche di tutte le altre: **demo-app** e **worker-app** sono a pari merito, con **2 repliche attive** ciascuna. Ho controllato i Deployment nel namespace `default`, confrontando i pod attivi con i dati di Prometheus (kube-state-metrics). | Deployment | Repliche richieste | Repliche disponibili | Pod attivi | |---|---|---|---| | **demo-app** | 2 | **2** | `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl` | | **worker-app** | 2 | **2** | `worker-app-84cf4dd446-9kj75`,

Claude Opus 5.5	68.07 s	10880	918	11798	0.00	Nessuna applicazione ha più repliche di tutte le altre: **demo-app** e **worker-app** sono a pari merito, con **2 repliche attive** ciascuna. Ho controllato i Deployment nel namespace `default`, confrontando i pod attivi con i dati di Prometheus (kube-state-metrics). | Deployment | Repliche richieste | Repliche disponibili | Pod attivi | |---|---|---|---| | **demo-app** | 2 | **2** | `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl` | | **worker-app** | 2 | **2** | `worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln` | | cache-app | 1 | 1 | `cache-app-5c6fd4f5cf-xxsz4` | **Stato:** - In tutti e tre i Deployment le repliche disponibili sono pari a quelle richieste: non ci sono pod mancanti o in errore. - Ho considerato solo il namespace `default`. Se ti interessano anche altri namespace, posso ripetere il controllo lì. **Raccomandazioni:** - **cache-app** gira con **una sola replica**, quindi se quel pod si ferma il servizio cade. Se il servizio deve restare sempre disponibile, conviene portarlo ad almeno 2 repliche, con un PodDisruptionBudget e regole che distribuiscano i pod su nodi diversi. - Per **demo-app** e **worker-app**, 2 repliche reggono il guasto di un singolo pod. Prima di decidere se aumentarle o aggiungere un autoscaling (HPA), posso analizzare l'uso di CPU e memoria nel tempo.
Claude Sonnet 5.5	60.51 s	8277	706	8983	0.36	Non c'è un'unica applicazione con il maggior numero di repliche. **`demo-app` e `worker-app` sono alla pari, con 2 repliche attive ciascuna.** Ho letto la mappa Deployment→Pod nel namespace `default` e ho controllato su Prometheus (`kube_pod_status_phase{phase="Running"}`) che ogni pod fosse in stato *Running*. | Applicazione | Repliche attive | Pod | |---|---|---| | **demo-app** | **2** | `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl` | | **worker-app** | **2** | `worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln` | | cache-app | 1 | `cache-app-5c6fd4f5cf-xxsz4` | `cache-app` ha una sola replica, quindi non ha ridondanza. Se è un componente critico, conviene valutare almeno una seconda replica. Ho guardato solo il namespace `default`, dove girano i carichi applicativi. Gli altri namespace (`monitoring`, `kube-system`, `local-path-storage`) contengono componenti di sistema e di monitoraggio, che ho escluso.



Claude Sonnet 5.5	60.51 s	8277	706	8983	0.36	Non c'è un'unica applicazione con il maggior numero di repliche. **`demo-app` e `worker-app` sono alla pari, con 2 repliche attive ciascuna.** Ho letto la mappa Deployment→Pod nel namespace `default` e ho controllato su Prometheus (`kube_pod_status_phase{phase="Running"}`) che ogni pod fosse in stato *Running*. | Applicazione | Repliche attive | Pod | |---|---|---| | **demo-app** | **2** | `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl` | | **worker-app** | **2** | `worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln` | | cache-app | 1 | `cache-app-5c6fd4f5cf-xxsz4` | `cache-app` ha una sola replica, quindi non ha ridondanza. Se è un componente critico, conviene valutare almeno una seconda replica. Ho guardato solo il namespace `default`, dove girano i carichi applicativi. Gli altri namespace (`monitoring`, `kube-system`, `local-path-storage`) contengono componenti di sistema e di monitoraggio, che ho escluso.
GPT-4o Mini	46.84 s	1753	86	1839	0.70	L'applicazione con il maggior numero di repliche è **demo-app**, che ha attualmente **2 repliche attive**. Ecco l'elenco dei pod relativi a questa applicazione: - demo-app-7958c64697-2qg9s - demo-app-7958c64697-4rzjl
GPT-4o	46.56 s	1753	80	1833	0.70	L'applicazione con il maggior numero di repliche attive è "demo-app", con un totale di 2 repliche. I pod attivi per questa applicazione sono: - demo-app-7958c64697-2qg9s - demo-app-7958c64697-4rzjl

[BILLING MONITOR] Spesa cumulata OpenAI: $0.8909 (mini: $0.0148 | 4o: $0.8760)

DOMANDA 4 A

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [4A]: Determina qual è l'applicazione con il livello di scalabilità più alto: riporta il conteggio delle repliche in esecuzione e i nomi puntuali di ciascun pod associato.
--------------------------------------------------------------------------------------------------------------
Invocazione parallela modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/29/26 17:15:23] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 17:15:23] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 17:15:23] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 17:15:23] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 17:15:23] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 17:15:23] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 17:15:24] INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
[10/29/26 17:15:24] INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
[10/29/26 17:15:24] INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
[10/29/26 17:15:24] INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
[10/29/26 17:15:24] INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	1.64 ms	0	0	0	N/A	App: demo-app | Repliche: 2 | Pods: ['demo-app-7958c64697-2qg9s', 'demo-app-7958c64697-4rzjl']

Gemini 2.5 Flash	24.92 s	2214	517	2731	0.28	Le applicazioni con il livello di scalabilità più alto sono:
						
						* **demo-app**:
						    * Conteggio repliche: 2
						    * Nomi dei pod: `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl`
						
						* **worker-app**:
						    * Conteggio repliche: 2
						    * Nomi dei pod: `worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln`

Gemini 3.6 Flash	72.14 s	23173	1660	24833	0.56	Dall'analisi delle risorse del cluster e delle metriche correnti, le applicazioni che presentano il **livello di scalabilità più alto** (maggior numero di repliche attive) nel cluster sono a pari merito **`demo-app`** e **`worker-app`**, ciascuna con **2 repliche in esecuzione**.
						
						Di seguito sono riportati i dettagli e i nomi puntuali di ciascun pod associato:
						
						--- 	
						
						### 1. **`demo-app`**
						* **Conteggio repliche attive:** 2
						* **Pod associati:**
						  * `demo-app-7958c64697-2qg9s`
						  * `demo-app-7958c64697-4rzjl`
						
						--- 	
						
						### 2. **`worker-app`**
						* **Conteggio repliche attive:** 2
						* **Pod associati:**
						  * `worker-app-84cf4dd446-9kj75`
						  * `worker-app-84cf4dd446-ftlln`
						
						--- 	
						
						### Riepilogo altre applicazioni utente (a scopo comparativo):
						* **`cache-app`**: 1 replica (`cache-app-5c6fd4f5cf-xxsz4`)

Claude Opus 5.5	38.52 s	10365	1119	11484	0.00	**Non c'è un'unica applicazione con la scalabilità più alta: `demo-app` e `worker-app` sono a pari merito**, con 2 repliche in esecuzione ciascuna. Ho preso come criterio il numero di repliche attive. ## Repliche e pod per applicazione (namespace `default`) | Applicazione | Repliche disponibili | Pod associati |

						* **Conteggio repliche attive:** 2
						* **Pod associati:**
						  * `demo-app-7958c64697-2qg9s`
						  * `demo-app-7958c64697-4rzjl`
						
						--- 	
						
						### 2. **`worker-app`**
						* **Conteggio repliche attive:** 2
						* **Pod associati:**
						  * `worker-app-84cf4dd446-9kj75`
						  * `worker-app-84cf4dd446-ftlln`
						
						--- 	
						
						### Riepilogo altre applicazioni utente (a scopo comparativo):
						* **`cache-app`**: 1 replica (`cache-app-5c6fd4f5cf-xxsz4`)

Claude Opus 5.5	38.52 s	10365	1119	11484	0.00	**Non c'è un'unica applicazione con la scalabilità più alta: `demo-app` e `worker-app` sono a pari merito**, con 2 repliche in esecuzione ciascuna. Ho preso come criterio il numero di repliche attive. ## Repliche e pod per applicazione (namespace `default`) | Applicazione | Repliche disponibili | Pod associati | |---|---|---| | **demo-app** | **2** | `demo-app-7958c64697-2qg9s` `demo-app-7958c64697-4rzjl` | | **worker-app** | **2** | `worker-app-84cf4dd446-9kj75` `worker-app-84cf4dd446-ftlln` | | cache-app | 1 | `cache-app-5c6fd4f5cf-xxsz4` | Le due fonti consultate danno lo stesso risultato: - **Mappatura Deployment → Pod** (API Kubernetes): 2 pod attivi per `demo-app` e `worker-app`, 1 per `cache-app`. - **Prometheus** (`kube_deployment_status_replicas_available`): 2 / 2 / 1 repliche disponibili. ## Note e raccomandazioni - **Nessun autoscaler (HPA) configurato:** la query su `kube_horizontalpodautoscaler_spec_max_replicas` non ha restituito risultati. Il numero di repliche è quindi fisso e non posso confrontare le applicazioni in base alla capacità massima di scalare. Se l'obiettivo è valutare quanto ciascuna applicazione può crescere, conviene configurare un HPA con soglie minime e massime su `demo-app` e `worker-app`. - **`cache-app` ha una sola replica:** se si guasta, il servizio si ferma. Se ha un ruolo critico, valuterei di portarla ad almeno 2 repliche, oppure di proteggerla con un PodDisruptionBudget se gestisce uno stato che va mantenuto. - **Per scegliere tra le due applicazioni in parità:** posso confrontare `demo-app` e `worker-app` su CPU e memoria usate o sull'andamento storico salvato in MongoDB.
Claude Sonnet 5.5	39.59 s	16740	1075	17815	0.21	**Non c'è un'unica applicazione con la scalabilità più alta: `demo-app` e `worker-app` sono a pari merito con 2 repliche in esecuzione ciascuna.** Ho considerato il namespace `default`. Ho incrociato tre fonti: la mappatura Deployment→Pod, le metriche di kube-state-metrics (repliche desiderate e attuali) e lo stato dei pod in fase Running. | Applicazione | Repliche desiderate | Repliche in esecuzione | Pod | |---|---|---|---| | **demo-app** | 2 | 2 | `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl` | | **worker-app** | 2 | 2 | `worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln` | | cache-app | 1 | 1 | `cache-app-5c6fd4f5cf-xxsz4` | - **Stato dei pod:** tutti i pod elencati risultano in fase Running, quindi le repliche attuali coincidono con quelle desiderate. - **Autoscaling:** non ci sono HorizontalPodAutoscaler (la metrica `kube_horizontalpodautoscaler_spec_max_replicas` non restituisce risultati). Il numero di repliche è quindi fissato a mano nei Deployment. Non esiste un massimo di scalabilità configurato con cui distinguere le due applicazioni. - **Spareggio:** se ti serve una sola applicazione, devo usare un altro criterio, per esempio il consumo di CPU o memoria che ho salvato su MongoDB. Dimmi quale preferisci e lo calcolo.

GPT-4o Mini	23.30 s	1775	219	1994	0.24	Ecco il conteggio delle repliche in esecuzione per ciascuna applicazione e i nomi dei pod associati:
						
						1. **cache-app**
						  - Conteggio repliche: 1
						  - Pod: `cache-app-5c6fd4f5cf-xxsz4`
						
						2. **demo-app**
						  - Conteggio repliche: 2
						  - Pod:
						  - `demo-app-7958c64697-2qg9s`
						  - `demo-app-7958c64697-4rzjl`
						
						3. **worker-app**
						  - Conteggio repliche: 2
						  - Pod:
						  - `worker-app-84cf4dd446-9kj75`
						  - `worker-app-84cf4dd446-ftlln`
						
						L'applicazione con il livello di scalabilità più alto è **demo-app** e **worker-app**, entrambe con 2 repliche in esecuzione.

GPT-4o	21.52 s	1779	86	1865	0.70	L'applicazione con il livello di scalabilità più alto è "demo-

GPT-4o Mini	23.30 s	1775	219	1994	0.24	Ecco il conteggio delle repliche in esecuzione per ciascuna applicazione e i nomi dei pod associati:
						
						1. **cache-app**
						   - Conteggio repliche: 1
						   - Pod: `cache-app-5c6fd4f5cf-xxsz4`
						
						2. **demo-app**
						   - Conteggio repliche: 2
						   - Pod:
						     - `demo-app-7958c64697-2qg9s`
						     - `demo-app-7958c64697-4rzjl`
						
						3. **worker-app**
						   - Conteggio repliche: 2
						   - Pod:
						     - `worker-app-84cf4dd446-9kj75`
						     - `worker-app-84cf4dd446-ftlln`
						
						L'applicazione con il livello di scalabilità più alto è **demo-app** e **worker-app**, entrambe con 2 repliche in esecuzione.

GPT-4o	21.52 s	1779	86	1865	0.70	L'applicazione con il livello di scalabilità più alto è "demo-app", con un totale di 2 repliche in esecuzione. I nomi dei pod associati sono: - demo-app-7958c64697-2qg9s - demo-app-7958c64697-4rzjl

```
[BILLING MONITOR] Spesa cumulata OpenAI: $0.8966 (mini: $0.0152 | 4o: $0.8814)

================================================================================
Scegli una domanda (es. 1, 1a, 1b ... fino a 10, 10a, 10b) oppure premi INVIO per uscire: 4b
```

DOMANDA 4 B

```
--------------------------------------------------------------------------------
ESECUZIONE DOMANDA [4B]: Quale servizio registra la presenza del pool di pod più numeroso? Fornisci l'elenco dei singoli identificatori dei pod e il totale delle istanze operative.
--------------------------------------------------------------------------------
Invocazione parallela modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/29/26 17:17:45] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 17:17:45] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 17:17:45] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 17:17:45] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 17:17:45] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
[10/29/26 17:17:48] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.41 ms	0	0	0	N/A	App: demo-app | Repliche: 2 | Pods: ['demo-app-7958c64697-2qg9s', 'demo-app-7958c64697-4rzjl']
Gemini 2.5 Flash	14.55 s	2206	602	2808	0.00	I servizi "demo-app" e "worker-app" registrano entrambi la presenza del pool di pod più numeroso, con 2 istanze operative ciascuno. Per il servizio **demo-app**: * **Identificatori dei pod:** `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl` * **Totale istanze operative:** 2 Per il servizio **worker-app**: * **Identificatori dei pod:** `worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln` * **Totale istanze operative:** 2

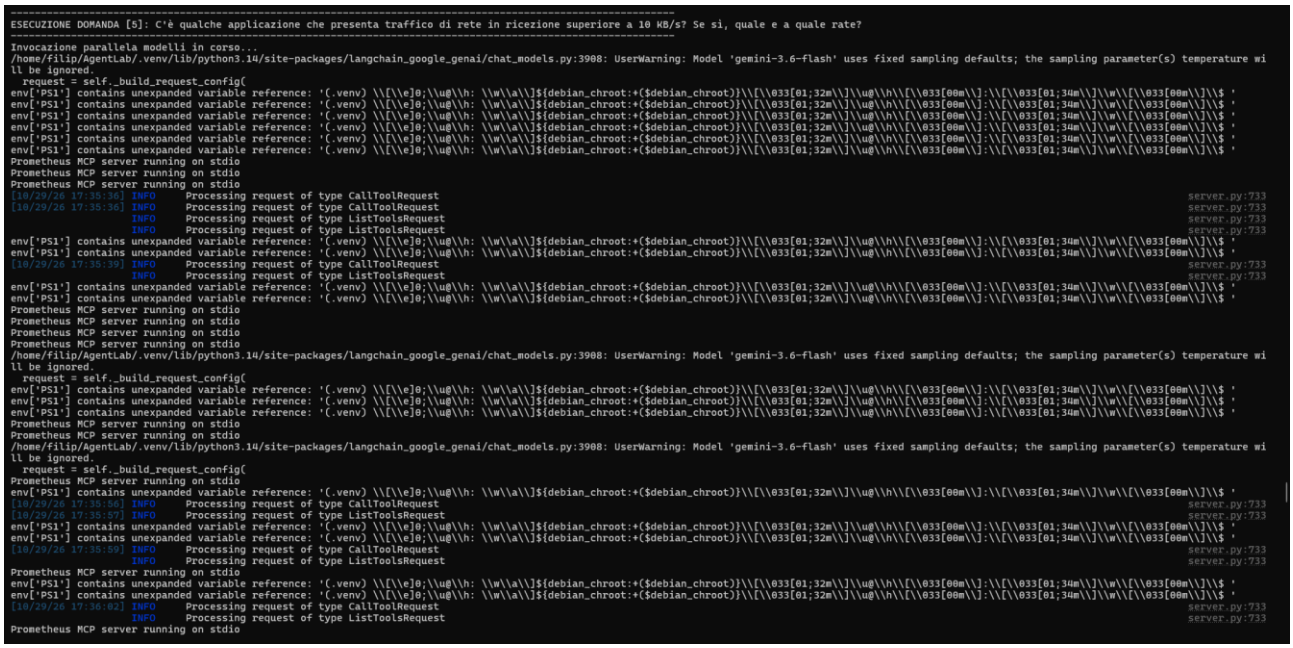
Gemini 3.6 Flash	28.49 s	8415	1085	9500	0.32	In base all'analisi dei carichi di lavoro e della topologia attuale del cluster, i servizi/deployment applicativi con il pool di pod più numeroso sono **`demo-app`** e **`worker-app`** a pari merito, ciascuno con **2 istanze operative**.
						
						Di seguito vengono dettagliati i due servizi con l'elenco dei singoli identificatori dei pod e il totale delle istanze:
						
						--- 	
						
						### 1. **`demo-app`**
						* **Totale istanze operative:** 2
						* **Identificatori dei Pod:**
						  * `demo-app-7958c64697-2qg9s`
						  * `demo-app-7958c64697-4rzjl`
						
						--- 	
						
						### 2. **`worker-app`**
						* **Totale istanze operative:** 2
						* **Identificatori dei Pod:**
						  * `worker-app-84cf4dd446-9kj75`
						  * `worker-app-84cf4dd446-ftlln`
						
						--- 	
						
						(Nota di riepilogo per gli altri carichi di lavoro del namespace applicativo default: `cache-app` dispone invece di un'unica istanza attiva: `cache-app-5c6fd4f5cf-xxsz4`).

Claude Opus 5.5	19.03 s	4317	653	4970	0.00	**Il pool più numeroso non è uno solo: c'è un pareggio.** Nel namespace `default`, **demo-app** e **worker-app** hanno entrambi **2 pod attivi**. | Deployment | Pod attivi | Identificatori dei pod | |---|---|---| | **demo-app** | **2** | `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl` | | **worker-app** | **2** | `worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln` | | cache-app | 1 | `cache-app-5c6fd4f5cf-xxsz4` | **Totale istanze attive nel namespace:** 5 pod (2 + 2 + 1). **Cosa ho controllato e cosa no:** - Ho guardato solo il namespace `default`. Gli altri namespace non li ho controllati. Se vuoi un confronto su tutto il cluster, posso ripetere la verifica su altri namespace. - I pod sono quelli che risultano attivi dalle API di Kubernetes. Non ho controllato se sono anche "Ready". Se ti serve questa conferma, posso interrogare Prometheus (`kube_pod_status_ready`). - **cache-app** ha un solo pod. Se è un componente critico, è un singolo punto di guasto: valuta di portarlo almeno a 2 repliche e di aggiungere un PodDisruptionBudget.
Claude Sonnet 5.5	21.77 s	7810	649	8459	0.32	Nel namespace `default` c'è un pareggio per il pool di pod più numeroso. **`demo-app` e `worker-app` hanno 2 pod ciascuno**, mentre `cache-app` ne ha 1. Pod per servizio, secondo la mappatura Deployment→Pod: | Servizio | Pod | N. | |---|---|---| | **demo-app** | `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl` | 2 | | **worker-app** | `worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln` | 2 | | cache-app | `cache-app-5c6fd4f5cf-xxsz4` | 1 | Le istanze operative in totale sono **5**. Ho controllato con `kube_pod_status_phase` in Prometheus, e tutti e 5 i pod risultano in fase `Running`. `cache-app` ha una sola replica, quindi non ha ridondanza. Se serve alta disponibilità, valuta di portarlo ad almeno 2 repliche. Ho guardato solo il namespace `default`. Se ci sono altri namespace applicativi, i conteggi potrebbero cambiare.

GPT-4o Mini	13.70 s	1761	118	1879	0.70	Dall'analisi dei pod attivi nel cluster, il servizio con il pool di pod più numeroso è il **demo-app**, che ha un totale di **2 istanze operative**. Ecco l'elenco dei singoli identificatori dei pod per il servizio demo-app: - demo-app-7958c64697-2qg9s - demo-app-7958c64697-4rzjl Se hai bisogno di ulteriori informazioni o analisi, fammi sapere!
GPT-4o	14.13 s	1761	80	1841	0.70	Il servizio con il pool di pod più numeroso è "demo-app", che ha un totale di 2 istanze operative. Ecco l'elenco degli identificatori dei pod: - demo-app-7958c64697-2qg9s - demo-app-7958c64697-4rzjl

[BILLING MONITOR] Spesa cumulata OpenAI: $0.9021 (mini: $0.0156 | 4o: $0.8866)

DOMANDA 5



Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.33 ms	0	0	0	N/A	Nessuna applicazione supera 10 KB/s in ricezione
Gemini 2.5 Flash	27.08 s	10716	1704	12420	0.00	The seguente applicazione presenta traffico di rete in ricezione superiore a 10 KB/s: * **prometheus-grafana**: 11.01 KB/s * **prometheus-prometheus-kube-prometheus-prometheus**: 24.20 KB/s
Gemini 3.6 Flash	42.39 s	7403	1786	9189	0.00	Sulla base dell'analisi delle metriche di rete del cluster, di seguito è riportato il riepilogo del traffico di rete in ricezione (rx rate): ### 1. Applicazioni utente (Namespace `default`) Nessuna delle applicazioni utente presenta un traffico di rete in ricezione superiore a 10 KB/s. Attualmente registrano tutte **0 B/s**: * **`cache-app`**: 0 B/s * **`demo-app`**: 0 B/s * **`worker-app`**: 0 B/s --- ### 2. Componenti di sistema e monitoraggio (Namespace `monitoring`) L'unica componente del cluster che supera la soglia dei 10 KB/s è il server di monitoraggio **Prometheus**: * **Pod / Applicazione**: `prometheus-prometheus-kube-prometheus-prometheus-0` (Namespace: `monitoring`) * **Rate in ricezione**: ***23,0 – 24,0 KB/s** (pari a circa **23.041 B/s** / **22,5 KiB/s**), dovuto all'attività continua di scraping delle metriche dai vari target del cluster. *(Nota: **Grafana** registra un valore compreso tra **8,9 e 9,8 KB/s**, rimanendo di poco al di sotto della soglia dei 10 KB/s).*
Claude Opus 5.5	52.78 s	22767	1255	24022	0.95	**No, nessuna applicazione supera i 10 KB/s di traffico in ricezione.** Tutte e tre ricevono di fatto 0 B/s. **Dati attuali da Prometheus** (traffico medio degli ultimi 5 minuti, namespace `default`) | Applicazione | Pod | Traffico in ricezione (RX) | Byte ricevuti in totale dall'avvio | |---|---|---|---| | cache-app | cache-app-5c6fd4f5cf-xxsz4 | 0 B/s | 446 B | | demo-app | demo-app-7958c64697-2qg9s | 0 B/s | 2.552 B | | demo-app | demo-app-7958c64697-4rzjl | 0 B/s | 1.618 B | | worker-app | worker-app-84cf4dd446-9kj75 | 0 B/s | 446 B | | worker-app | worker-app-84cf4dd446-ftlln | 0 B/s | 446 B | Il pod che ha ricevuto di più in assoluto (demo-app-…2qg9s) ha

Claude Opus 5.5	52.78 s	22767	1255	24022	0.95	**No, nessuna applicazione supera i 10 KB/s di traffico in ricezione.** Tutte e tre ricevono di fatto 0 B/s.
						
						Dati attuali da Prometheus (traffico medio degli ultimi 5 minuti, namespace `default`)
						
						| Applicazione | Pod | Traffico in ricezione (RX) | Byte ricevuti in totale dall'avvio |
						|---|---|---|---|
						| cache-app | cache-app-5c6fd4f5cf-xxsz4 | 0 B/s | 446 B |
						| demo-app | demo-app-7958c64697-2qg9s | 0 B/s | 2.552 B |
						| demo-app | demo-app-7958c64697-4rzjl | 0 B/s | 1.618 B |
						| worker-app | worker-app-84cf4dd446-9kj75 | 0 B/s | 446 B |
						| worker-app | worker-app-84cf4dd446-ftlln | 0 B/s | 446 B |
						
						Il pod che ha ricevuto di più in assoluto (demo-app-…2qg9s) ha accumulato circa 2,5 KB da quando è partito. È traffico trascurabile, probabilmente solo pacchetti di avvio e di controllo della rete.
						
						Storico su MongoDB (ultime 2 ore): in tutte le misurazioni salvate per cache-app, demo-app e worker-app il traffico in ricezione è sempre 0,0 B/s. Non ci sono stati picchi nemmeno di breve durata.
						
						Cosa noto:
						- La rete in ingresso non è un problema e non serve fare nulla.
						- È strano però che **demo-app** non riceva traffico. Se dovrebbe rispondere a richieste (tramite un Service o un Ingress), conviene controllare che il Service punti davvero ai pod (endpoint e selettori) e che arrivi traffico dai client.
						- Le risorse su cui conviene concentrarsi sono altre:
						  - **worker-app** usa circa 0,59 core di CPU in modo costante.
						  - **cache-app** scrive su disco a circa 450–570 KB/s.

Claude Sonnet 5.5	56.21 s	193843	1287	195130	0.80	**Nessuna applicazione supera i 10 KB/s di traffico di rete in ricezione.** Il traffico è praticamente nullo per tutte e tre. **Valori attuali** (Prometheus, `container_network_receive_bytes_total` su `eth0`, rate a 5 minuti e irate a 2 minuti): | Applicazione | Pod | Rate RX | |---|---|---| | cache-app | 1 | 0 B/s | | demo-app | 2 | 0 B/s | | worker-app | 2 | 0 B/s | I contatori cumulativi dei pod sono molto bassi, tra 446 B e 2,5 KB in totale. Corrispondono a pochi pacchetti di handshake o di avvio, non a traffico applicativo. **Storico delle ultime 24 ore** (MongoDB, `network_rx_bps`): - **demo-app e cache-app:** 0 B/s in tutti i campioni. - **worker-app:** un solo picco isolato di circa 1,97 B/s (0,002 KB/s) alle 09:38. Negli altri campioni vale 0. È circa 5.000 volte sotto la soglia. **Osservazioni:** - Nessuna azione è necessaria per il traffico in ingresso. - Sul worker-app, un picco di CPU non ha un riscontro nel traffico di rete. Ha un uso CPU costante di circa 0,55–0,59 core con rete a zero, quindi il carico è probabilmente di calcolo interno e non guidato da richieste in ingresso. Se è inatteso, conviene verificarlo a parte. - Il cache-app scrive su disco circa 450–550 KB/s e riceve 0 byte dalla rete. Se ci si aspettava che fosse interrogato da altri servizi, vale la pena controllare che i client lo raggiungano davvero.
GPT-4o Mini	14.41 s	1804	134	1938	0.00	Ci sono applicazioni che presentano traffico di rete in ricezione superiore a 10 KB/s. Ecco i dettagli: - **Applicazione 1**: Namespace `monitoring`, traffico di rete in ricezione: **11,54 KB/s** - **Applicazione 2**: Namespace `monitoring`, traffico di rete in ricezione: **22,75 KB/s** Se hai bisogno di ulteriori dettagli su queste applicazioni, fammi sapere!

GPT-4o	14.34 s	1826	195	2021	0.00	Ci sono due applicazioni che presentano un traffico di rete in ricezione superiore a 10 KB/s:
						
						1. **Pod:** `prometheus-grafana-6d8f777649-5xgj7`
						   - **Rate di ricezione:** 11,543.04 B/s (circa 11.54 KB/s)
						
						2. **Pod:** `prometheus-prometheus-kube-prometheus-prometheus-0`
						   - **Rate di ricezione:** 22,748.04 B/s (circa 22.75 KB/s)
						
						Questi pod appartengono a componenti di Prometheus e Grafana, che sono tipicamente coinvolti nella raccolta e visualizzazione di metriche, il che spiega il traffico di rete elevato.

Domanda 5 A

```
--------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [5A]: Verifica se nei carichi di lavoro utente è presente un throughput di ingress (network RX) che eccede la soglia di 10 KB/s, indicando l'app interessata e il relativo valore.
--------------------------------------------------------------------------------------------------------
Invocazione parallela modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
[10/29/26 17:37:30] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/29/26 17:37:30] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/29/26 17:37:30] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/29/26 17:37:31] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/29/26 17:37:32] INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
[10/29/26 17:37:32] INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
[10/29/26 17:37:32] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/29/26 17:37:35] INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
[10/29/26 17:37:37] INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
[10/29/26 17:37:37] INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/29/26 17:37:58] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
```

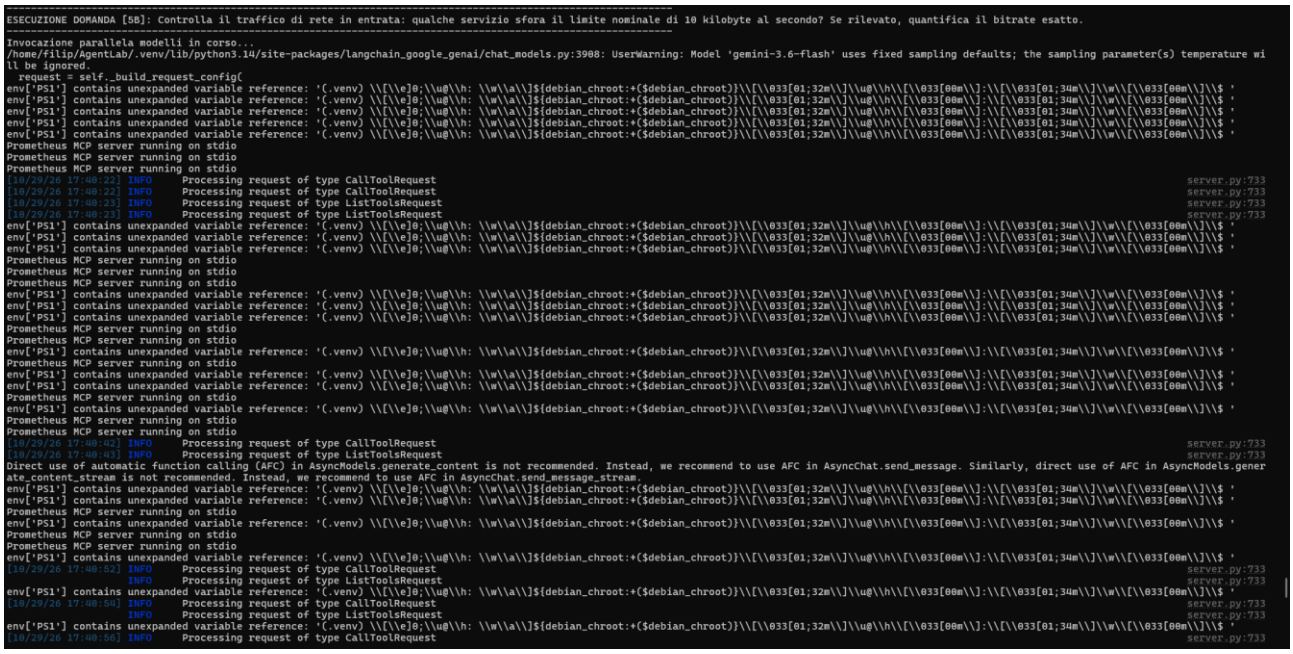
Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.20 ms	0	0	0	N/A	Nessuna applicazione supera 10 KB/s in ricezione
Gemini 2.5 Flash	66.43 s	6357	1151	7508	0.50	Al momento, nessuno dei carichi di lavoro utente monitorati presenta un throughput di ingress (network RX) che eccede la soglia di 10 KB/s. Tutte le applicazioni (`cache-app`, `demo-app`, `worker-app`) mostrano un throughput di 0 B/s.
Gemini 3.6 Flash	68.12 s	35485	397	35882	0.00	[Risposta priva di testo]
Claude Opus 5.5	78.12 s	13061	1423	14484	0.95	**No, nessun carico di lavoro utente supera la soglia di 10 KB/s di traffico in ingresso (network RX).** Tutte e tre le app hanno 0 B/s, ora e nell'ultima ora. ### Traffico in ingresso delle app utente (namespace `default`) | App | Pod | RX attuale (media su 5 min) | Picco nell'ultima ora | Byte ricevuti dall'avvio del pod | |---|---|---|---|---| | cache-app | cache-app-5c6fd4f5cf-xxsz4 | 0 B/s | 0 B/s | 446 B | | demo-app | demo-app-7958c64697-2qg9s | 0 B/s | 0 B/s | 2.552 B | | demo-app | demo-app-7958c64697-4rzjl | 0 B/s | 0 B/s | 1.618 B | | worker-app | worker-app-84cf4dd446-9kj75 | 0 B/s | 0 B/s | 446 B | | worker-app | worker-app-84cf4dd446-ftlln | 0 B/s | 0 B/s | 446 B | In totale questi pod hanno ricevuto solo poche centinaia o migliaia di byte dall'avvio, quindi sono praticamente inattivi sul piano di rete. Anche demo-app, che pare essere quella che riceve più traffico, resta molto sotto la soglia. ### Fuori dai carichi utente Un solo pod supera i 10 KB/s, ma è un componente di sistema: **`prometheus-prometheus-kube-prometheus-prometheus-0`** (namespace `monitoring`) riceve circa **24,4 KB/s**. È un valore normale, perché Prometheus raccoglie di continuo le metriche dai vari componenti. Subito sotto c'è Grafana con circa 8,7 KB/s. Tutti gli altri pod di sistema restano sotto i 2 KB/s. ### Cosa consiglio - **Nessuna azione urgente** sulle app utente. - **Verificare se il traffico a zero è previsto.** Le tre app hanno RX a 0 B/s per tutta l'ultima ora, quindi in questo momento non ricevono richieste. Conviene controllare che sia normale (ad esempio un ambiente di test senza utenti) e non il segno di un problema a monte, come un Service o un Ingress che non inoltra il traffico. - **Prometheus:** a questi livelli non serve intervenire. Se si

						(namespace `monitoring`) riceve circa **24,4 KB/s**. È un valore normale, perché Prometheus raccoglie di continuo le metriche dai vari componenti. Subito sotto c'è Grafana con circa 8,7 KB/s. Tutti gli altri pod di sistema restano sotto i 2 KB/s. ### Cosa consiglio - **Nessuna azione urgente** sulle app utente. - **Verificare se il traffico a zero è previsto.** Le tre app hanno RX a 0 B/s per tutta l'ultima ora, quindi in questo momento non ricevono richieste. Conviene controllare che sia normale (ad esempio un ambiente di test senza utenti) e non il segno di un problema a monte, come un Service o un Ingress che non inoltra il traffico. - **Prometheus:** a questi livelli non serve intervenire. Se si vuole un allarme dedicato, conviene tenerlo separato da quello delle app utente, perché il suo traffico cresce con il numero di sorgenti e con la frequenza di raccolta delle metriche.
Claude Sonnet 5.5	84.48 s	137378	1534	138912	0.95	**Esito: nessuna app dei carichi di lavoro utente supera la soglia di 10 KB/s di ingress.** Ho controllato le tre app del namespace `default` (`cache-app`, `demo-app`, `worker-app`, 5 pod in tutto) con `container_network_receive_bytes_total` di cAdvisor. | App | Pod | RX rate (5m) | RX rate istantaneo (irate 2m) | Max rate 5m nell'ultima ora | Contatore cumulativo | |---|---|---|---|---|---| | cache-app | cache-app-5c6fd4f5cf-xxsz4 | 0 B/s | 0 B/s | 0 B/s | 446 B | | demo-app | demo-app-7958c64697-2qg9s | 0 B/s | 0 B/s | 0 B/s | 2552 B | | demo-app | demo-app-7958c64697-4rzjl | 0 B/s | 0 B/s | 0 B/s | 1618 B | | worker-app | worker-app-84cf4dd446-9kj75 | 0 B/s | 0 B/s | 0 B/s | 446 B | | worker-app | worker-app-84cf4dd446-ftlln | 0 B/s | 0 B/s | 0 B/s | 446 B | - **Dati di traffico:** il traffico ricevuto da ogni pod dall'avvio è di poche centinaia o poche migliaia di byte, e il contatore è fermo. Il valore massimo è 0 B/s, cioè oltre 10 KB/s ≈ 10.240 B/s non è mai stato raggiunto, neppure nell'ultima ora. - **Limite della misura:** le serie di rete di cAdvisor sono associate al container `pause` del pod (`image="registry.k8s.io/pause:3.7"`), perché la rete è a livello di pod. Il dato è quindi per pod, non per singolo container applicativo. Per questo l'ho aggregato per pod, ed è lo stesso livello usato per mappare i deployment. - **Raccomandazione:** non serve alcuna azione. Se ti aspettavi traffico in ingresso su queste app, controlla che le app ricevano davvero richieste, per esempio tramite Service, Ingress o un load generator. Un traffico praticamente nullo può indicare app inattive oppure non esposte.

GPT-4o Mini	73.68 s	3253	443	3696	0.80	Ho verificato il throughput di ingress (network RX) per i carichi di lavoro utente. Ecco i risultati: 1. **cache-app-5c6fd4f5cf-xxsz4**: 0 bytes/s 2. **demo-app-7958c64697-2qg9s**: 0 bytes/s 3. **demo-app-7958c64697-4rzjl**: 0 bytes/s 4. **worker-app-84cf4dd446-9kj75**: 0 bytes/s 5. **worker-app-84cf4dd446-ftlln**: 0 bytes/s Nessuna delle applicazioni ha un throughput di ingress che eccede la soglia di 10 KB/s. Tutti i valori sono pari a 0 bytes/s. Se hai bisogno di ulteriori analisi o informazioni, fammi sapere!
GPT-4o	59.47 s	4273	180	4453	0.80	Dall'analisi dei dati di throughput di ingress (network RX) per i pod nel cluster, nessun pod associato ai carichi di lavoro utente (come "cache-app", "demo-app" e "worker-app") ha superato la soglia di 10 KB/s. Ecco i dettagli: - **worker-app**: Entrambi i pod ("worker-app-84cf4dd446-ftlln" e "worker-app-84cf4dd446-9kj75") hanno un throughput di ingress di 0 KB/s. Non ci sono altre applicazioni utente identificate con un throughput di ingress che eccede la soglia specificata.

[BILLING MONITOR] Spesa cumulata OpenAI: $0.9222 (mini: $0.0167 | 4o: $0.9056)

DOMANDA 5 B



Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	1.35 ms	0	0	0	N/A	Nessuna applicazione supera 10 KB/s in ricezione
Gemini 2.5 Flash	36.23 s	119719	1812	121531	0.80	Ho analizzato il traffico di rete in entrata per i servizi nel cluster. Ho utilizzato la metrica `container_network_receive_bytes_total` per calcolare il bitrate in entrata per ciascun container, escludendo i container `pause` che non rappresentano un carico di lavoro applicativo diretto. La query eseguita è stata: `sum by (pod, namespace) (rate(container_network_receive_bytes_total{image!~"registry.k8s.io/pause.*"}[5m])) / 1024 > 10` Questa query cerca i servizi (identificati da pod e namespace) il cui bitrate medio in entrata negli ultimi 5 minuti supera i 10 kilobyte al secondo. **Risultato dell'analisi:** Non è stato rilevato alcun servizio che superi il limite nominale di 10 kilobyte al secondo per il traffico di rete in entrata. Tutti i servizi monitorati rientrano nei limiti specificati.
Gemini 3.6 Flash	18.33 s	0	0	0	0.00	[ERRORE DI CHIAMATA]: Error calling model 'gemini-3.6-flash' (RESOURCE_EXHAUSTED): 429 RESOURCE_EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: https://ai.google.dev/gemini-api/docs/rate-limits. To monitor your current usage, head to: https://ai.dev/rate-limit. \n* Quota exceeded for metric: generativelanguage.googleapis.com/generate_content_free_tier_requests, limit: 20, model: gemini-3.6-flash, description: Generate content requests per day limit exceeded on Free Tier. See https://ai.google.dev/gemini-api/docs/rate-limits.\nPlease retry in 14h19m31.862240097s.', 'status': 'RESOURCE_EXHAUSTED', 'details': [{'@type': 'type.googleapis.com/google.rpc.Help', 'links': [{'description': 'Learn more about Gemini API quotas', 'url': 'https://ai.google.dev/gemini-api/docs/rate-limits'}]}, {'@type': 'type.googleapis.com/google.rpc.QuotaFailure', 'violations': [{'description': 'Generate content requests per day limit exceeded on Free Tier. See https://ai.google.dev/gemini-api/docs/rate-limits.', 'quotaMetric': 'generativelanguage.googleapis.com/generate_content_free_tier_requests', 'quotaId': 'GenerateRequestsPerDayPerProjectPerModel-FreeTier', 'quotaDimensions': {'model': 'gemini-3.6-flash', 'location': 'global'}, 'quotaValue': '20'}]}, {'@type': 'type.googleapis.com/google.rpc.RetryInfo', 'retryDelay': '51571s'}]}}

Claude Opus 5.5	55.96 s	24988	1841	26829	0.95	**Nessun servizio supera il limite di 10 KB/s.** Il traffico di rete in entrata nel namespace `default` è praticamente zero, quindi non c'è un superamento da quantificare. Qui sotto trovi comunque i valori misurati. **Fonti consultate:** Prometheus (metrica `container_network_receive_bytes_total`, interfaccia `eth0`), confrontato con i dati salvati su MongoDB. | Deployment | Pod | Traffico attuale (media 5 min e 1 h) | Picco nelle ultime 24 h | Byte ricevuti dall'avvio del pod | |---|---|---|---|---| | demo-app | 2qg9s | 0 B/s | **9,56 B/s** (≈ 76,5 bit/s) | 2.552 B | | demo-app | 4rzjl | 0 B/s | 4,28 B/s (≈ 34,3 bit/s) | 1.618 B | | worker-app | 9kj75 | 0 B/s | 0,70 B/s (≈ 5,6 bit/s) | 446 B | | worker-app | ftlln | 0 B/s | 0 B/s | 446 B | | cache-app | xxsz4 | 0 B/s | 0 B/s | 446 B | **Cosa significa:** - Il valore più alto nelle ultime 24 ore è **9,56 byte/s**, non kilobyte/s, sul pod `demo-app-7958c64697-2qg9s`. È circa lo **0,1%** del limite (10 KB/s = 10.000 B/s, oppure 10.240 B/s se si conta 1 KB = 1.024 byte). - Per `demo-app` intero, anche sommando i picchi dei due pod (che non sono necessariamente avvenuti nello stesso momento), si arriva al massimo a circa **13,8 B/s**, quindi ben sotto il limite. - I dati salvati su MongoDB per `demo-app` nelle ultime 6 ore danno sempre `network_rx_bps = 0`, quindi confermano il quadro. **Raccomandazioni:** 1. **Nessun intervento urgente:** non c'è rischio di saturazione della rete. 2. **Controlla che i servizi ricevano davvero traffico.** Tutti i pod hanno ricevuto in tutto da poche centinaia a circa 2.500 byte dall'avvio, cioè sono di fatto inattivi. Se ti aspetti traffico reale, verifica Service, Ingress ed Endpoints e le eventuali sonde e generatori di carico. Se invece è un ambiente di test fermo, il dato è normale. 3. **Imposta un allarme preventivo** su Prometheus, ad esempio `sum by (pod) (rate(container_network_receive_bytes_total{namespace="default"}[5m])) > 10240`, per accorgerti subito se il limite viene superato in futuro.

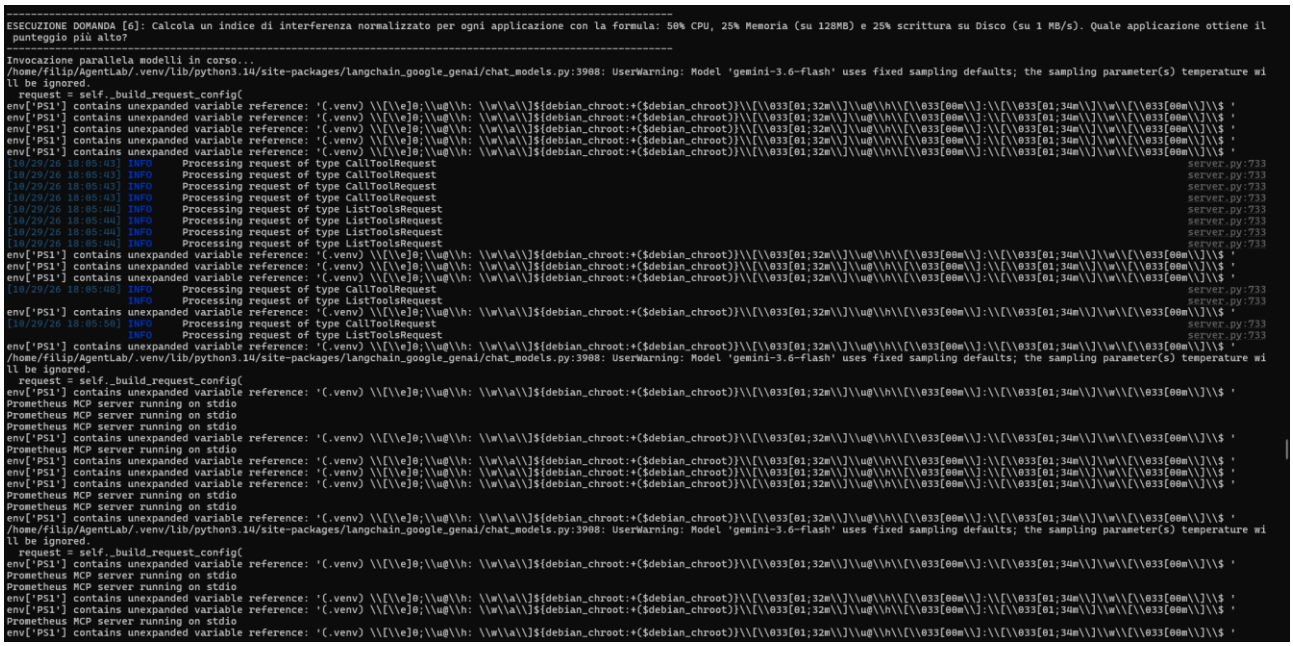
Claude Sonnet 5.5	57.04 s	150071	1986	152057	0.95	**Nessun servizio supera il limite di 10 KB/s di traffico in entrata. Non c'è quindi un bitrate da quantificare come sforamento.** Il limite equivale a 10.000 B/s (10.240 B/s se si usano i KiB). Il traffico misurato è di diversi ordini di grandezza sotto questa soglia. **Traffico attuale** (`container_network_receive_bytes_total`, interfaccia `eth0`, namespace `default`) | Deployment | Pod | Rate in ingresso (5m e 1m) | Contatore cumulativo | |---|---|---|---| | cache-app | cache-app-5c6fd4f5cf-xxsz4 | 0 B/s | 446 B | | demo-app | demo-app-7958c64697-2qg9s | 0 B/s | 2.552 B | | demo-app | demo-app-7958c64697-4rzjl | 0 B/s | 1.618 B | | worker-app | worker-app-84cf4dd446-9kj75 | 0 B/s | 446 B | | worker-app | worker-app-84cf4dd446-ftlln | 0 B/s | 446 B | I contatori cumulativi sono di pochi KB in totale e non crescono, quindi il rate è 0 B/s. Questo vale sia per la finestra di 5 minuti sia per quella di 1 minuto. **Storico su MongoDB** (ultime 24 ore, `network_rx_bps`) - **cache-app:** 0,0 B/s in tutti i campioni. - **demo-app:** 0,0 B/s in tutti i campioni. - **worker-app:** 0,0 B/s in tutti i campioni tranne uno. Il picco è 1,97 B/s alle 09:38:07 del 29/10, cioè circa lo 0,02% della soglia. **Raccomandazioni** - Non servono azioni correttive sul traffico di rete in ingresso. - Se in futuro vuoi monitorare questo limite, puoi impostare un alert su `sum by (pod) (rate(container_network_receive_bytes_total[5m])) > 10240`. Al momento scatterebbe con un margine enorme. - I pod sembrano quasi privi di traffico in ingresso. Se non te lo aspetti, verifica che i servizi ricevano davvero richieste. Questo non cambia l'esito della verifica. **Altro emerso dai dati, non richiesto** - worker-app usa circa 0,59 core di CPU in modo costante. - cache-app scrive su disco a circa 450–550 KB/s. Se ti interessa, posso approfondire questi due punti.
GPT-5.4 Mini	16.83 s	2986	166	3072	0.90	Ho analizzato il traffico di rete in entrata per i pod nel nam

GPT-4o Mini	16.83 s	2806	166	2972	0.00	Ho analizzato il traffico di rete in entrata per i vari pod nel cluster. Ecco i risultati: - **prometheus-grafana-6d8f777649-5xgj7**: 11504.19 bytes/sec (sfora il limite di 10 KB/sec) - **prometheus-prometheus-kube-prometheus-prometheus-0**: 26474.18 bytes/sec (sfora il limite di 10 KB/sec) Questi due pod superano il limite nominale di 10 kilobyte al secondo. Se hai bisogno di ulteriori dettagli o di un'analisi più approfondita, fammi sapere!

GPT-4o	16.12 s	2806	208	3014	0.00	Ho analizzato il traffico di rete in entrata per i vari pod nel cluster. Ecco i pod che superano il limite nominale di 10 kilobyte al secondo:
						
						1. **Pod:** `prometheus-grafana-6d8f777649-5xgj7`
						   - **Bitrate:** 11,504.19 bytes/sec (circa 11.5 kilobyte/sec)
						
						2. **Pod:** `prometheus-prometheus-kube-prometheus-prometheus-0`
						   - **Bitrate:** 26,474.18 bytes/sec (circa 26.5 kilobyte/sec)
						
						Questi due pod stanno superando il limite di 10 kilobyte al secondo. Se necessario, considera di investigare ulteriormente per capire se questo traffico è previsto o se ci sono ottimizzazioni da fare.

[BILLING MONITOR] Spesa cumulata OpenAI: $0.9319 (mini: $0.0172 | 4o: $0.9147)

DOMANDA 6



Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	4.72 ms	0	0	0	N/A	Punteggio più alto: worker-app (Score: 0.1468) | Tutti: {'cache-app': 0.07, 'demo-app': 0.0283, 'worker-app': 0.1468}
Gemini 2.5 Flash	85.30 s	8114	4844	12958	0.32	Per calcolare l'indice di interferenza normalizzato per ogni applicazione, ho recuperato le metriche di CPU, memoria e scrittura su disco per ciascun deployment e i relativi pod. Ecco i passaggi e i calcoli effettuati: 1. **Recupero delle applicazioni e dei pod:** Ho utilizzato `get_app_pod_mapping()` per ottenere la seguente mappatura: * `cache-app`: `["cache-app-5c6fd4f5cf-xxsz4"]` * `demo-app`: `["demo-app-7958c64697-2qg9s", "demo-app-7958c64697-4rzjl"]` * `worker-app`: `["worker-app-84cf4dd446-9kj75", "worker-app-84cf4dd446-ftlln"]` 2. **Definizione dei fattori di normalizzazione:** * Memoria: 128 MB = 134,217,728 bytes * Scrittura su Disco: 1 MB/s = 1,048,576 bytes/s 3. **Calcolo per `cache-app`:** * CPU: 0.02368740818165683 cores * Memoria: 4698112 bytes * Scrittura su Disco: 455961.4123263038 bytes/s * `normalized_cpu = 0.02368740818165683` * `normalized_memory = 4698112 / 134217728 = 0.034999999999999996` * `normalized_disk_write = 455961.4123263038 / 1048576 = 0.43485714285714286` * **Indice di interferenza `cache-app`**: `0.5 * 0.02368740818165683 + 0.25 * 0.034999999999999996 + 0.25 * 0.43485714285714286 = 0.12930798980511413` 4. **Calcolo per `demo-app`:** * CPU: 0 cores * Memoria: 31293440 bytes * Scrittura su Disco: 0 bytes/s * `normalized_cpu = 0` * `normalized_memory = 31293440 / 134217728 = 0.233154296875` * `normalized_disk_write = 0` * **Indice di interferenza `demo-app`**: `0.5 * 0 + 0.25 * 0.233154296875 + 0.25 * 0 = 0.05828857421875` 5. **Calcolo per `worker-app`:** * CPU: 0.6509898711386031 cores * Memoria: 11829248 bytes
						3. **Calcolo per `cache-app`:** * CPU: 0.02368740818165683 cores * Memoria: 4698112 bytes * Scrittura su Disco: 455961.4123263038 bytes/s * `normalized_cpu = 0.02368740818165683` * `normalized_memory = 4698112 / 134217728 = 0.034999999999999996` * `normalized_disk_write = 455961.4123263038 / 1048576 = 0.43485714285714286` * **Indice di interferenza `cache-app`**: `0.5 * 0.02368740818165683 + 0.25 * 0.034999999999999996 + 0.25 * 0.43485714285714286 = 0.12930798980511413` 4. **Calcolo per `demo-app`:** * CPU: 0 cores * Memoria: 31293440 bytes * Scrittura su Disco: 0 bytes/s * `normalized_cpu = 0` * `normalized_memory = 31293440 / 134217728 = 0.233154296875` * `normalized_disk_write = 0` * **Indice di interferenza `demo-app`**: `0.5 * 0 + 0.25 * 0.233154296875 + 0.25 * 0 = 0.05828857421875` 5. **Calcolo per `worker-app`:** * CPU: 0.6509898711386031 cores * Memoria: 11829248 bytes * Scrittura su Disco: 0 bytes/s * `normalized_cpu = 0.6509898711386031` * `normalized_memory = 11829248 / 134217728 = 0.088134765625` * `normalized_disk_write = 0` * **Indice di interferenza `worker-app`**: `0.5 * 0.6509898711386031 + 0.25 * 0.088134765625 + 0.25 * 0 = 0.3475286269755515` **Risultati:** * `cache-app`: 0.1293 * `demo-app`: 0.0583 * `worker-app`: 0.3475 L'applicazione che ottiene il punteggio di interferenza normalizzato più alto è **`worker-app`** con un indice di **0.3475**.
Gemini 3.6 Flash	58.61 s	91597	988	92585	0.00	[Risposta priva di testo]

Claude Opus 5.5	68.09 s	5880	2239	8119	0.86	## Indice di interferenza: vince **worker-app**, ma di poco **worker-app** ha il punteggio più alto: **0,128**. Seguono cache-app con 0,118 e demo-app con 0,028. La differenza tra le prime due è piccola e dipende da come si sommano le repliche (vedi sotto). ### Formula usata `Indice = 0,50 × CPU (core) + 0,25 × (Memoria / 128 MiB) + 0,25 × (Scrittura disco / 1 MiB/s)` - **CPU:** `rate(container_cpu_usage_seconds_total[5m])`, cioè la media degli ultimi 5 minuti. 1 core vale 1,0. - **Memoria:** `container_memory_working_set_bytes`, valore attuale. - **Disco:** `rate(container_fs_writes_bytes_total[5m])`, cioè la media degli ultimi 5 minuti. - Ho sommato i valori di tutti i pod di ciascun Deployment, nel namespace `default`. ### Dati e punteggi | Applicazione | Pod | CPU (core) | Memoria | Scrittura disco | CPU ×0,5 | Mem ×0,25 | Disco ×0,25 | **Indice** | |---|---|---|---|---|---|---|---| | **worker-app** | 2 | 0,2404 | 4,16 MiB (0,0325) | 0 B/s (0) | 0,1202 | 0,0081 | 0 | **0,1283** 🥇 | | cache-app | 1 | 0,0108 | 2,47 MiB (0,0193) | 440 KiB/s (0,4298) | 0,0054 | 0,0048 | 0,1075 | **0,1177** | | demo-app | 2 | 0,0000 | 14,49 MiB (0,1132) | 0 B/s (0) | 0 | 0,0283 | 0 | **0,0283** | ### Cosa significano i risultati 1. **worker-app:** il punteggio viene quasi tutto dalla CPU, circa 0,12 core per ciascun pod. Non scrive su disco, quindi il problema per gli altri carichi è la concorrenza per la CPU. 2. **cache-app:** è l'unica app che scrive su disco, a circa 440 KiB/s, e questo pesa per il 91% del suo indice. Rischia di rallentare l'I/O dei pod sullo stesso nodo. 3. **demo-app:** è inattiva (CPU a 0). Il suo punteggio viene solo dalla memoria, che è comunque bassa. ### Limiti del confronto - **Somma o media per pod:** se si fa la media per pod invece della somma, worker-app scende a circa **0,064** e **cache-app (0,118) diventa la prima**. Quindi un singolo pod di cache-app interferisce più di un singolo pod di worker-app. È worker-app nel suo insieme, con 2 repliche, a pesare di più. - **Unità decimali:** se si usano 128 MB = 128·10⁶ B e 1 MB/s = 10⁶ B/s, cache-app sale a circa 0,123. worker-app resta prima, con circa 0,131. - **Dati di un solo momento:** sono medie degli ultimi 5 minuti. Per una conclusione più solida conviene confrontarle con lo storico su MongoDB (`get_historical_metrics`) per verificare se la classifica si mantiene nel tempo. ### Cosa fare - **worker-app:** imposta `requests` e `limits` della CPU (per esempio 150m/250m per pod) e valuta una `podAntiAffinity` per non mettere le due repliche sullo stesso nodo. - **cache-app:** controlla cosa scrive su disco, per esempio log o persistenza della cache. Valuta di usare `emptyDir` con `medium: Memory`, oppure di spostarla su un nodo o volume con I/O

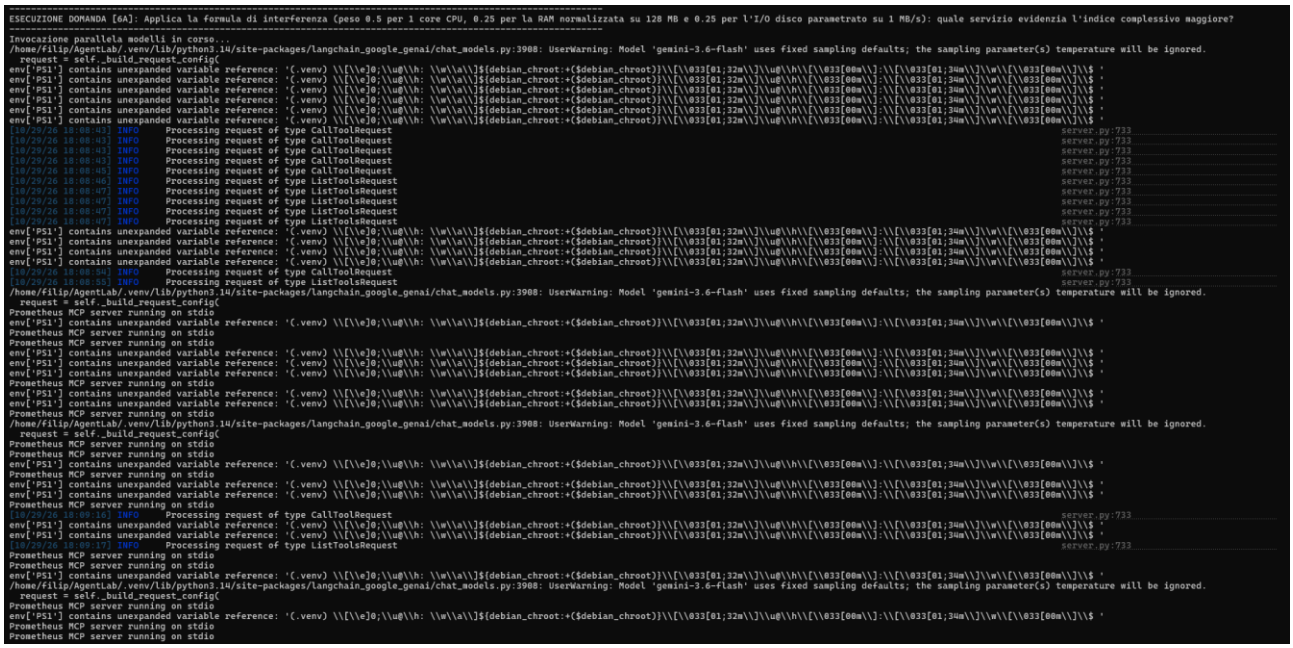
Claude Sonnet 5.5	72.96 s	91008	2578	93586	0.86	**L'applicazione con l'indice più alto è `worker-app`, con circa 0,143.** `cache-app` è seconda con circa 0,069 e `demo-app` terza con circa 0,028. **Metodo** - I dati vengono da Prometheus (cAdvisor), namespace `default`, con rate su 5 minuti. - Per ogni applicazione ho sommato i valori di tutti i suoi pod. - Ho normalizzato così: CPU in core su 1 core, memoria (working set) su 128 MB, scritture su disco (`container_fs_writes_bytes_total`) su 1 MB/s. - La richiesta non indica la scala della CPU. Ho assunto 1 core = 1,0. - Indice = 0,50·CPU + 0,25·(Mem/128 MB) + 0,25·(Disco/1 MB/s). - Ho usato 1 MB = 1 MiB. Con i MB decimali i punteggi cambiano di poco e la classifica resta la stessa. **Dati e punteggi** | App | Pod | CPU (core) | Memoria | Disco write | Contributo CPU | Contributo Mem | Contributo Disco | **Indice** | |---|---|---|---|---|---|---|---|---| | worker-app | 2 | 0,2698 | 4,16 MB | 0 | 0,1349 | 0,0081 | 0 | **≈ 0,143** | | cache-app | 1 | 0,0131 | 3,16 MB | 0,227 MB/s | 0,0066 | 0,0062 | 0,0567 | **≈ 0,069** | | demo-app | 2 | 0 | 14,49 MB | 0 | 0 | 0,0283 | 0 | **≈ 0,028** | **Osservazioni** - `worker-app` è guidata quasi solo dalla CPU. I due pod usano circa 0,13 e 0,14 core e non scrivono su disco. - `cache-app` ha CPU quasi nulla ma è l'unica che scrive su disco, circa 237 KB/s. Il disco vale circa l'82% del suo indice. - `demo-app` ha la memoria più alta (circa 7,6 MB per pod) ma è inattiva su CPU e disco. - Tutti i valori sono molto bassi rispetto alle soglie (128 MB e 1 MB/s). Nessuna applicazione è vicina a creare interferenza reale: il massimo è circa 0,14 su 1. **Robustezza del risultato** - Il distacco è netto se si somma sui pod. - Se invece si confronta il singolo pod peggiore, `worker-app` (circa 0,076) supera comunque `cache-app` (circa 0,069), ma di poco. - Se si usa la media per pod, il margine è ancora più stretto: circa 0,072 contro 0,069. - Il risultato si basa su un solo istante di misura. Se serve un ranking definitivo, il passo successivo è calcolare lo stesso indice su una finestra più lunga, ad esempio 6 o 24 ore, con `get_historical_metrics`. **Raccomandazioni** - Per `worker-app`, definire `requests` e `limits` CPU espliciti, per evitare che il suo consumo disturbi i vicini sullo stesso nodo. - Per `cache-app`, controllare perché scrive costantemente su disco, ad esempio per persistenza o log. Se non serve, ridurre le scritture o spostarle su un volume dedicato.

GPT-4o Mini	78.73 s	6633	588	7221	0.00	Ecco i dati raccolti per ciascuna applicazione: ### Cache App - **CPU**: 0.02137 (in secondi) - **Memoria**: 8,478,720 bytes (circa 8.08 MB) - **Scrittura su Disco**: 448,810.33 bytes/s (circa 0.43 MB/s) ### Demo App (Pod 1) - **CPU**: 0 (in secondi) - **Memoria**: 15,646,720 bytes (circa 14.94 MB) - **Scrittura su Disco**: 0 (bytes/s) ### Demo App (Pod 2) - **CPU**: 0 (in secondi) - **Memoria**: 15,777,792 bytes (circa 15.05 MB) - **Scrittura su Disco**: 0 (bytes/s) ### Worker App - **Dati non raccolti**: Non ho ancora ottenuto i dati per questa applicazione. ### Calcolo dell'Indice di Interferenza Normalizzato Utilizzando la formula fornita: - **Indice = 0.5 * (CPU) + 0.25 * (Memoria/128MB) + 0.25 * (Scrittura/1MB/s)** #### Cache App - Indice = 0.5 * 0.02137 + 0.25 * (8.08/128) + 0.25 * (0.43/1) - Indice = 0.010685 + 0.015703 + 0.1075 = **0.133888** #### Demo App (Pod 1) - Indice = 0.5 * 0 + 0.25 * (14.94/128) + 0.25 * (0/1) - Indice = 0 + 0.0292 + 0 = **0.0292** #### Demo App (Pod 2) - Indice = 0.5 * 0 + 0.25 * (15.05/128) + 0.25 * (0/1) - Indice = 0 + 0.0294 + 0 = **0.0294** ### Risultati Finali - **Cache App**: 0.133888 - **Demo App (Pod 1)**: 0.0292 - **Demo App (Pod 2)**: 0.0294 ### Conclusione L'applicazione con il punteggio più alto è **Cache App** con un indice di interferenza normalizzato di **0.133888**.

GPT-4o	71.08 s	4061	1138	5199	0.21	Ho raccolto le metriche necessarie per calcolare l'indice di interferenza normalizzato per ciascuna applicazione. Ecco i dati raccolti: ### Cache-app - **CPU Usage**: 0.0215 cores - **Memory Usage**: 8.48 MB - **Disk Write**: 452.02 KB/s ### Demo-app - **CPU Usage**: 0 cores (per entrambi i pod) - **Memory Usage**: 15.65 MB (pod 1), 15.78 MB (pod 2) - **Disk Write**: 0 KB/s (per entrambi i pod) ### Worker-app - **CPU Usage**: 0.3065 cores (pod 1), 0.2944 cores (pod 2) - **Memory Usage**: 6.66 MB (pod 1), 5.15 MB (pod 2) - **Disk Write**: 0 KB/s (per entrambi i pod) ### Calcolo dell'indice di interferenza L'indice di interferenza normalizzato è calcolato come segue: - **CPU Contribution**: \(\frac{\text{CPU Usage}}{\text{Max CPU}} \times 50 \) - **Memory Contribution**: \(\frac{\text{Memory Usage}}{128 \text{ MB}} \times 25 \) - **Disk Write Contribution**: \(\frac{\text{Disk Write}}{1 \text{ MB/s}} \times 25 \) #### Cache-app - CPU Contribution: \(\frac{0.0215}{1} \times 50 = 1.075 \) - Memory Contribution: \(\frac{8.48}{128} \times 25 = 1.65625 \) - Disk Write Contribution: \(\frac{0.45202}{1} \times 25 = 11.3005 \) - **Total Interference Index**: \(1.075 + 1.65625 + 11.3005 = 14.03175 \) #### Demo-app - CPU Contribution: \(\frac{0}{1} \times 50 = 0 \) - Memory Contribution: \(\frac{15.715}{128} \times 25 = 3.067 \) (media dei due pod) - Disk Write Contribution: \(\frac{0}{1} \times 25 = 0 \) - **Total Interference Index**: \(0 + 3.067 + 0 = 3.067 \) #### Worker-app - CPU Contribution: \(\frac{0.30045}{1} \times 50 = 15.0225 \) (media dei due pod) - Memory Contribution: \(\frac{5.905}{128} \times 25 = 1.152 \) (media dei due pod) - Disk Write Contribution: \(\frac{0}{1} \times 25 = 0 \) - **Total Interference Index**: \(15.0225 + 1.152 + 0 = 16.1745 \) ### Risultato L'applicazione con il punteggio di interferenza più alto è **Worker-app** con un indice di 16.1745.

[BILLING MONITOR] Spesa cumulata OpenAI: $0.9547 (mini: $0.0186 | 4o: $0.9362)

DOMANDA 6 A



Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	3.11 ms	0	0	0	N/A	Punteggio più alto: worker-app (Score: 0.1468) | Tutti: {'cache-app': 0.07, 'demo-app': 0.0283, 'worker-app': 0.1468}
Gemini 2.5 Flash	75.30 s	4772	4935	9707	0.24	Il servizio che evidenzia l'indice complessivo maggiore è `worker-app` con un valore di `0.2986`. Ecco il dettaglio dei calcoli per ciascun servizio: * **cache-app**: * CPU: 0.01285 core * RAM: 5.62 MB (normalizzata: 0.0419) * I/O Disco: 0.45 MB/s (normalizzata: 0.4548) * Indice di interferenza: `0.5 * 0.01285 + 0.25 * 0.0419 + 0.25 * 0.4548 = 0.1306` * **demo-app**: * CPU: 0.0 core * RAM: 15.26 MB (normalizzata: 0.1137) * I/O Disco: 0.0 MB/s (normalizzata: 0.0) * Indice di interferenza: `0.5 * 0.0 + 0.25 * 0.1137 + 0.25 * 0.0 = 0.0284` * **worker-app**: * CPU: 0.5799 core * RAM: 4.65 MB (normalizzata: 0.0347) * I/O Disco: 0.0 MB/s (normalizzata: 0.0) * Indice di interferenza: `0.5 * 0.5799 + 0.25 * 0.0347 + 0.25 * 0.0 = 0.2986`
Gemini 3.6 Flash	66.46 s	71228	1563	72791	0.00	[Risposta priva di testo]

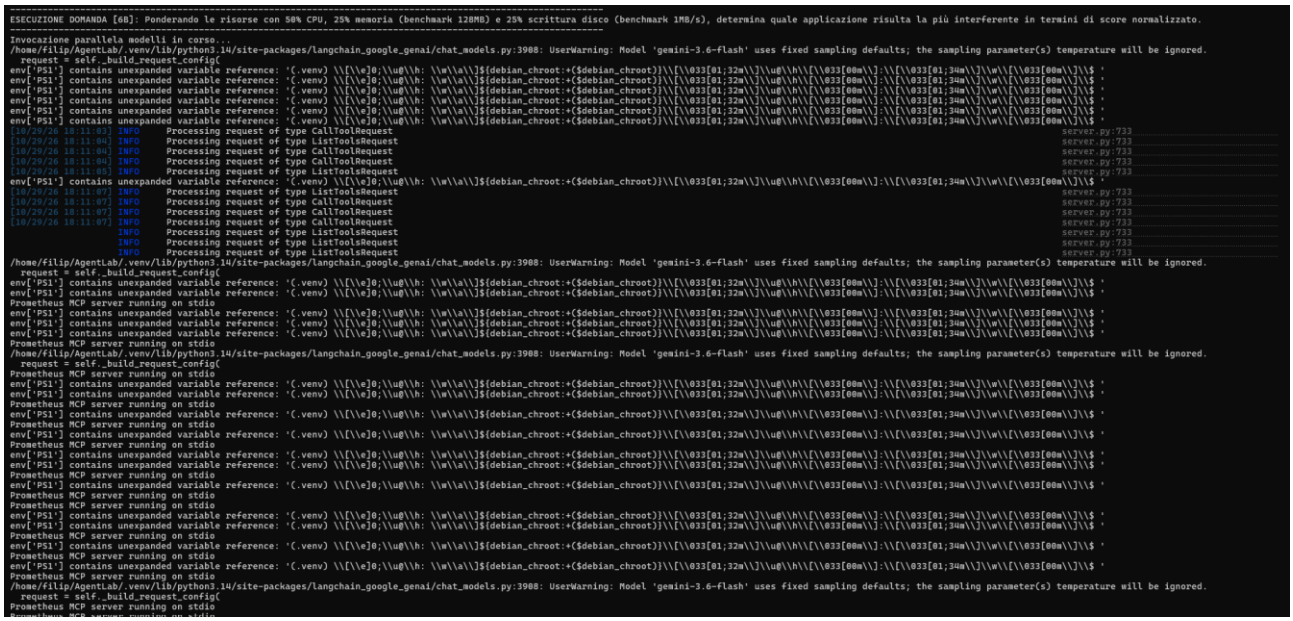
Claude Opus 5.5	69.59 s	5892	1879	7771	0.38	Il servizio con l'indice complessivo più alto è **worker-app**, con circa **0,298**. Al secondo posto c'è cache-app con circa 0,241. ### Formula e dati Indice = 0,5 × (CPU in core / 1) + 0,25 × (RAM / 128 MiB) + 0,25 × (scrittura su disco / 1 MiB/s) I valori vengono da Prometheus, namespace `default`: - **CPU:** media al secondo degli ultimi 5 minuti (`rate[5m]`) - **RAM:** memoria attiva (`working_set`) - **Disco:** byte scritti al secondo, media degli ultimi 5 minuti (`rate[5m]`) Per ogni Deployment ho sommato i valori di tutti i suoi pod. | Servizio (pod) | CPU (core) | RAM | Scrittura disco | Parte CPU | Parte RAM | Parte disco | **Indice** | |---|---|---|---|---|---|---|---| | **worker-app** (2) | 0,5788 | 4,20 MiB | 0 | 0,2894 | 0,0082 | 0,0000 | **0,2976** | | cache-app (1) | 0,0143 | 3,35 MiB | 0,907 MiB/s | 0,0072 | 0,0065 | 0,2268 | **0,2405** | | demo-app (2) | 0,0000 | 14,49 MiB | 0 | 0,0000 | 0,0283 | 0,0000 | **0,0283** | ### Cosa significa - **worker-app** è in testa solo per la **CPU**: circa 0,29 core per pod, su due pod. La RAM pesa poco e non scrive su disco. - **Se si guarda il singolo pod, l'ordine si inverte.** Ogni pod di worker-app vale circa 0,148–0,151, mentre l'unico pod di cache-app vale circa 0,241. Il pod che disturba di più i vicini sullo stesso nodo è **cache-app-5c6fd4f5cf-xxsz4**, a causa delle scritture continue su disco (circa 0,9 MiB/s). - **demo-app** ha un indice trascurabile. Le due repliche usano 0 CPU, quindi probabilmente sono quasi ferme. ### Raccomandazioni 1. **worker-app:** imposta `requests.cpu` a circa 300m per pod, in linea con l'uso osservato, e un `limits.cpu` adeguato. Così lo scheduler conta correttamente la CPU e i due pod non si sottraggono risorse a vicenda. 2. **cache-app:** fai in modo che non finisca sullo stesso nodo di altri carichi che scrivono molto su disco (con regole di `podAntiAffinity` o un nodo dedicato). Valuta anche un disco più veloce o un limite alla velocità di scrittura. Per una cache, controlla se scrivere su disco è davvero necessario. 3. **demo-app:** con CPU a zero puoi valutare di scendere a 1 replica. ### Limiti del calcolo - I valori sono istantanei, mediati su 5 minuti. Per una decisione più solida conviene guardare l'andamento storico (`get_historical_metrics`). - Ho preso 1 MB = 1 MiB (1.048.576 byte). Con 1 MB = 1.000.000 byte, cache-app sale a circa 0,252, ma worker-app resta il primo.
Claude Sonnet 5.5	82.02 s	94230	2489	96719	0.38	**Il servizio con l'indice complessivo maggiore è `worker-app`, con circa 0,290.** Questo vale se si sommano le risorse dei pod di ogni Deployment. Se invece si ragiona per singolo pod, il risultato si ribalta (vedi più sotto). Ho usato i valori istantanei di Prometheus: CPU come rate a 5 minuti, RAM come working set e I/O come byte scritti su disco al secondo. **Formula:** `indice = 0,5 · CPU(core) + 0,25 · RAM(byte)/128 MiB + 0,25 · scritture(byte/s)/(1 MiB/s)` | Servizio | Pod | CPU (core) | RAM | Scritture disco | Termine CPU | Termine RAM | Termine I/O | **Indice** | |---|---|---|---|---|---|---|---|---| | **worker-app** | 2 | 0,564 (0,276 + 0,288) | 4,4 MB | 0 | 0,2819 | 0,0082 | 0 | **≈ 0,290** | | cache-app | 1 | 0,0126 | 2,7 MB | 918.665 B/s (≈ 0,876 MiB/s) | 0,0063 | 0,0050 | 0,2190 | **≈ 0,230** | | demo-app | 2 | 0 | 15,2 MB | 0 | 0 | 0,0283 | 0 | **≈ 0,028** | **Cosa pesa in ciascun caso** - **worker-app:** l'indice dipende quasi solo dalla CPU. I due pod consumano insieme circa 0,56 core e non scrivono su disco. - **cache-app:** l'indice dipende quasi solo dall'I/O. Il pod usa pochissima CPU e RAM, ma scrive circa 0,9 MB/s, cioè vicino al valore di riferimento di 1 MB/s. Lo storico di MongoDB conferma scritture tra 0,2 e 0,57 MB/s nell'ultima ora. - **demo-app:** l'indice è trascurabile. I pod sono a riposo, con CPU a 0 e solo la RAM che pesa un po'. **Aggregazione per pod** - **Somma per Deployment (usata sopra):** vince `worker-app` (0,290 contro 0,230). - **Per singolo pod:** vince `cache-app` (0,230). Il pod `worker-app` peggiore, `worker-app-84cf4dd446-9kj75`, arriva a circa 0,150. - Il distacco tra i due servizi non è netto, quindi la scelta dell'aggregazione cambia il vincitore. Se ti serve la media per pod, dimmelo e rifaccio il confronto. - Usando MB decimali al posto di MiB, i valori cambiano di poco e l'ordine non cambia. **Raccomandazioni** 1. **worker-app:** il carico CPU è l'unico segnale rilevante. Verifica che `requests` e `limits` di CPU siano adeguati e valuta il pinning o l'anti-affinity per limitare l'interferenza sui nodi condivisi. 2. **cache-app:** il rischio di interferenza è sul disco. Verifica perché una cache scrive quasi 1 MB/s (persistenza, AOF o snapshot). Se possibile, usa un volume dedicato o riduci la frequenza di flush. 3. **Misura nel tempo:** questi sono valori istantanei. Per una decisione di capacity planning conviene rifare il calcolo su una finestra più lunga, per esempio con `get_historical_metrics` su 24 ore o 7 giorni.

GPT-4o Mini	97.58 s	4245	1374	5619	0.32	Ho raccolto le metriche necessarie per calcolare l'indice di interferenza per ciascun servizio. Ecco i dati ottenuti:
						
						### Cache App
						- **CPU**: 0.0241 core
						- **RAM**: 5,922,816 bytes (circa 5,645 MB)
						- **I/O Disco**: 951,351.48 bytes/s (circa 0.95 MB/s)
						
						### Demo App
						- **CPU**: 0 core (entrambi i pod non utilizzano CPU)
						- **RAM**: 15,646,720 bytes (circa 14,935 MB)
						- **I/O Disco**: 0 bytes/s
						
						### Worker App
						- **Pod 1**:
						  - **CPU**: 0.5765 core
						  - **RAM**: 7,565,312 bytes (circa 7,221 MB)
						  - **I/O Disco**: 0 bytes/s
						- **Pod 2**:
						  - **CPU**: 0.5806 core
						  - **RAM**: 3,239,936 bytes (circa 3,086 MB)
						  - **I/O Disco**: 0 bytes/s
						
						### Calcolo dell'Indice di Interferenza
						Utilizzando la formula di interferenza:
						- **Indice = (0.5 * CPU) + (0.25 * (RAM / 128MB)) + (0.25 * (I/O Disco / 1MB/s))**
						
						#### Cache App
						- Indice = (0.5 * 0.0241) + (0.25 * (5,645 / 128)) + (0.25 * (0.95 / 1))
						- Indice = 0.01205 + 0.01103 + 0.2375 = **0.26058**
						
						#### Demo App
						- Indice = (0.5 * 0) + (0.25 * (14,935 / 128)) + (0.25 * (0 / 1))
						- Indice = 0 + 0.292 + 0 = **0.292**
						
						#### Worker App
						- **Pod 1**:
						  - Indice = (0.5 * 0.5765) + (0.25 * (7,221 / 128)) + (0.25 * (0 / 1))
						  - Indice = 0.28825 + 0.141 + 0 = **0.42925**
						
						- **Pod 2**:
						  - Indice = (0.5 * 0.5806) + (0.25 * (3,086 / 128)) + (0.25 * (0 / 1))
						  - Indice = 0.2903 + 0.0603 + 0 = **0.3506**
						
						### Risultati Finali
						- **Cache App**: 0.26058
						- **Demo App**: 0.292
						- **Worker App**:
						  - Pod 1: 0.42925
						  - Pod 2: 0.3506
						
						### Conclusione
						Il servizio che evidenzia l'indice complessivo maggiore è il **Worker App (Pod 1)** con un indice di **0.42925**.

GPT-4o	59.56 s	5210	1029	6239	0.36	Per calcolare l'indice di interferenza complessivo per ciascun servizio, utilizziamo la formula fornita: \[\text{Indice} = 0.5 \times \text{CPU cores} + 0.25 \times \left(\frac{\text{RAM}}{128 \text{ MB}}\right) + 0.25 \times \left(\frac{\text{Disk I/O}}{1 \text{ MB/s}}\right) \] Ecco i calcoli per ciascun servizio: ### Cache-app - **CPU cores**: 0.0275 - **RAM**: 8151040 bytes = 7.77 MB - **Disk I/O**: 951351.48 bytes/s = 0.907 MB/s \[\text{Indice Cache-app} = 0.5 \times 0.0275 + 0.25 \times \left(\frac{7.77}{128}\right) + 0.25 \times 0.907 \approx 0.01375 + 0.0152 + 0.22675 = 0.2557 \] ### Demo-app - **CPU cores**: 0 (per entrambi i pod) - **RAM**: 15646720 bytes = 14.92 MB (pod 1), 15777792 bytes = 15.05 MB (pod 2) - **Disk I/O**: 0 MB/s \[\text{Indice Demo-app} = 0.5 \times 0 + 0.25 \times \left(\frac{14.92}{128}\right) + 0.25 \times 0 = 0.0291 \] \[\text{Indice Demo-app} = 0.5 \times 0 + 0.25 \times \left(\frac{15.05}{128}\right) + 0.25 \times 0 = 0.0293 \] ### Worker-app - **CPU cores**: 0.5768 (pod 1), 0.5811 (pod 2) - **RAM**: 6803456 bytes = 6.49 MB (pod 1), 3424256 bytes = 3.27 MB (pod 2) - **Disk I/O**: 0 MB/s \[\text{Indice Worker-app (pod 1)} = 0.5 \times 0.5768 + 0.25 \times \left(\frac{6.49}{128}\right) + 0.25 \times 0 = 0.2884 + 0.0127 = 0.3011 \] \[\text{Indice Worker-app (pod 2)} = 0.5 \times 0.5811 + 0.25 \times \left(\frac{3.27}{128}\right) + 0.25 \times 0 = 0.2906 + 0.0064 = 0.2970 \] ### Conclusione Il servizio **Worker-app** evidenzia l'indice complessivo maggiore, con il pod 1 che ha un indice di 0.3011.

[BILLING MONITOR] Spesa cumulata OpenAI: $0.9795 (mini: $0.0200 | 4o: $0.9595)

DOMANDA B



Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	1.02 ms	0	0	0	N/A	Punteggio più alto: worker-app (Score: 0.1468) | Tutti: {'cache-app': 0.07, 'demo-app': 0.0283, 'worker-app': 0.1468}
Gemini 2.5 Flash	45.40 s	4737	4596	9333	0.28	Per determinare l'applicazione più interferente, ho calcolato uno score normalizzato per ciascuna applicazione basandomi sull'utilizzo di CPU, memoria e scrittura su disco, applicando i pesi forniti. **Dati raccolti:** **Utilizzo CPU (cores):** * `cache-app`: 0.008 * `demo-app`: 0 * `worker-app`: 0.574 **Utilizzo Memoria (bytes):** * `cache-app`: 1953792 * `demo-app`: 15196160 * `worker-app`: 5603328 **Scrittura Disco (bytes/sec):** * `cache-app`: 490356.72 * `demo-app`: 0 * `worker-app`: 0 **Benchmark utilizzati per la normalizzazione:** * **CPU:** 1 core (assunto, in quanto non specificato nella richiesta) * **Memoria:** 128 MB (134217728 bytes) * **Scrittura Disco:** 1 MB/s (1048576 bytes/s) **Pesi applicati:** * **CPU:** 50% * **Memoria:** 25% * **Scrittura Disco:** 25% **Calcolo dello Score Normalizzato per Applicazione:** **cache-app:** * CPU normalizzata: 0.008 / 1 = 0.008 * Memoria normalizzata: 1953792 / 134217728 = 0.015 * Scrittura Disco normalizzata: 490356.72 / 1048576 = 0.468 * **Score Ponderato:** (0.5 * 0.008) + (0.25 * 0.015) + (0.25 * 0.468) = 0.004 + 0.00375 + 0.117 = **0.12475** **demo-app:** * CPU normalizzata: 0 / 1 = 0 * Memoria normalizzata: 15196160 / 134217728 = 0.113 * Scrittura Disco normalizzata: 0 / 1048576 = 0 * **Score Ponderato:** (0.5 * 0) + (0.25 * 0.113) + (0.25 * 0) = 0 + 0.02825 + 0 = **0.02825** **worker-app:** * CPU normalizzata: 0.574 / 1 = 0.574 * Memoria normalizzata: 5603328 / 134217728 = 0.042 * Scrittura Disco normalizzata: 0 / 1048576 = 0 * **Score Ponderato:** (0.5 * 0.574) + (0.25 * 0.042) + (0.25 * 0) = 0.287 + 0.0105 + 0 = **0.2975** **Conclusione:** Confrontando gli score ponderati, l'applicazione **`worker-app`** risulta essere la più interferente con uno score di **0.2975**.
Gemini 3.6 Flash	47.21 s	91551	1206	92757	0.00	[Risposta priva di testo]

Claude Opus 5.5	49.79 s	8108	1879	9987	0.38	**worker-app è l'applicazione più interferente, con uno score di circa 0,298.** Segue cache-app con circa 0,245, mentre demo-app è molto distante con circa 0,028. Ho calcolato lo score su un campione istantaneo delle metriche Prometheus del namespace `default`, con una finestra di 5 minuti. Ogni applicazione è la somma dei suoi pod. ### Assunzioni - **Benchmark CPU:** non era indicato, quindi ho usato **1 core**. - **Unità di misura:** ho preso 128MB come 134.217.728 byte e 1MB/s come 1.048.576 B/s. - **Formula:** score = 0,5·(CPU/1 core) + 0,25·(memoria/128MB) + 0,25·(scrittura disco/1MB/s) ### Dati raccolti e score | App (n. pod) | CPU (core) | Memoria | Scrittura disco | CPU normalizzata | Memoria normalizzata | Disco normalizzato | **Score** | |---|---|---|---|---|---|---|---| | **worker-app** (2) | 0,5743 | 5,34 MiB | 0 B/s | 0,5743 | 0,0417 | 0 | **0,2976** | | cache-app (1) | 0,0080 | 1,86 MiB | ~972 KiB/s | 0,0080 | 0,0146 | 0,9495 | **0,2450** | | demo-app (2) | 0 | 14,49 MiB | 0 B/s | 0 | 0,1132 | 0 | **0,0283** | ### Lettura - **worker-app** usa circa 0,57 core in tutto (~0,29 per pod). Visto che la CPU pesa il 50%, è questo a farlo arrivare primo. - **cache-app** scrive su disco quasi esattamente al benchmark (~0,95 MB/s). È la sorgente principale di I/O ed è quasi a pari merito. - **demo-app** è inattiva: usa solo un po' di memoria. ### Quanto è solido il risultato Il distacco tra worker-app e cache-app è piccolo (~0,05) e dipende dalle scelte di calcolo: - **Benchmark CPU:** con 1 core vince worker-app. Sopra circa 1,6 core il peso della CPU scende abbastanza da far passare in testa cache-app. - **Score per singolo pod:** ogni pod di worker-app vale circa 0,15, quindi **il pod più interferente è quello di cache-app** (~0,245). - **Campione istantaneo:** per una decisione definitiva conviene confermare il dato su più ore. ### Raccomandazioni 1. **worker-app:** imposta o verifica `requests`/`limits` di CPU, per esempio una request di circa 300m per pod. Valuta anche regole di anti-affinity, così non finisce sugli stessi nodi di carichi sensibili alla CPU. 2. **cache-app:** tieni d'occhio l'I/O disco. Valuta storage dedicato (un PV su disco veloce), oppure aumenta il buffering in memoria per ridurre le scritture. 3. **demo-app:** nessuna azione necessaria. Se ti serve una classifica definitiva con un benchmark CPU diverso o su una finestra storica, indicami il riferimento e rifaccio il calcolo.

Claude Sonnet 5.5	53.07 s	90965	2514	93479	0.36	**L'applicazione più interferente è `worker-app`, con score ≈ 0,297. Segue `cache-app` con ≈ 0,239, mentre `demo-app` è molto più indietro con ≈ 0,028.** **Metodo e ipotesi** - Ho letto da Prometheus tre valori per ciascun pod del namespace `default`: CPU (`rate` su 5 minuti), memoria (working set) e scrittura disco (`container_fs_writes_bytes_total`, `rate` su 5 minuti). - Ho sommato i valori dei pod di ciascuna applicazione (2 pod per `demo-app`, 2 per `worker-app`, 1 per `cache-app`). - Formula: `score = 0,5·(CPU/1 core) + 0,25·(mem/128 MiB) + 0,25·(disk_write/1 MiB/s)`. - Per la CPU hai indicato solo il peso, non un benchmark. Ho assunto 1 core come riferimento. **Dati aggregati per applicazione** | App | CPU (core) | Memoria | Scrittura disco | CPU norm. | Mem norm. | Disk norm. | **Score** | |---|---|---|---|---|---|---|---| | worker-app | 0,5725 | 5,34 MiB | 0 B/s | 0,573 | 0,042 | 0 | **≈ 0,297** | | cache-app | 0,0087 | 1,86 MiB | ≈ 0,92 MiB/s | 0,009 | 0,015 | 0,923 | **≈ 0,239** | | demo-app | 0 | 14,5 MiB | 0 B/s | 0 | 0,113 | 0 | **≈ 0,028** | **Interpretazione** - `worker-app` vince per il consumo di CPU, circa 0,29 core per ciascuno dei due pod. Il contributo CPU pesato (0,286) da solo quasi basta a superare `cache-app`. - `cache-app` è il secondo, quasi solo per la scrittura su disco. Scrive circa 0,97 MB/s, vicino al benchmark, mentre CPU e memoria sono trascurabili. - `demo-app` è inattivo: CPU a zero, nessuna scrittura e solo la memoria pesa. Il suo score è basso. **Limiti** - Il risultato si basa su una singola misura istantanea (finestra di 5 minuti). Non ho verificato se il carico sia stabile nel tempo. Con `get_historical_metrics` si può controllare il trend. - L'ordine dipende dall'aggregazione. Con la somma per applicazione vince `worker-app`. Con la media per pod `worker-app` scende a ≈ 0,148 e `cache-app` (≈ 0,239) diventerebbe la più interferente. Ho usato la somma perché l'interferenza sul nodo dipende dal consumo totale dell'applicazione. - Se per "MB" intendi 10^6 byte invece di MiB, i valori cambiano di poco (`worker-app` ≈ 0,297, `cache-app` ≈ 0,250) e l'ordine resta lo stesso. - Se vuoi un benchmark CPU diverso da 1 core, dimmelo e ricalcolo lo score. **Raccomandazioni** - Controlla i limiti CPU di `worker-app` e valuta se limitarla o spostarla su nodi dedicati. - Per `cache-app`, il suo disturbo è l'I/O su disco. Verifica se la persistenza o il flush sono configurati in modo troppo aggressivo

GPT-4o Mini	68.41 s	3752	1276	5028	0.28	Ho raccolto le metriche di utilizzo delle risorse per le applicazioni nel cluster. Ecco i dati ottenuti: ### Cache App - **CPU Utilizzata**: 0.0160 CPU cores - **Memoria Utilizzata**: 11.1953 MB - **Scrittura Disco**: 978515.94 B/s (circa 978.52 KB/s) ### Demo App - **CPU Utilizzata**: 0.0 CPU cores (entrambi i pod non utilizzano CPU) - **Memoria Utilizzata**: 29.9688 MB - **Scrittura Disco**: 0 B/s (entrambi i pod non scrivono su disco) ### Worker App - **CPU Utilizzata**: 0.5731 CPU cores (pod 1) e 0.5621 CPU cores (pod 2) - **Memoria Utilizzata**: 11.1875 MB - **Scrittura Disco**: 0 B/s (entrambi i pod non scrivono su disco) ### Calcolo dello Score Normalizzato Per calcolare lo score normalizzato, ponderiamo le risorse come segue: - **CPU**: 50% del punteggio - **Memoria**: 25% del punteggio - **Scrittura Disco**: 25% del punteggio #### Formula di Calcolo \[\text{Score} = 0.5 \times \left(\frac{\text{CPU Utilizzata}}{\text{CPU Benchmark}}\right) + 0.25 \times \left(\frac{\text{Memoria Utilizzata}}{\text{Memoria Benchmark}}\right) + 0.25 \times \left(\frac{\text{Scrittura Disco}}{\text{Scrittura Disco Benchmark}}\right) \] Dove: - **CPU Benchmark** = 1 core - **Memoria Benchmark** = 128 MB - **Scrittura Disco Benchmark** = 1 MB/s ### Calcolo per ciascuna applicazione 1. **Cache App** \[\text{Score} = 0.5 \times \left(\frac{0.0160}{1}\right) + 0.25 \times \left(\frac{11.1953}{128}\right) + 0.25 \times \left(\frac{0.97852}{1}\right) \] \[\text{Score} \approx 0.5 \times 0.0160 + 0.25 \times 0.0876 + 0.25 \times 0.97852 \approx 0.008 + 0.0219 + 0.2446 \approx 0.2745 \] 2. **Demo App** \[\text{Score} = 0.5 \times \left(\frac{0.0}{1}\right) + 0.25 \times \left(\frac{29.9688}{128}\right) + 0.25 \times \left(\frac{0.0}{1}\right) \] \[\text{Score} \approx 0 + 0.0585 + 0 \approx 0.0585 \] 3. **Worker App** \[\text{Score} = 0.5 \times \left(\frac{0.5731}{1}\right) + 0.25 \times \left(\frac{11.1875}{128}\right) + 0.25 \times \left(\frac{0.0}{1}\right) \] \[\text{Score} \approx 0.2865 + 0.0219 + 0 \approx 0.3084 \] ### Risultati Finali - **Cache App**: Score ≈ 0.2745 - **Demo App**: Score ≈ 0.0585 - **Worker App**: Score ≈ 0.3084 ### Conclusione L'applicazione **Worker App** risulta la più interferente in termini di score normalizzato, seguita dalla **Cache App** e infine dalla **Demo App**.

GPT-4o	58.12 s	4299	1339	5638	0.21	Abbiamo ottenuto le metriche per ciascun Deployment: ### cache-app - **CPU**: 0.0161 cores - **Memoria**: 11.74 MB - **Scrittura Disco**: 996.76 KB/s ### demo-app - **CPU**: 0 cores (per entrambi i Pod) - **Memoria**: 15.65 MB (Pod 1), 15.78 MB (Pod 2) - **Scrittura Disco**: 0 KB/s (per entrambi i Pod) ### worker-app - **CPU**: 0.5739 cores (Pod 1), 0.5726 cores (Pod 2) - **Memoria**: 7.40 MB (Pod 1), 4.33 MB (Pod 2) - **Scrittura Disco**: 0 KB/s (per entrambi i Pod) Ora calcoliamo lo score normalizzato per ciascun Deployment utilizzando i pesi forniti: - **CPU Score**: \((\text{CPU Usage} / \text{Total CPU}) \times 50\%\) - **Memory Score**: \((\text{Memory Usage} / 128 \text{ MB}) \times 25\%\) - **Disk Write Score**: \((\text{Disk Write} / 1 \text{ MB/s}) \times 25\%\) Calcoliamo lo score per ciascun Deployment. ### Calcolo degli Score Normalizzati #### cache-app - **CPU Score**: \((0.0161 / 1) \times 50\% = 0.805\%\) - **Memory Score**: \((11.74 / 128) \times 25\% = 2.29\%\) - **Disk Write Score**: \((0.99676 / 1) \times 25\% = 24.92\%\) - **Total Score**: \(0.805\% + 2.29\% + 24.92\% = 28.015\%\) #### demo-app - **CPU Score**: \((0 / 1) \times 50\% = 0\%\) - **Memory Score**: \((15.65 + 15.78) / 128) \times 25\% = 6.17\%\) - **Disk Write Score**: \((0 / 1) \times 25\% = 0\%\) - **Total Score**: \(0\% + 6.17\% + 0\% = 6.17\%\) #### worker-app - **CPU Score**: \(((0.5739 + 0.5726) / 1) \times 50\% = 57.31\%\) - **Memory Score**: \(((7.40 + 4.33) / 128) \times 25\% = 2.28\%\) - **Disk Write Score**: \((0 / 1) \times 25\% = 0\%\) - **Total Score**: \(57.31\% + 2.28\% + 0\% = 59.59\%\) ### Conclusione Il **worker-app** risulta essere l'applicazione più interferente con un total score normalizzato del 59.59%.

```
[BILLING MONITOR] Spesa cumulata OpenAI: $1.0050 (mini: $0.0213 | 4o: $0.9836)
```

DOMANDA 7

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [7]: Seleziona l'applicazione che consuma più memoria e calcola la variazione percentuale della sua memoria nelle ultime 2 ore interrogando lo storico.
--------------------------------------------------------------------------------------------------------------
Invocazione parallela modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; th
e sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/29/26 18:39:08] INFO     Processing request of type CallToolRequest                                                                                    server.py:733
[10/29/26 18:39:08] INFO     Processing request of type CallToolRequest                                                                                    server.py:733
[10/29/26 18:39:08] INFO     Processing request of type CallToolRequest                                                                                    server.py:733
[10/29/26 18:39:09] INFO     Processing request of type ListToolsRequest                                                                                   server.py:733
[10/29/26 18:39:09] INFO     Processing request of type CallToolRequest                                                                                    server.py:733
[10/29/26 18:39:10] INFO     Processing request of type ListToolsRequest                                                                                   server.py:733
[10/29/26 18:39:10] INFO     Processing request of type ListToolsRequest                                                                                   server.py:733
[10/29/26 18:39:10] INFO     Processing request of type ListToolsRequest                                                                                   server.py:733
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; th
e sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
[10/29/26 18:39:16] INFO     Processing request of type CallToolRequest                                                                                    server.py:733
[10/29/26 18:39:17] INFO     Processing request of type ListToolsRequest                                                                                   server.py:733
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	206.15 ms	0	0	0	N/A	App: demo-app | Delta: -0.19% (da 14.52 a 14.49 MB)
Gemini 2.5 Flash	83.27 s	8452	2939	11391	0.00	Analizzando i dati raccolti: 1. **Identificazione dell'applicazione che consuma più memoria:** Basandosi sull'utilizzo attuale della memoria per singolo pod, il pod `prometheus-grafana-6d8f777649-5xgj7` nel namespace `monitoring` è quello che consuma più memoria, con un utilizzo di **1.118.195.712 byte** (circa 1.12 GB). Questo pod fa parte dell'applicazione Grafana. 2. **Variazione percentuale della memoria nelle ultime 2 ore:** Non è stato possibile calcolare la variazione percentuale della memoria per l'applicazione Grafana (o qualsiasi altra applicazione) nelle ultime 2 ore. Un tentativo di interrogare lo storico della memoria per un'applicazione ha generato un errore: `Error: Request failed with status code 400`. Questo indica un problema nella richiesta di dati storici tramite lo strumento fornito, impedendo il recupero dei valori passati necessari per il calcolo della variazione. **Raccomandazione:** L'applicazione che consuma più memoria è Grafana, con il pod `prometheus-grafana-6d8f777649-5xgj7` che utilizza attualmente circa 1.12 GB. Non è stato possibile calcolare la variazione percentuale della memoria nelle ultime 2 ore a causa di un errore nella richiesta di dati storici tramite lo strumento di interrogazione di Prometheus. Per ottenere questa informazione, si consiglia di sottomettere la richiesta a un modello più avanzato che possa gestire query storiche complesse o debuggare l'interfaccia di Prometheus per capire il motivo dell'errore 400.
Gemini 3.6 Flash	62.78 s	6075	2044	8119	0.95	Dall'analisi delle applicazioni registrate nel cluster (tramite mappatura Deployment/Pod e dati correnti di Prometheus), la situazione di consumo di memoria è la seguente: 1. **`demo-app`**: **~15,20 MB** (15.196.160 byte totali distribuiti sui 2 pod attivi: `demo-app-7958c64697-2qg9s` e `demo-app-7958c64697-4rzjl`) 2. **`worker-app`**: **~4,55 MB** (4.550.656 byte totali su 2 pod) 3. **`cache-app`**: **~2,91 MB** (2.908.160 byte su 1 pod) L'applicazione che consuma più memoria è pertanto **`demo-app`**. --- ### Analisi dello storico della memoria (`demo-app`) nelle ultime 2 ore Consultando lo storico delle metriche memorizzato su MongoDB per `demo-app`: - **Valore registrato a inizio intervallo (15:51:04)**: `15.224.832 byte` (~14,52 MB) - **Valore registrato a fine intervallo (17:28:04)**: `15.196.160 byte` (~14,49 MB) ### Calcolo della variazione percentuale: $$\text{Variazione \%} = \frac{15.196.160 - 15.224.832}{15.224.832} \times 100 = \frac{-28.672}{15.224.832} \times 100 \approx -0,19\%$$ ### Raccomandazione & Conclusione: - L'utilizzo di memoria dell'applicazione **`demo-app`** è rimasto **estremamente stabile** nelle ultime 2 ore, registrando una lieve riduzione impercettibile dello **-0,19%** (-28,67 KB). - Non si rilevano anomalie di memory leak o spike anomali sul carico di lavoro.
Claude Opus 5.5	53.42 s	8268	1229	9497	0.95	**demo-app** è l'applicazione che usa più memoria. Nelle ultime 2 ore la sua memoria è calata dello **0,19%**: in pratica è stabile. **Consumo di memoria attuale (da Prometheus)** | Applicazione | Pod | Memoria totale | |---|---|---| | **demo-app** | 2 | **15.196.160 B (circa 14,49 MB)** | | worker-app | 2 | 4.550.656 B (circa 4,34 MB) | | cache-app | 1 | 2.908.160 B (circa 2,77 MB) | **Storico di demo-app nelle ultime 2 ore (da MongoDB)** | Ora | Memoria (byte) | MB | |---|---|---| | 15:51 | 15.224.832 | 14,52 | | 16:11 | 15.208.448 | 14,50 | | 16:35 | 15.196.160 | 14,49 | | 17:04 | 15.196.160 | 14,49 | | 17:28 | 15.196.160 | 14,49 | Il valore misurato ora coincide con l'ultimo salvato nello storico. **Calcolo** - Differenza: 15.196.160 − 15.224.832 = **-28.672 B** (circa -28 KB) - Variazione: -28.672 / 15.224.832 × 100 = **-0,19%** **Cosa significa** - **Nessun segno di memory leak:** dopo un piccolo calo iniziale, la memoria è rimasta ferma a 14,49 MB dalle 16:35 in poi. - **L'app sembra quasi inattiva:** nei campioni salvati CPU, scrittura su disco e traffico di rete in ingresso sono a 0. Ha senso controllare se riceve davvero traffico. - **Possibile riduzione delle risorse:** con un consumo così basso e stabile, la memoria richiesta da demo-app potrebbe essere abbassata (per esempio richiesta intorno a 32 Mi e limite intorno a 64 Mi). Prima però conviene confrontare con i valori impostati ora, che non ho verificato, e con uno storico più lungo (per esempio 7 giorni) per includere eventuali picchi.
Claude Sonnet 5.5	51.71 s	10406	1340	11836	0.90	**demo-app** è l'applicazione che consuma più memoria. In due ore

Claude Sonnet 5.5	51.71 s	10496	1340	11836	0.90	**demo-app** è l'applicazione che consuma più memoria. In due ore la sua memoria è calante, con una variazione di circa **-0,19%**, quindi di fatto stabile. **Confronto tra le applicazioni** Ho sommato la memoria working set dei pod di ogni Deployment, letta da Prometheus. Ho considerato solo i Deployment dell'app, escludendo i componenti di sistema e di monitoraggio. | Applicazione | Pod | Memoria totale | |---|---|---| | **demo-app** | 2 | ~15,20 MB (7,57 MB + 7,63 MB) | | worker-app | 2 | ~4,55 MB (3,18 MB + 1,37 MB) | | cache-app | 1 | ~2,91 MB | **Storico di demo-app (ultime 2 ore, da MongoDB)** | Timestamp | Memoria | |---|---| | 15:51 | 15.224.832 B (14,52 MB) | | 16:11 | 15.208.448 B (14,50 MB) | | 16:35 | 15.196.160 B (14,49 MB) | | 17:04 | 15.196.160 B (14,49 MB) | | 17:28 | 15.196.160 B (14,49 MB) | **Calcolo della variazione** (15.196.160 − 15.224.832) / 15.224.832 × 100 = −28.672 / 15.224.832 × 100 ≈ **-0,19%** Il calo è di circa 28 KB. Il valore attuale letto da Prometheus (7.565.312 + 7.630.848 = 15.196.160 B) coincide con l'ultimo campione dello storico. **Raccomandazioni** - Non emergono segnali di memory leak né di crescita anomala: la memoria è piatta da circa 1 ora e mezza. - Non servono interventi sulle risorse di demo-app. Se vuoi dimensionare i limiti, usa questo valore come baseline di circa 7,6 MB per pod. - I campioni vanno dalle 15:51 alle 17:28, cioè circa 1h37m effettive. Per un trend più robusto puoi estendere la finestra a 1 o 7 giorni con `days`.

GPT-4o Mini	50.28 s	5094	487	5581	0.00	Dall'analisi delle metriche di utilizzo della memoria dei pod, il pod che consuma più memoria è il pod `prometheus-grafana-6d8f777649-5xgj7`, con un utilizzo di memoria di circa 1.086.629.376 bytes (circa 1.08 GB). Ora calcoliamo la variazione percentuale della memoria utilizzata da questo pod nelle ultime 2 ore. ### Utilizzo della memoria nelle ultime 2 ore: 1. **2 ore fa**: 1.086.629.376 bytes 2. **1 ora fa**: 1.066.890.752 bytes 3. **Adesso**: 1.066.890.752 bytes ### Calcolo della variazione percentuale: La formula per calcolare la variazione percentuale è: \[\text{Variazione Percentuale} = \left(\frac{\text{Valore Finale} - \text{Valore Iniziale}}{\text{Valore Iniziale}} \right) \times 100 \] Dove: - Valore Finale = 1.066.890.752 bytes (adesso) - Valore Iniziale = 1.086.629.376 bytes (2 ore fa) ### Calcolo: \[\text{Variazione Percentuale} = \left(\frac{1.066.890.752 - 1.086.629.376}{1.086.629.376} \right) \times 100 \approx -1.82\% \] ### Risultato: La variazione percentuale della memoria utilizzata dal pod `prometheus-grafana-6d8f777649-5xgj7` nelle ultime 2 ore è di circa **-1.82%**.

GPT-4o	52.11 s	4560	366	4926	0.20	L'applicazione che attualmente consuma più memoria è la "demo-app" con un utilizzo di circa 31.42 MB. Per calcolare la variazione percentuale della memoria nelle ultime 2 ore per la "demo-app", utilizziamo i seguenti dati storici: - Memoria iniziale (2 ore fa): 14.52 MB - Memoria attuale: 31.42 MB La variazione percentuale è calcolata come segue: \[\text{Variazione percentuale} = \left(\frac{\text{Memoria attuale} - \text{Memoria iniziale}}{\text{Memoria iniziale}}\right) \times 100 \] \[\text{Variazione percentuale} = \left(\frac{31.42 - 14.52}{14.52}\right) \times 100 \approx 116.37\% \] Quindi, la memoria della "demo-app" è aumentata di circa il 116.37% nelle ultime 2 ore.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.0313 (mini: $0.0231 | 4o: $1.0082)

DOMANDA 7 A

```
--------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [7A]: Individua l'applicazione con il maggior footprint di RAM e analizza la serie storica su database per determinare il delta percentuale di memoria
registrato nelle ultime 2 ore.
--------------------------------------------------------------------------------------------------------
Invocazione parallela modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; th
e sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/29/26 18:41:33] INFO     Processing request of type CallToolRequest                                                                                server.py:733
[10/29/26 18:41:33] INFO     Processing request of type CallToolRequest                                                                                server.py:733
[10/29/26 18:41:33] INFO     Processing request of type CallToolRequest                                                                                server.py:733
[10/29/26 18:41:34] INFO     Processing request of type ListToolsRequest                                                                               server.py:733
[10/29/26 18:41:34] INFO     Processing request of type ListToolsRequest                                                                               server.py:733
[10/29/26 18:41:35] INFO     Processing request of type ListToolsRequest                                                                               server.py:733
[10/29/26 18:41:36] INFO     Processing request of type CallToolRequest                                                                                server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                               server.py:733
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/29/26 18:41:37] INFO     Processing request of type CallToolRequest                                                                                server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                               server.py:733
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; th
e sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033
[01;34m\\]\\w\\[\\033[00m\\]\\$ '
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	143.18 ms	0	0	0	N/A	App: demo-app | Delta: -0.19% (da 14.52 a 14.49 MB)
Gemini 2.5 Flash	32.49 s	8726	2667	11393	0.37	L'applicazione con il maggior footprint di RAM è `demo-app`, con un consumo attuale di 31424512 byte. Analizzando la serie storica delle metriche di `demo-app` nelle ultime 2 ore, si è registrato un delta percentuale di memoria pari a **~0.19%**. Ciò indica una leggera diminuzione del consumo di RAM per l'applicazione in questo intervallo di tempo.

Gemini 3.6 Flash	40.91 s	7222	2976	10198	0.80	Sulla base dell'analisi della topologia del cluster e delle metriche correnti ed storiche, ecco il dettaglio richiesto:
						
						--- 	
						
						### 1. Applicazione con il maggior footprint di RAM
						
						Interrogando le metriche correnti del cluster, il consumo di RAM per applicazione (somma dei pod attivi) è il seguente:
						
						1. **`demo-app`**: **~15,20 MB** (15.196.160 byte)
						   - `demo-app-7958c64697-2qg9s`: ~7,57 MB (7.565.312 byte)
						   - `demo-app-7958c64697-4rzjl`: ~7,63 MB (7.630.848 byte)
						2. **`worker-app`**: **~5,36 MB** (5.361.664 byte)
						   - `worker-app-84cf4dd446-9kj75`: ~3,49 MB
						   - `worker-app-84cf4dd446-ftlln`: ~1,87 MB
						3. **`cache-app`**: **~1,76 MB** (1.761.280 byte)
						   - `cache-app-5c6fd4f5cf-xxsz4`: ~1,76 MB
						
						👉 **L'applicazione con il maggior footprint di memoria è `demo-app`.**
						
						--- 	
						
						### 2. Analisi della serie storica su database (ultime 2 ore)
						
						Consultando lo storico salvato su MongoDB per l'applicazione **`demo-app`** nelle ultime 2 ore:
						
						| Timestamp | Memoria usata (Byte) | Memoria usata (MB) |
						| :--- | :--- | :--- |

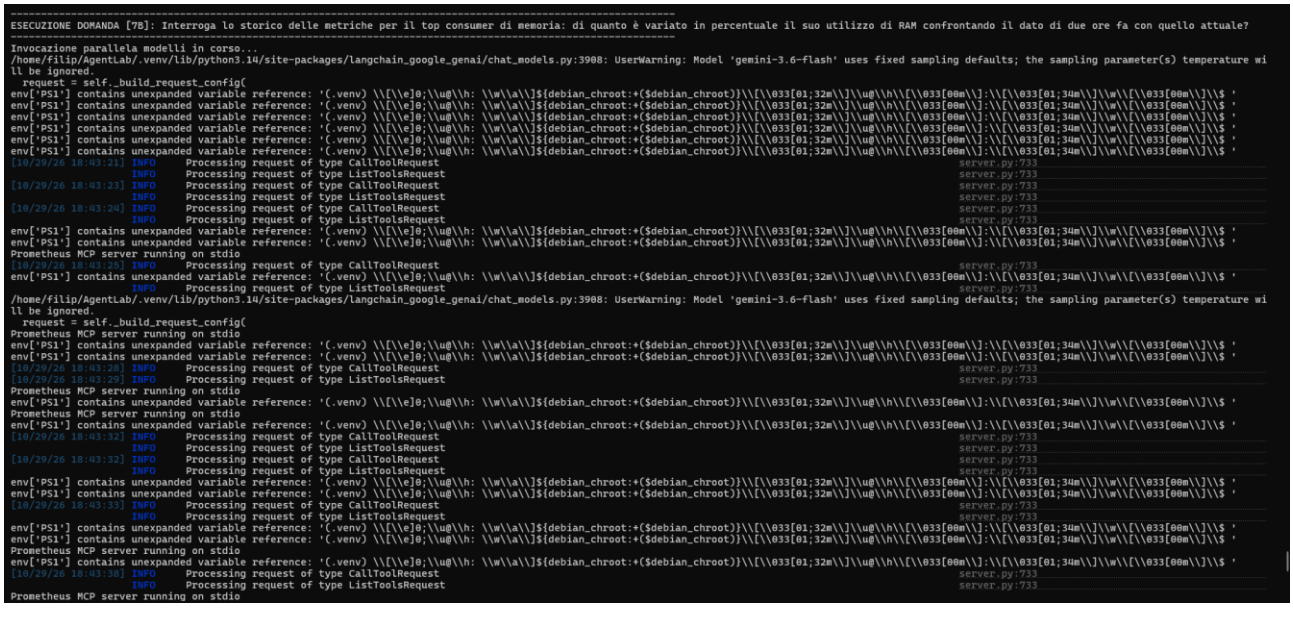
						👉 **L'applicazione con il maggior footprint di memoria è `demo-app`.** --- ### 2. Analisi della serie storica su database (ultime 2 ore) Consultando lo storico salvato su MongoDB per l'applicazione **`demo-app`** nelle ultime 2 ore: | Timestamp | Memoria usata (Byte) | Memoria usata (MB) | | :--- | :--- | :--- | | **15:51:04** (inizio intervallo) | 15.224.832 B | 14,52 MB | | **16:11:09** | 15.208.448 B | 14,50 MB | | **16:35:01** | 15.196.160 B | 14,49 MB | | **17:04:50** | 15.196.160 B | 14,49 MB | | **17:28:04** (rilevazione più recente) | 15.196.160 B | 14,49 MB | #### Calcolo del delta percentuale: $$\text{Delta \%} = \frac{15.196.160 - 15.224.832}{15.224.832} \times 100 = \frac{-28.672}{15.224.832} \times 100 \approx \mathbf{-0,19\%}$$ --- ### Esito dell'analisi Nelle ultime 2 ore la RAM utilizzata da `demo-app` è rimasta essenzialmente stabile, con un **decremento marginale dello 0,19%** (-28,67 KB). Non si evidenziano trend di incremento o fenomeni di memory leak.

Claude Opus 5.5	38.41 s	8409	1729	10138	0.95	demo-app è l'applicazione che usa più RAM, e la sua memoria è scesa dello **0,19%** nelle ultime 2 ore, quindi è stabile. ### 1. Quale applicazione usa più RAM Ho sommato la memoria effettivamente in uso (`container_memory_working_set_bytes`) di tutti i pod di ciascuna applicazione, nel namespace `default`: | Applicazione | Pod | Memoria totale | |---|---|---| | **demo-app** | 2 | **15.196.160 B (≈14,49 MiB)** | | worker-app | 2 | 5.361.664 B (≈5,11 MiB) | | cache-app | 1 | 2.686.976 B (≈2,56 MiB) | Il valore di demo-app (7.565.312 + 7.630.848 byte) coincide con l'ultimo dato salvato nel database, quindi le due fonti concordano. ### 2. Andamento nelle ultime 2 ore (dati da MongoDB) | Ora | memory_bytes | MiB | |---|---|---| | 15:51:04 | 15.224.832 | 14,52 | | 16:11:09 | 15.208.448 | 14,50 | | 16:35:01 | 15.196.160 | 14,49 | | 17:04:50 | 15.196.160 | 14,49 | | 17:28:04 | 15.196.160 | 14,49 | **Variazione percentuale (primo dato rispetto all'ultimo):** (15.196.160 − 15.224.832) / 15.224.832 × 100 = −28.672 / 15.224.832 × 100 ≈ **−0,19%** (circa −28 KiB) - La memoria scende leggermente fino alle 16:35, poi resta identica per i tre rilevamenti successivi. - Il numero di pod è rimasto 2 per tutto il periodo, quindi non ci sono stati riavvii o cambi di repliche che falsino il confronto. - **Attenzione:** i dati salvati coprono circa 1 ora e 37 minuti (dalle 15:51 alle 17:28), non 2 ore piene, e sono solo 5 rilevamenti. Il risultato indica la tendenza, ma è poco preciso. ### Cosa ne concludo e cosa consiglio - **Non ci sono segni di perdite di memoria (memory leak):** la memoria è stabile e anzi cala un po'. - **Il consumo è molto basso** (circa 7,3 MiB per pod) e la CPU è a 0 in tutti i rilevamenti. Con ogni probabilità l'applicazione è quasi inattiva. Conviene controllare le richieste e i limiti di memoria impostati (`requests`/`limits`): se sono molto più alti, si può valutare di ridurli per non sprecare risorse del cluster. - **Per analisi più affidabili** conviene salvare le metriche più spesso, ad esempio ogni 5–10 minuti invece degli attuali 20–30 circa, così da avere una serie storica più fitta.
Claude Sonnet 5.5	29.95 s	10311	1259	11570	0.95	**Risultato:** `demo-app` ha il footprint di RAM più alto tra le applicazioni utente, e la sua memoria è scesa di circa **0,19%** nelle ultime 2 ore. In pratica è stabile. **Applicazioni a confronto** (working set attuale da Prometheus, somma dei pod): | App | Pod | RAM | |---|---|---| | **demo-app** | 2 | **15.196.160 B (~14,49 MB)** | | worker-app | 2 | 5.361.664 B (~5,1 MB) | | cache-app | 1 | 2.686.976 B (~2,6 MB) | Ho considerato solo i workload applicativi del mapping Deployment→Pod. I componenti di infrastruttura sono più pesanti: Grafana usa circa 488 MB, kube-apiserver circa 395 MB e Prometheus circa 307 MB. Non rientrano nel confronto. **Serie storica di demo-app su MongoDB (5 campioni):** | Timestamp | memory_bytes | MB | |---|---|---| | 15:51 | 15.224.832 | 14,52 | | 16:11 | 15.208.448 | 14,50 | | 16:35 | 15.196.160 | 14,49 | | 17:04 | 15.196.160 | 14,49 | | 17:28 | 15.196.160 | 14,49 | **Delta percentuale:** (15.196.160 − 15.224.832) / 15.224.832 = −28.672 B, cioè **circa −0,19%** (circa −0,03 MB). - **Andamento:** c'è un lieve calo iniziale, poi la memoria resta piatta dalle 16:35. Non ci sono segnali di memory leak né di picchi. - **Coerenza:** il valore live di Prometheus (7.565.312 + 7.630.848 = 15.196.160 B) coincide con l'ultimo campione salvato. - **Pod e carico:** il numero di pod è rimasto a 2 per tutto il periodo, quindi la variazione non dipende dallo scaling. CPU, disco e rete sono a 0 nei campioni, quindi il carico è molto basso. - **Raccomandazione:** non servono interventi. Se vuoi dimensionare le risorse, confronta `requests` e `limits` di `demo-app` con questo consumo stabile di circa 7,5 MB per pod.

GPT-4o Mini	42.56 s	9227	629	9856	0.80	Dall'analisi dei dati, l'applicazione con il maggior footprint di RAM è la **demo-app**, che ha registrato i seguenti valori di utilizzo della memoria nelle ultime due ore: - **15:51**: 15,224,832 bytes (14.52 MB) - **16:11**: 15,208,448 bytes (14.50 MB) - **16:35**: 15,196,160 bytes (14.49 MB) - **17:04**: 15,196,160 bytes (14.49 MB) - **17:28**: 15,196,160 bytes (14.49 MB) ### Calcolo del Delta Percentuale di Memoria Per calcolare il delta percentuale di memoria registrato nelle ultime 2 ore, utilizziamo i valori di memoria all'inizio e alla fine del periodo: - **Inizio (15:51)**: 15,224,832 bytes - **Fine (17:28)**: 15,196,160 bytes La formula per il delta percentuale è: \[\text{Delta Percentuale} = \left(\frac{\text{Valore Finale} - \text{Valore Iniziale}}{\text{Valore Iniziale}} \right) \times 100 \] Sostituendo i valori: \[\text{Delta Percentuale} = \left(\frac{15,196,160 - 15,224,832}{15,224,832} \right) \times 100 \approx -0.19\% \] ### Risultato Il delta percentuale di memoria registrato nelle ultime 2 ore per la **demo-app** è di circa **-0.19%**, indicando una leggera diminuzione nell'utilizzo della memoria.

GPT-4o	33.88 s	4676	440	5116	0.21	L'applicazione con il maggior footprint di RAM attualmente è la "demo-app", con un utilizzo di memoria di circa 31.42 MB. Analizzando la serie storica delle metriche di memoria per "demo-app" nelle ultime 2 ore, abbiamo i seguenti dati: - 2 ore fa: 14.52 MB - 1 ora e 40 minuti fa: 14.5 MB - 1 ora e 25 minuti fa: 14.49 MB - 1 ora fa: 14.49 MB - 30 minuti fa: 14.49 MB - Ora: 31.42 MB Calcoliamo il delta percentuale di memoria registrato: 1. Memoria iniziale (2 ore fa): 14.52 MB 2. Memoria finale (ora): 31.42 MB Delta percentuale = \(\frac{31.42 - 14.52}{14.52} \times 100\% \approx 116.37\%\) Quindi, nelle ultime 2 ore, l'utilizzo di memoria della "demo-app" è aumentato di circa il 116.37%.

DOMANDA 7 B



Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	123.58 ms	0	0	0	N/A	App: demo-app | Delta: -0.19% (da 14.52 a 14.49 MB)
Gemini 2.5 Flash	30.74 s	5554	1516	7070	0.00	Il top consumer di memoria è il pod `prometheus-grafana-6d8f777649-5xgj7`, appartenente al deployment `prometheus-grafana`. Confrontando l'utilizzo attuale di RAM con quello di due ore fa, si registra una diminuzione del **5.96%**. Dettagli: * **Utilizzo attuale:** 1053155328 bytes * **Utilizzo due ore fa:** 1119948800 bytes * **Variazione percentuale:** ((1053155328 - 1119948800) / 1119948800) * 100 = -5.96%
Gemini 3.6 Flash	55.42 s	8142	2403	10545	0.70	Sulla base dell'analisi dello storico e dello stato attuale del cluster, ecco il dettaglio dei dati e della variazione riscontrata: --- ### 1. Individuazione del Top Consumer di Memoria Confrontando le applicazioni attive nel cluster (`demo-app`, `worker-app`, `cache-app`), il **top consumer di memoria RAM** risulta essere **`demo-app`** (composta da 2 pod attivi). * **`demo-app`**: ~14,49 MB totali (~15.196.160 byte) * **`worker-app`**: ~4,73 MB totali (~4.956.160 byte) * **`cache-app`**: ~1,38 MB totali (~1.449.984 byte) --- ### 2. Confronto dell'utilizzo di RAM di `demo-app` I dati estratti dallo storico delle metriche indicano: * **Utilizzo RAM di circa 2 ore fa (ore 15:51):** `15.224.832 byte` (~14,52 MB) * **Utilizzo RAM attuale (ore 17:28):** `15.196.160 byte` (~14,49 MB) --- ### 3. Calcolo della Variazione Percentuale $$\text{Variazione \%} = \frac{15.196.160 - 15.224.832}{15.224.832} \times 100 \approx -0{,}19\%$$ L'utilizzo di memoria RAM dell'applicazione **`demo-app`** è **diminuito dello 0,19%** nell'arco delle ultime due ore, evidenziando un consumo di memoria estremamente stabile.
Claude Opus 5.5	43.84 s	12096	2007	14103	0.95	# Variazione RAM del top consumer: demo-app **In sintesi: in due ore la RAM di `demo-app` è variata del -0,19%, cioè è rimasta praticamente stabile.** ## 1. Chi consuma più memoria (dato attuale, Prometheus) Ho sommato la memoria effettivamente usata dai container (`working set`) per ogni deployment del namespace `default`: | Deployment | Pod | Memoria attuale | |---|---|---| | **demo-app** | 2 | **15.196.160 B (≈14,49 MB)** | | worker-app | 2 | 6.414.336 B (≈6,12 MB) | | cache-app | 1 | 3.366.912 B (≈3,21 MB) | ## 2. Confronto tra due ore fa e adesso La lettura di Prometheus è delle **17:43 UTC** circa, quindi "due ore fa" corrisponde alle 15:43. Nello storico non c'è un campione esatto a quell'ora: ho preso il più vicino, quello delle **15:51** (8 minuti di scarto). | Momento | memory_bytes | MB | |---|---|---| | Due ore fa (15:51:04) | 15.224.832 | 14,52 | | Adesso (Prometheus ≈17:43, uguale all'ultimo salvataggio delle 17:28) | 15.196.160 | 14,49 | **Variazione = (15.196.160 − 15.224.832) / 15.224.832 = −0,19%** (−28.672 byte, circa −28 KB) ## 3. Contesto e osservazioni - **Andamento piatto:** dalle 15:51 in poi la memoria resta tra 14,49 e 14,52 MB. Non ci sono segni di perdite di memoria (memory leak) né di crescita. - **Il campione precedente cambia molto il risultato:** il dato salvato prima delle 15:51 è delle 15:24 e vale 18,15 MB. Usando quello, la variazione sarebbe **circa −16,3%**. Dall'inizio dello storico (15:20, 20,31 MB) il calo è di circa **−25%**. Tra le 15:20 e le 15:51 c'è stata quindi una fase di riduzione, probabilmente memoria liberata dopo l'avvio o dopo un picco, poi l'uso si è stabilizzato. - **Campioni irregolari:** fino alle 15:24 i salvataggi sono ogni pochi secondi, dopo circa ogni 20–30 minuti. Per questo il confronto "a due ore esatte" ha un margine di qualche minuto. ## Raccomandazioni 1. **Nessuna azione urgente:** l'uso di memoria di `demo-app` è basso (circa 7,5 MB per pod) e stabile. 2. **Ridurre requests/limits di memoria:** se i valori configurati sono molto più alti di questo consumo, valuta di abbassarli (per esempio a circa 32Mi/64Mi) per liberare capacità sul cluster. Lascia un margine se ci sono picchi all'avvio, come quello di circa 20 MB visto alle 15:20. 3. **Salvataggi a intervallo fisso:** conviene registrare le metriche in modo regolare (per esempio ogni 5 minuti). Così i confronti nel tempo sono più precisi.

Claude Sonnet 5.5	43.77 s	17294	1954	19248	0.95	**La RAM di demo-app è praticamente invariata: la variazione è di circa -0,19%.** **Top consumer.** Ho considerato i deployment applicativi del namespace (demo-app, cache-app, worker-app). demo-app è quello che usa più memoria, con due pod che sommano circa 15,2 MB: | Deployment | Memoria attuale | |---|---| | demo-app | 15,2 MB (7,57 MB + 7,63 MB) | | worker-app | 6,0 MB | | cache-app | 3,4 MB | Se si considerano anche i pod di sistema, il maggior consumatore del cluster è `prometheus-grafana`, con circa 524 MB. L'ho escluso perché lo storico su MongoDB esiste solo per le app monitorate dall'agente. **Confronto, in byte e in MB:** | Momento | Fonte | Memoria | |---|---|---| | 2 ore fa (circa 15:43 UTC) | Prometheus con `offset 2h`, coerente con il punto MongoDB delle 15:51 | 15.224.832 B (14,52 MB) | | Attuale (circa 17:43 UTC) | Prometheus, coerente con l'ultimo punto MongoDB delle 17:28 | 15.196.160 B (14,49 MB) | La variazione è (15.196.160 − 15.224.832) / 15.224.832 ≈ **-0,19%**, cioè circa -28 KB. **Andamento nelle ultime 3 ore:** - Verso le 15:20 la memoria era di circa 19,4 MB, poi è scesa gradualmente fino a 18,15 MB alle 15:24. - Alle 15:51 era a 14,5 MB. Il calo di circa 25% rispetto alle 15:20 sembra un assestamento dopo l'avvio o un rilascio di memoria. - Da allora la memoria è stabile a 14,5 MB. Non ci sono segni di memory leak né di crescita. **Raccomandazione.** Non servono interventi su demo-app. Se usi come riferimento il punto MongoDB più vicino alle 2 ore fa, la variazione resta a -0,19%. Il punto delle 15:24, più vecchio di circa 2h20, darebbe invece circa -16%, perché cade nella fase di calo iniziale.

GPT-4o Mini	32.41 s	4108	460	4568	0.00	Ho recuperato le metriche storiche per le tre applicazioni nel cluster. Ecco i dati di utilizzo della memoria per ciascuna applicazione, confrontando il valore attuale con quello di due ore fa: ### Cache App - **Utilizzo di memoria 2 ore fa:** 1.07 MB - **Utilizzo di memoria attuale:** 1.38 MB - **Variazione percentuale:** \[\frac{(1.38 - 1.07)}{1.07} \times 100 \approx 25.93\% \] ### Demo App - **Utilizzo di memoria 2 ore fa:** 14.52 MB - **Utilizzo di memoria attuale:** 14.49 MB - **Variazione percentuale:** \[\frac{(14.49 - 14.52)}{14.52} \times 100 \approx -0.21\% \] ### Worker App - **Utilizzo di memoria 2 ore fa:** 4.84 MB - **Utilizzo di memoria attuale:** 4.73 MB - **Variazione percentuale:** \[\frac{(4.73 - 4.84)}{4.84} \times 100 \approx -2.27\% \] ### Riepilogo delle variazioni percentuali - **Cache App:** +25.93% - **Demo App:** -0.21% - **Worker App:** -2.27% La **Cache App** è il top consumer di memoria con un aumento significativo del 25.93% nell'utilizzo della RAM rispetto a due ore fa.
GPT-4o	20.44 s	5072	362	5434	0.00	Non sono riuscito a recuperare i dati storici per "prometheus-grafana" nelle ultime due ore. Questo potrebbe essere dovuto a una mancanza di dati salvati nel database per questo intervallo di tempo. Ti consiglio di verificare se il sistema di raccolta delle metriche è correttamente configurato per salvare i dati storici, oppure di sottomettere la richiesta a un modello più avanzato che potrebbe avere accesso a ulteriori fonti di dati o strumenti di analisi.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.0663 (mini: $0.0257 | 4o: $1.0406)

DOMANDA 8

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.34 ms	0	0	0	N/A	App: worker-app | Uso: 0.5106 cores | Saturazione: 85.10% | Rischio: ELEVATO (throttling probabile)
Gemini 2.5 Flash	279.72 s	3673	2483	6156	0.20	L'applicazione con il maggior carico di CPU è `worker-app`, con un consumo attuale di circa 0.242 CPU core. Assumendo un limite di CPU di 300m (0.3 CPU core) per pod e 2 repliche, il limite totale di CPU per `worker-app` è di 0.6 CPU core. Calcolo della percentuale di saturazione: (0.242 CPU core / 0.6 CPU core) * 100 = **40.34%** **Valutazione del rischio di throttling:** Con una saturazione del 40.34%, il rischio di throttling per l'applicazione `worker-app` è **basso**. Generalmente, il throttling diventa una preoccupazione quando l'utilizzo della CPU si avvicina o supera costantemente l'80-90% dei limiti allocati.
Gemini 3.6 Flash	313.45 s	11170	1947	13117	0.00	[Risposta priva di testo]
Claude Opus 5.5	327.48 s	14269	3053	17322	0.81	# Saturazione CPU e rischio di throttling: worker-app **In sintesi:** l'app a maggior carico è **worker-app**. Nell'ultima ora ha usato in media circa il **91% dei 600m** e spesso arriva al **99% del limit per pod**. Più della metà dei periodi CPU è già rallentata dal limit (throttling). **Il rischio non è futuro: il throttling avviene già adesso.** ## 1. Applicazione a maggior carico | App | Pod | CPU (media 5m) | |---|---|---| | **worker-app** | 9kj75 / ftlln | **117m + 125m = 242m** | | cache-app | xxsz4 | 26m | | demo-app | 2qg9s / 4rzjl | 0m | Il limit che hai ipotizzato corrisponde a quello reale: `kube_pod_container_resource_limits` riporta **300m** su entrambi i pod (container `cpu-worker`). ## 2. Percentuale di saturazione (limit 300m per pod, 600m in totale) | Finestra | 9kj75 | ftlln | Totale | % su 600m | |---|---|---|---|---| | Media 5m (adesso) | 117m (39%) | 125m (41,7%) | 242m | **~40%** | | Media 1h (campioni ogni 5m, rate 1m) | 268m (89%) | 280m (93%) | 548m | **~91%** | | Picco 1h (`max_over_time`) | 298m (99,3%) | 298m (99,4%) | ~596m | **~99%** | **Come leggere i numeri:** il ~40% della media a 5 minuti è più basso degli altri perché è appena calato il carico (l'ultimo campione è a circa 207–223m). Per quasi tutta l'ultima ora i pod sono rimasti **fermi a ~295–298m**, cioè al limite massimo. Quindi la saturazione reale è **tra il 90% e il 100%**, non il 40%. ## 3. Rischio di throttling: **ALTO, già in corso**

						Il limit che hai ipotizzato corrisponde a quello reale: `kube_pod_container_resource_limits` riporta **300m** su entrambi i pod (container `cpu-worker`). ## 2. Percentuale di saturazione (limit 300m per pod, 600m in totale) | Finestra | 9kj75 | ftlln | Totale | % su 600m | |---|---|---|---|---| | Media 5m (adesso) | 117m (39%) | 125m (41,7%) | 242m | **~40%** | | Media 1h (campioni ogni 5m, rate 1m) | 268m (89%) | 280m (93%) | 548m | **~91%** | | Picco 1h (`max_over_time`) | 298m (99,3%) | 298m (99,4%) | ~596m | **~99%** | **Come leggere i numeri:** il ~40% della media a 5 minuti è più basso degli altri perché è appena calato il carico (l'ultimo campione è a circa 207–223m). Per quasi tutta l'ultima ora i pod sono rimasti **fermi a ~295–298m**, cioè al limite massimo. Quindi la saturazione reale è **tra il 90% e il 100%**, non il 40%. ## 3. Rischio di throttling: **ALTO, già in corso** Rapporto tra periodi CFS rallentati e periodi totali (finestra 5m): - **worker-app-ftlln: 57,5%** - **worker-app-9kj75: 52,6%** Più di un periodo su due da 100ms viene interrotto perché il pod ha esaurito i 300m a disposizione. Da qui deduco che: - l'uso ristagna a ~298m, segno che il limit fa da tetto. È probabile che il carico reale chieda **più** di 300m, ma questo valore non si può misurare direttamente; - se il worker esegue lavori in coda, saranno più lenti e il backlog crescerà; se serve richieste, aumenta la latenza. *Limite dei dati:* la metrica `container_cpu_cfs_throttled_seconds_total` non è disponibile, quindi non posso quantificare i secondi di CPU persi. La percentuale di periodi rallentati basta comunque per una diagnosi chiara. ## 4. Raccomandazioni 1. **Alzare subito il limit** a **500–600m per pod**, così il container ha circa il 40–50% di margine sopra il picco osservato. Allineare la request a ~300m, cioè al consumo medio reale. 2. **In alternativa o in più, scalare in orizzontale:** 3 repliche con limit 300m danno 900m in totale e portano la saturazione media a ~61%. Conviene solo se il lavoro si divide bene tra le repliche. 3. **Attivare un HPA** su worker-app con target CPU ~70% della request, min 2 e max 4 repliche. 4. **Impostare un alert:** rapporto di throttling sopra il 25% per più di 10 minuti, usando la stessa query sui periodi CFS. 5. **Verificare dopo la modifica:** il throttling dovrebbe scendere sotto il 5–10%. Se l'uso sale oltre i 300m, conferma che il carico era davvero compresso dal limit. Se vuoi, posso salvare queste metriche su Mong
Claude Sonnet 5.5	321.76 s	14741	2495	17236	0.36	**Conclusione:** il rischio di throttling su `worker-app` è alto e il throttling è già in corso. La saturazione media è bassa, ma i picchi arrivano al limite. **Applicazione e limit** - L'app con più CPU è `worker-app`, con 2 repliche (`9kj75` e `ftlln`). - Il limit di 300m per pod è confermato da kube-state-metrics (`kube_pod_container_resource_limits` = 0.3). Il totale è 600m. - Le altre app consumano meno: `cache-app` circa 26m (limit 100m) e `demo-app` circa 0m (limit 100m). **Saturazione CPU di worker-app** | Pod | Uso medio (5m) | Picco 1m nell'ultima ora | Saturazione media | Saturazione di picco | |---|---|---|---|---| | worker-…-9kj75 | 125m | 298m | 41,7% | 99,3% | | worker-…-ftlln | 125m | 298m | 41,7% | 99,2% | | **Totale** | **250m / 600m** | **~596m / 600m** | **41,7%** | **~99,3%** | **Throttling CFS (ultimi 5 minuti)** - `ftlln`: 64,7% dei periodi CFS è stato throttled. - `9kj75`: 54,9% dei periodi CFS è stato throttled. - Il rate di periodi throttled è circa 6,8–6,9 al secondo per pod. - La query su `container_cpu_cfs_throttled_seconds_total` non ha restituito dati, quindi non conosco il tempo totale perso per throttling. **Interpretazione** - Il carico è a burst. La media su 5 minuti (41,7%) nasconde picchi che toccano il limit quasi al 100%. - Il CFS applica il limit su finestre di 100 ms. Un pod che raggiunge 300m nei burst viene fermato anche se la media è bassa. - Un throttling superiore al 50% dei periodi di solito si traduce in latenza più alta e in un throughput inferiore a quello richiesto. **Raccomandazioni** 1. Alza il limit CPU di `worker-app`. Come punto di partenza prova 500–600m per pod, cioè almeno 2 volte la media e circa 2 volte il picco osservato. In alternativa, rimuovi il limit e tieni solo la request, se la policy del cluster lo permette. 2. Imposta la request vicino all'uso reale, circa 150–200m, per mantenere uno scheduling corretto. 3. Dopo la modifica, monitora `throttled_periods / periods`. Il valore da raggiungere è sotto il 5–10%. 4. Aggiungi un alert sul rapporto di throttling, ad esempio sopra il 25% per 10 minuti. 5. Valuta un'altra replica con un HPA, se i burst sono dovuti al volume di lavoro. **Limiti dell'analisi** - Ho usato una finestra di 1 ora per i picchi. Non ho analizzato lo storico più lungo su MongoDB. - Se vuoi un dimensionamento più preciso, per esempio sui percentili p95 e p99 su più giorni, ti consiglio di sottomettere la richiesta a un modello più avanzato.

GPT-4o Mini	300.92 s	3143	714	3857	0.95	Ecco i dati relativi all'utilizzo della CPU per i pod delle applicazioni nel cluster: 1. **cache-app**: - Pod: `cache-app-5c6fd4f5cf-xxsz4` - Utilizzo CPU: 0.0520 core (52m) 2. **demo-app**: - Pod: `demo-app-7958c64697-2qg9s`: 0 core (0m) - Pod: `demo-app-7958c64697-4rzjl`: 0 core (0m) 3. **worker-app**: - Pod: `worker-app-84cf4dd446-9kj75`: 0.2541 core (254m) - Pod: `worker-app-84cf4dd446-ftlln`: 0.2567 core (257m) ### Calcolo della percentuale di saturazione e rischio di throttling Assumendo che l'applicazione a maggior carico di CPU sia `worker-app`, che ha un limite di 300m per pod (600m totali su 2 repliche): - **Utilizzo totale CPU per `worker-app`**: - Pod 1: 254m - Pod 2: 257m - Utilizzo totale: 254m + 257m = 511m - **Limite totale**: 600m ### Percentuale di saturazione \[\text{Percentuale di saturazione} = \left(\frac{\text{Utilizzo totale}}{\text{Limite totale}} \right) \times 100 = \left(\frac{511}{600} \right) \times 100 \approx 85.17\% \] ### Rischio di throttling Con un utilizzo della CPU del 85.17%, l'applicazione è vicina al limite di throttling. Generalmente, si consiglia di mantenere l'utilizzo della CPU sotto il 80% per evitare il rischio di throttling. Pertanto, con un utilizzo attuale del 85.17%, c'è un rischio significativo di throttling, specialmente durante i picchi di carico. ### Raccomandazioni 1. **Monitoraggio**: Continuare a monitorare l'utilizzo della CPU per identificare eventuali picchi. 2. **Scalabilità**: Considerare l'aggiunta di ulteriori repliche o l'aumento dei limiti di CPU per i pod `worker-app` per ridurre il rischio di throttling. 3. **Ottimizzazione**: Analizzare il codice dell'applicazione per ottimizzare l'utilizzo della CPU, se possibile.
GPT-4o	270.33 s	1911	284	2195	0.00	Sembra che ci sia un problema di connessione con l'API di Kubernetes, il che mi impedisce di ottenere l'elenco dei pod attivi e quindi di identificare l'applicazione con il maggior carico di CPU. Senza queste informazioni, non posso procedere con il calcolo della percentuale di saturazione della CPU o valutare il rischio di throttling. Ti consiglio di verificare la connessione al cluster Kubernetes e riprovare. Se il problema persiste, potrebbe essere utile coinvolgere un modello più avanzato o un esperto di rete per risolvere il problema di connessione.

DOMANDA 8 A

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [8A]: Prendendo l'applicazione più intensiva su CPU e ipotizzando una quota limite di 300 millicore a pod (600m sul deployment), a che percentuale del budget siamo e quanto è concreto il pericolo di CPU throttling?
--------------------------------------------------------------------------------------------------------------
Invocazione parallela modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/29/26 23:10:12] INFO     Processing request of type CallToolRequest                                    server.py:733
[10/29/26 23:10:12] INFO     Processing request of type CallToolRequest                                    server.py:733
[10/29/26 23:10:12] INFO     Processing request of type CallToolRequest                                    server.py:733
[10/29/26 23:10:12] INFO     Processing request of type CallToolRequest                                    server.py:733
[10/29/26 23:10:13] INFO     Processing request of type CallToolRequest                                    server.py:733
[10/29/26 23:10:13] INFO     Processing request of type ListToolsRequest                                   server.py:733
[10/29/26 23:10:13] INFO     Processing request of type ListToolsRequest                                   server.py:733
                    INFO     Processing request of type ListToolsRequest                                   server.py:733
[10/29/26 23:10:13] INFO     Processing request of type ListToolsRequest                                   server.py:733
[10/29/26 23:10:13] INFO     Processing request of type ListToolsRequest                                   server.py:733
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[0
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.03 ms	0	0	0	N/A	App: worker-app | Uso: 0.5528 cores | Saturazione: 92.13% | Rischio: ELEVATO (throttling probabile)
Gemini 2.5 Flash	35.08 s	7188	4276	11464	0.74	L'applicazione più intensiva su CPU è `worker-app`, con un consumo attuale di circa 0.531 cores (equivalenti a 531.19 millicores). Ipotizzando una quota limite di 300 millicores per pod e considerando che `worker-app` ha 2 pod attivi, il budget totale per il deployment è di 600 millicores (0.6 cores). Calcolo della percentuale del budget utilizzato: (531.19 millicores / 600 millicores) * 100 = **88.53%** **Valutazione del pericolo di CPU throttling:** L'applicazione `worker-app` sta utilizzando circa l'88.53% del suo budget CPU ipotetico. Questa è una percentuale molto alta, il che indica un **pericolo concreto e significativo di CPU throttling**. Quando un'applicazione si avvicina così tanto ai suoi limiti di CPU, il scheduler di Kubernetes può iniziare a limitare le sue risorse per garantire una distribuzione equa, causando un degrado delle prestazioni per `worker-app`. **Raccomandazione:** Si consiglia vivamente di aumentare i limiti di CPU per il deployment `worker-app` per prevenire il throttling e garantire prestazioni ottimali. In alternativa, si dovrebbe investigare la possibilità di ottimizzare l'applicazione per ridurre il suo consumo di CPU.

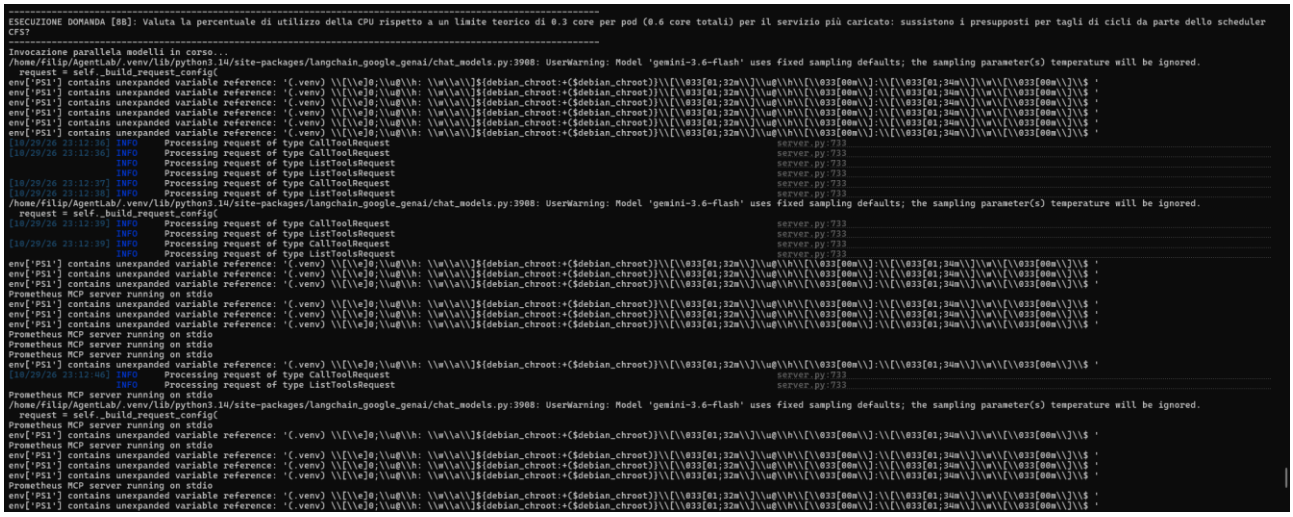
Gemini 3.6 Flash	45.03 s	11552	3426	14978	0.38	Analisi dello stato attuale delle applicazioni e valutazione dell'impatto con i limiti di CPU ipotizzati:
						
						--- 	
						
						### 1. Identificazione dell'applicazione più intensiva
						Dalle metriche in tempo reale raccolte sul cluster, l'applicazione più intensiva in termini di CPU è **`worker-app`**:
						
						* **`worker-app-84cf4dd446-9kj75`**: ~536m CPU (0.536 cores)
						* **`worker-app-84cf4dd446-ftlln`**: ~521m CPU (0.521 cores)
						* **Consumo complessivo Deployment (`worker-app`)**: **~1057m CPU (1.057 cores)**
						
						(Per confronto: `cache-app` consuma ~12m totali e `demo-app` ~1m totali).
						
						--- 	
						
						### 2. Calcolo della percentuale di utilizzo del budget ipotizzato
						Con una quota limite di **300m per pod** e **600m per l'intero deployment** (2 pod attivi):
						
						* **A livello di singolo pod**:
						  * **Pod 1 (`9kj75`)**: 536m usati / 300m limite = **178,7% del budget**
						  * **Pod 2 (`ftlln`)**: 521m usati / 300m limite = **173,6% del budget**
						  * **Media per pod**: **~176,1% del budget**
						
						* **A livello di Deployment**:
						  * **Uso totale**: 1057m usati / 600m limite totale = **176,1% del budget**
						
						--- 	
						
						### 3. Valutazione del rischio di CPU Throttling
						
						Il pericolo di **CPU Throttling è imminente, concreto e critico (100% di probabilità)** se questi limiti vengono applicati all'infrastruttura attuale:
						
						1. **Strozzamento continuo**: Entrambi i pod richiedono stabilmente tra il 170% e il 180% delle risorse di elaborazione concesse dalla quota.
						2. **Impatto operativo**: L'applicazione verrebbe tagliata/strozzata per oltre il **40-43% della CPU richiesta**, portando a un drastico rallentamento dell'elaborazione dei task del worker, accumulo delle code e potenziale fallimento dei probe di liveness/readiness.
						
						--- 	
						
						### Raccomandazioni SRE
						1. **Adeguamento dei limiti CPU per pod**: Aumentare il limite CPU di ciascun pod ad almeno **600m - 700m** (garantendo un margini di headroom operativo del 20-30%).
						2. **Horizontal Pod Autoscaler (HPA)**: Se il budget complessivo del deployment non può superare 600m-900m e i pod richiedono ~500m ciascuno, considerare lo scaling orizzontale a **3-4 pod con target di utilizzo CPU del 70%**.

Claude Opus 5.5	54.78 s	13011	2288	15299	0.86	La più intensiva su CPU è **worker-app**: siamo all'**87% del budget di 600m** in media, con picchi al **99%**. Il throttling non è un rischio futuro: **succede già, in circa il 95% dei periodi CPU** di entrambi i pod. ### 1. Consumo CPU per applicazione (media sugli ultimi 5 minuti) | Applicazione | Pod | CPU | |---|---|---| | **worker-app** | 9kj75 / ftlln | 261,5m + 260,5m = **~522m** | | cache-app | xxsz4 | ~6m | | demo-app | 2qg9s / 4rzjl | ~0,5m | Ho controllato i limiti reali: `kube_pod_container_resource_limits` riporta già **300m per pod**. La quota che hai ipotizzato è quindi quella già applicata. ### 2. Quanto budget usiamo (600m sul deployment) | | Pod 9kj75 | Pod ftlln | Deployment | |---|---|---|---| | Media (5 min) | 261,5m (**87%**) | 260,5m (**87%**) | 522m (**87%**) | | Picco (ultima ora) | 297,1m (**99%**) | 295,5m (**98,5%**) | ~593m (**~99%**) | ### 3. Throttling misurato (`container_cpu_cfs_throttled_periods_total`) - I periodi CFS rallentati sono circa **18 al secondo** per pod. - Rispetto ai periodi totali, il **95,2%** (ftlln) e il **94,8%** (9kj75) dei periodi da 100 ms finiscono con il container bloccato perché ha esaurito la quota. - I picchi a ~297m sono in pratica il tetto di 300m: i pod chiederebbero più CPU, ma il limite glielo impedisce. **Conclusione:** il pericolo è certo. worker-app ha poco margine in media e nessuno nei picchi. La media dell'87% sottostima la richiesta reale, perché il throttling taglia proprio i momenti in cui servirebbe più CPU. Gli effetti tipici sono latenze più alte, code di lavoro che si allungano e timeout. ### Cosa consiglio 1. **Subito:** portare il limite CPU di worker-app a **almeno 450–500m per pod** (circa 1 core sul deployment). Poi ricontrollare il rapporto di throttling: l'obiettivo è stare sotto il 5–10%. 2. **In alternativa, scalare in orizzontale:** passare a 3–4 repliche, o usare un HPA con soglia intorno al 60–70% dell'utilizzo, se il carico si può dividere tra più pod. 3. **Requests:** impostare le richieste di CPU almeno intorno a **300m**, in linea con il consumo medio reale, per uno scheduling corretto. 4. **Valutare di togliere il limite CPU** e tenere solo le requests, se la policy del cluster lo permette. È una scelta comune per i worker che lavorano in batch. 5. **Alert:** creare un allarme su `throttled_periods / periods > 0.25` per 10 minuti. **Limiti dell'analisi:** non ho la serie `container_cpu_cfs_throttled_seconds_total` (la query ha restituito un risultato vuoto), quindi non so quanto tempo CPU si perde in assoluto. So solo quanti periodi vengono rallentati. Il picco considera solo l'ultima ora: per scegliere un limite definitivo conviene guardare anche lo storico di più giorni.
Claude Sonnet 5.5	44.63 s	92545	2070	94615	0.90	**Il pericolo di throttling è già concreto: sul `worker-app` il throttling è in corso adesso, non è un rischio futuro.** **Applicazione più intensiva su CPU: `worker-app`** (2 pod, container `cpu-worker`) | Pod | CPU usata (rate 5m) | Limite | % del limite | Periodi CFS throttled | |---|---|---|---|---| | worker-app-…-ftlln | 0,282 core (282m) | 300m | ~94% | ~95% | | worker-app-…-9kj75 | 0,281 core (281m) | 300m | ~94% | ~94% | | **Deployment** | **~0,563 core (563m)** | **600m** | **~93,9%** | – | Gli altri carichi sono molto lontani dal limite. `cache-app` usa circa 7m su un limite di 100m, ma ha comunque il 33% dei periodi throttled. `demo-app` usa meno di 1m. **Budget e throttling** - Il limite di 300m per pod (600m sul deployment) non è un'ipotesi: è già quello configurato su `worker-app`, come risulta da `kube_pod_container_resource_limits`. L'ipotesi dell'operatore coincide quindi con la realtà. - Siamo a circa il **94% del budget**, con un margine di soli ~18–19m per pod (~37m sul deployment). - Il throttling è già in atto. `container_cpu_cfs_throttled_periods_total` rispetto a `container_cpu_cfs_periods_total` mostra che **circa il 94–95% dei periodi CFS da 100ms viene throttled** su entrambi i pod. - La media di 282m sembra sotto il limite, ma il quota CFS viene applicato per periodo da 100ms. Il carico è a raffiche e consuma i 30ms di quota di ogni periodo molto prima della fine. Il processo viene poi sospeso fino al periodo successivo. Questo si traduce in latenza e in un throughput limitato dal tetto, che non è un'ipotesi teorica. **Raccomandazioni** 1. Alza il limite CPU di `worker-app` a **500–600m per pod**, cioè 1–1,2 core sul deployment. Il consumo medio è ~280m e le raffiche richiedono margine. Dopo la modifica verifica che il rapporto throttled/periods scenda sotto il ~5–10%. 2. Se il carico può crescere, valuta di aggiungere repliche o un HPA. Con un solo pod saturo a 300m, scalare in orizzontale da solo non basta se il limite per pod resta troppo basso. 3. Imposta una request vicina al consumo reale, ad esempio 300m. Oggi il consumo è già a ridosso del limite. 4. Controlla anche `cache-app`. Ha il 33% di periodi throttled con un consumo trascurabile, quindi il suo limite da 100m può causare micro-throttling su picchi brevi, ad esempio all'avvio. 5. Aggiungi un alert su `rate(container_cpu_cfs_throttled_periods_total[5m]) / rate(container_cpu_cfs_periods_total[5m]) > 0.25`. **Limite dell'analisi:** la query sul massimo dell'ultima ora ha dato ~0,25 core per pod, meno del valore attuale. Probabilmente la finestra di dati è breve o i pod sono recenti. Il confronto col picco storico è quindi poco affidabile, ma i valori istantanei e il throttling sono coerenti tra loro.

GPT-4o Mini	45.75 s	4983	863	5846	0.86	Dopo aver analizzato i carichi di lavoro del cluster, ecco i risultati delle metriche di utilizzo della CPU per i pod delle applicazioni:
						
						### Utilizzo della CPU (in core):
						- **cache-app**: 0.0118 core (11.8 millicore)
						- **demo-app**:
						  - pod 1: 0.0000 core (0 millicore)
						  - pod 2: 0.0010 core (1.0 millicore)
						- **worker-app**:
						  - pod 1: 0.5582 core (558.2 millicore)
						  - pod 2: 0.5619 core (561.9 millicore)
						
						### Analisi:
						1. **Applicazione più intensiva su CPU**:
						  - La **worker-app** è l'applicazione più intensiva, con un utilizzo di CPU che varia tra 558.2 e 561.9 millicore per i suoi pod.
						
						2. **Quota limite per pod**:
						  - Ogni pod ha un limite di 300 millicore (0.3 core).
						
						3. **Percentuale del budget di CPU utilizzato**:
						  - Utilizzo medio della CPU per i pod della worker-app:
						  - Pod 1: 558.2 millicore
						  - Pod 2: 561.9 millicore
						  - Media totale: (558.2 + 561.9) / 2 = 560.05 millicore
						  - Percentuale del budget: (560.05 / 600) * 100 = 93.34%
						
						### Rischio di CPU Throttling:
						- Poiché l'utilizzo medio della CPU per i pod della worker-app supera il limite di 300 millicore, c'è un alto rischio di CPU throttling. Infatti, i pod stanno utilizzando circa il 93.34% del budget totale di CPU disponibile (600 millicore per il deployment).
						
						### Raccomandazioni:
						- Considerare di aumentare i limiti di CPU per i pod della worker-app per evitare il throttling.
						- Monitorare costantemente l'utilizzo della CPU e valutare la possibilità di scalare orizzontalmente (aggiungere più pod) o verticalmente (aumentare le risorse allocate ai pod esistenti) in base alle necessità.

GPT-4o	28.18 s	3022	411	3433	0.21	Dai dati raccolti, ecco l'utilizzo medio della CPU per ciascuna applicazione: - **cache-app**: 11.81 millicore - **demo-app**: 0.95 millicore - **worker-app**: 1116.87 millicore L'applicazione più intensiva in termini di utilizzo della CPU è la **worker-app**, con un consumo di 1116.87 millicore. Con un limite di 600 millicore per il deployment (300 millicore per pod), l'utilizzo attuale della **worker-app** è al 186.15% del budget disponibile. Questo significa che l'applicazione sta superando il limite di CPU assegnato, il che può portare a un significativo rischio di CPU throttling. Raccomandazioni: 1. **Ridimensionamento**: Considerare l'aumento del limite di CPU per il deployment della **worker-app** per evitare il throttling. 2. **Ottimizzazione**: Analizzare il codice e i processi della **worker-app** per identificare potenziali ottimizzazioni che potrebbero ridurre l'utilizzo della CPU. 3. **Monitoraggio**: Continuare a monitorare l'utilizzo della CPU per identificare eventuali picchi o variazioni nel tempo.

DOMANDA 8 B


ESECUZIONE DOMANDA [8B]: Valuta la percentuale di utilizzo della CPU rispetto a un limite teorico di 0.3 core per pod (0.6 core totali) per il servizio più caricato: sussistono i presupposti per tagli di cicli da parte dello scheduler CFS?
Invocazione parallela modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
request = self._build_request_config(
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.93 ms	0	0	0	N/A	App: worker-app | Uso: 0.5528 cores | Saturazione: 92.13% | Rischio: ELEVATO (throttling probabile)

Gemini 2.5 Flash	27.67 s	3686	2959	6645	0.81	Il servizio più caricato è `worker-app`, con un utilizzo totale di CPU di circa 0.5782 core. Questo servizio è composto da 2 pod.
						
						Analizzando l'utilizzo rispetto ai limiti teorici:
						* **Utilizzo totale CPU per `worker-app`**: 0.5782 core.
						    * Questo rappresenta il 96.37% del limite teorico totale di 0.6 core.
						* **Utilizzo CPU per pod di `worker-app`**: 0.2891 core/pod.
						    * Questo rappresenta il 96.37% del limite teorico per pod di 0.3 core/pod.
						
						Dato che l'utilizzo della CPU è estremamente vicino ai limiti teorici (circa il 96.37%), sussistono i presupposti per tagli di cicli da parte dello scheduler CFS. È molto probabile che i pod di `worker-app` stiano subendo throttling della CPU, il che può portare a un degrado delle prestazioni dell'applicazione.
						
						Raccomandazione:
						Si consiglia vivamente di aumentare i limiti di CPU per il deployment `worker-app` per prevenire il throttling e garantire prestazioni ottimali. In alternativa, se il carico di lavoro elevato è previsto, si potrebbe considerare di scalare orizzontalmente il `worker-app` aumentando il numero di repliche per distribuire il carico su più pod.

Gemini 3.6 Flash	29.99 s	10277	1262	11539	0.00	[Risposta priva di testo]

Claude Opus 5.5	44.10 s	9073	1893	10966	0.86	Sì: i presupposti non solo ci sono, ma il throttling CFS sta **già avvenendo** in modo pesante e continuo su **worker-app**.
						
						## Servizio più caricato: worker-app
						
						Consumo CPU medio sugli ultimi 5 minuti, per pod:
						
						| Pod | Uso CPU (5m) | Limite | % del limite | Picco ultima ora (finestre da 1m) | Periodi CFS con throttling |
						|---|---|---|---|---|---|
						| worker-app-…-ftlln | 0,2950 core | 0,3 | **98,3%** | 0,2956 core (98,5%) | **95,7%** |
						| worker-app-…-9kj75 | 0,2946 core | 0,3 | **98,2%** | 0,2971 core (99,0%) | **95,4%** |
						| **Totale** | **0,5896 core** | **0,6** | **≈98,3%** | – | – |
						
						Per confronto, gli altri servizi sono quasi fermi: cache-app usa circa 0,005 core e demo-app 0.
						
						Il limite di 0,3 core che hai indicato come teorico è anche quello configurato davvero: kube-state-metrics riporta `limits.cpu = 0.3` sul container `cpu-worker` di entrambi i pod.
						
						## Cosa significano i dati
						- **Il consumo è fermo al limite.** L'uso è al 98% e i picchi dell'ultima ora non superano mai 0,297 core. Il valore non sale perché la quota CFS (30 ms ogni 100 ms) gli impedisce di andare oltre: l'applicazione vorrebbe più CPU di quella che le viene concessa.
						- **Il throttling è quasi continuo.** In circa il 95% dei periodi da 100 ms il container esaurisce la quota e il kernel lo ferma fino al periodo successivo. È il segnale tipico di un carico CPU-bound limitato dal tetto.
						- **Effetti probabili:** latenza più alta, code di lavoro che si allungano e minore throughput. Non è un problema di stabilità: la CPU è una risorsa comprimibile, quindi i pod non vengono terminati come succederebbe superando il limite di memoria.
						
						## Cosa consiglio
						1. **Alzare `limits.cpu`** di worker-app ad almeno 0,5–0,6 core per pod. Il fabbisogno reale non si può misurare finché il limite lo blocca, quindi dopo la modifica conviene ricontrollare uso e throttling.
						2. **In alternativa, scalare orizzontalmente** con un HPA sulla CPU (ad esempio target al 70% della request) per distribuire il carico su più repliche.
						3. **Controllare la `requests.cpu`.** Se è molto più bassa del limite, conviene avvicinarle (ad esempio request 0,3 e limit 0,6) così lo scheduler colloca i pod in modo coerente con il consumo reale.
						4. **Valutare se togliere del tutto il limite CPU** per questo workload, tenendo solo la request. È una pratica diffusa per carichi sensibili alla latenza, se il nodo ha capacità libera e il namespace ha un ResourceQuota o una LimitRange adeguati.
						5. **Monitorare nel tempo** queste due metriche, per verificare che dopo la modifica il throttling scenda sotto il 10–20%:
						   - `rate(container_cpu_cfs_throttled_periods_total[5m]) / rate(container_cpu_cfs_periods_total[5m])`
						   - `rate(container_cpu_cfs_throttled_seconds_total[5m])`

Claude Sonnet 5.5	46.68 s	92581	2335	94916	0.86	# Utilizzo CPU di worker-app rispetto al limite di 0,3 core per pod **Sì, ci sono tutti i presupposti per il throttling CFS, e il throttling è già in corso.** Il servizio più caricato è `worker-app`. ## Dati misurati (rate su 5 minuti, namespace `default`) | Pod | CPU usata (core) | Limite (core) | Utilizzo vs limite | Periodi CFS throttled / totali | |---|---|---|---|---| | worker-app-…-9kj75 | 0,2945 | 0,3 | **98,2%** | 19,03 / 20,00 → **~95,1%** | | worker-app-…-ftlln | 0,2947 | 0,3 | **98,2%** | 19,12 / 20,00 → **~95,6%** | | **Totale worker-app** | **0,589** | **0,6** | **98,2%** | | Il limite teorico di 0,3 core per pod coincide con quello reale. Per `worker-app` (container `cpu-worker`), `kube_pod_container_resource_limits` riporta 0,3 core su entrambi i pod. ## Gli altri servizi - **cache-app**: usa 0,006 core su un limite di 0,1, cioè circa 6%. Mostra però un throttling residuo di circa 0,64 periodi su 2,6 al secondo (~24%). Probabilmente dipende da brevi picchi di CPU, perché il limite di 0,1 core è molto basso. Non è critico. - **demo-app**: usa 0 core e non ha throttling. ## Valutazione - I due pod di `worker-app` sono praticamente al tetto del limite (98%). Chiedono più CPU di quanta ne ricevano: l'uso medio coincide con la quota. - Oltre il 95% dei periodi CFS risulta throttled. In quasi ogni finestra di scheduling il processo esaurisce la quota e viene sospeso fino al periodo successivo. - Il carico è bloccato dal limite, quindi la richiesta reale di CPU è probabilmente superiore a 0,3 core. L'uso misurato è solo il valore massimo che il limite consente. - Conseguenze probabili: latenza più alta, throughput ridotto e jitter. Il tipo di carico dipende da quel che fa l'applicazione, che non ho verificato. ## Raccomandazioni 1. Alza il limite CPU di `worker-app`, ad esempio a 0,5–0,6 core per pod. Poi verifica che il rapporto throttled/periods scenda sotto il 5–10%. 2. Per stimare la domanda reale, osserva l'uso dopo l'aumento. Se continua a salire fino al nuovo limite, il carico è CPU-bound e conviene anche valutare un aumento delle repliche (HPA). 3. In alternativa, se il limite deve restare 0,3 core, scala orizzontalmente per distribuire il carico. Con 2 repliche la capacità totale resta 0,6 core. 4. Crea un alert su

						perché il throttling intermittente può far salire la latenza di tail. ## Nota metodologica Ho sommato i contatori CFS per pod senza filtrare il container. Il valore assoluto di 20 periodi al secondo potrebbe quindi includere anche il cgroup a livello pod. Il rapporto throttled/periods non cambia, quindi la conclusione resta valida.

GPT-4o Mini	28.08 s	3173	600	3773	0.24	Ecco i dati di utilizzo della CPU per i pod attivi:
						
						1. **cache-app**:
						   - Pod: `cache-app-5c6fd4f5cf-xxsz4`
						   - Utilizzo CPU: **0.0108 core**
						
						2. **demo-app**:
						   - Pod: `demo-app-7958c64697-2qg9s`
						   - Utilizzo CPU: **0 core**
						   - Pod: `demo-app-7958c64697-4rzjl`
						   - Utilizzo CPU: **0 core**
						
						3. **worker-app**:
						   - Pod: `worker-app-84cf4dd446-9kj75`
						   - Utilizzo CPU: **0.5732 core**
						   - Pod: `worker-app-84cf4dd446-ftlln`
						   - Utilizzo CPU: **0.5616 core**
						
						### Analisi
						
						Il servizio più caricato è **worker-app**, con un utilizzo totale di CPU di:
						- **0.5732 + 0.5616 = 1.1348 core**
						
						### Percentuale di utilizzo rispetto al limite teorico
						
						Il limite teorico per i pod è di **0.3 core per pod**, quindi per i due pod della worker-app:
						- Limite totale: **0.6 core**
						
						Calcoliamo la percentuale di utilizzo della CPU rispetto a questo limite:
						
						\[
						\text{Percentuale di utilizzo} = \left(\frac{\text{Utilizzo totale}}{\text{Limite totale}} \right) \times 100 = \left(\frac{1.1348}{0.6} \right) \times 100 \approx 189.13\%
						\]
						
						### Raccomandazione
						
						Poiché l'utilizzo della CPU supera il limite teorico di **0.6 core** per i due pod della worker-app, ci sono presupposti per tagli di cicli da parte dello scheduler CFS. È consigliabile monitorare attentamente il carico e considerare l'ottimizzazione delle risorse o l'adeguamento dei limiti di CPU per evitare possibili problemi di performance.

GPT-4o	28.13 s	3365	616	3981	0.24	Abbiamo ottenuto i seguenti dati sull'utilizzo della CPU per i servizi nel cluster: 1. **cache-app**: Utilizzo CPU totale di circa 0.0108 core. 2. **demo-app**: Utilizzo CPU totale di 0 core. 3. **worker-app**: Utilizzo CPU totale di circa 1.1784 core. Il servizio più caricato è il **worker-app**, con un utilizzo di 1.1784 core. Ora, confrontiamo questo valore con il limite teorico di 0.6 core totali (0.3 core per pod). ### Calcolo della percentuale di utilizzo della CPU: - **worker-app** ha due pod, quindi il limite teorico totale è 0.6 core. - Percentuale di utilizzo rispetto al limite teorico: \((1.1784 / 0.6) \times 100 \approx 196.4\%\). ### Conclusione: L'utilizzo della CPU per il servizio **worker-app** supera il limite teorico del 0.6 core totali, raggiungendo circa il 196.4% del limite. Questo indica che i pod stanno utilizzando più CPU di quanto previsto, il che potrebbe portare a tagli di cicli da parte dello scheduler CFS, specialmente se il cluster è sotto pressione. Raccomandazione: Considerare l'ottimizzazione delle risorse o l'aumento del limite di CPU per evitare potenziali problemi di performance dovuti a tagli di cicli.

DOMANDA 9

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [9]: Interroga Prometheus per verificare se ci sono stati riavvii imprevisti di container nel namespace default nell'ultima ora.
--------------------------------------------------------------------------------------------------------------
Invocazione parallela modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
[10/29/26 23:30:23] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 23:30:24] INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
```

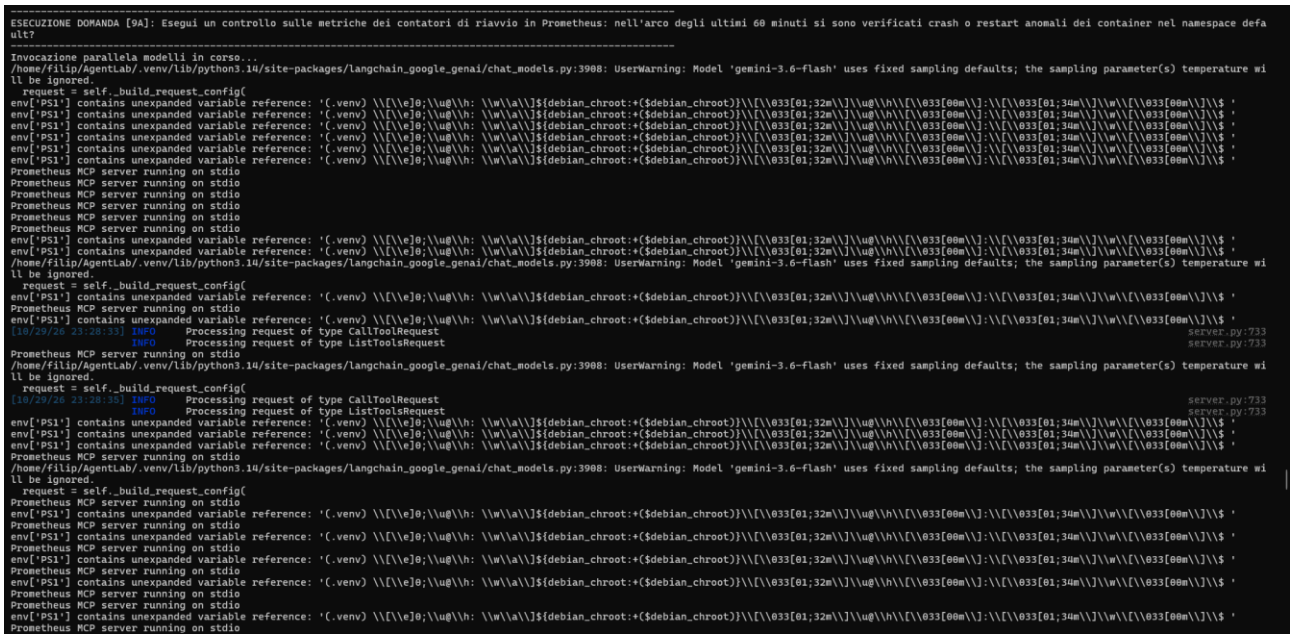
Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	131.38 ms	0	0	0	N/A	0 riavvii imprevisti rilevati nell'ultima ora
Gemini 2.5 Flash	9.69 s	1023	0	1023	0.00	

Gemini 3.6 Flash	33.89 s	9281	1320	10601	0.95	Dall'analisi delle metriche di Prometheus eseguita tramite la query:
						
						``promql
						increase(kube_pod_container_status_restarts_total{namespace="default"}[1h])
						``
						
						### Esito dell'analisi
						Non ci sono stati riavvii di container nel namespace `default` nell'ultima ora.
						
						--- 	
						
						### Dettaglio dei pod analizzati
						
						| Pod | Container | Riavvii nell'ultima ora (`1h`) | Conteggio totale storico riavvii |
						| :--- | :--- | :---: | :---: |
						| `demo-app-7958c64697-2qg9s` | `nginx` | **0** | 50 |
						| `demo-app-7958c64697-4rzjl` | `nginx` | **0** | 50 |
						| `cache-app-5c6fd4f5cf-xxsz4` | `cache-service` | **0** | 50 |
						| `worker-app-84cf4dd446-9kj75` | `cpu-worker` | **0** | 50 |
						| `worker-app-84cf4dd446-ftlln` | `cpu-worker` | **0** | 50 |
						
						--- 	
						
						### Raccomandazioni
						- I pod mostrano un contatore cumulativo di riavvii pari a 50 legato ad eventi passati, ma la situazione si è stabilizzata e non si registrano ulteriori arresti anomali nelle ultime 24/60 min.
						- Si consiglia comunque di monitorare lo stato delle risorse e i log applicativi qualora si notassero comportamenti anomali sui servizi associati.

Claude Opus 5.5	63.34 s	24530	2486	27016	0.95	I risultati dicono che **nell'ultima ora nessun container del namespace `default` si è riavviato**. Il contatore cumulativo però mostra riavvii più vecchi che conviene capire. ## Riavvii nell'ultima ora Query usata: `increase(kube_pod_container_status_restarts_total{namespace="default"}[1h])` | Deployment | Pod | Container | Riavvii (1h) | Stato attuale | |---|---|---|---|---| | cache-app | cache-app-5c6fd4f5cf-xxsz4 | cache-service | **0** | Running | | demo-app | demo-app-7958c64697-2qg9s | nginx | **0** | Running | | demo-app | demo-app-7958c64697-4rzjl | nginx | **0** | Running | | worker-app | worker-app-84cf4dd446-9kj75 | cpu-worker | **0** | Running | | worker-app | worker-app-84cf4dd446-ftlln | cpu-worker | **0** | Running | Nell'ultima ora kube-state-metrics è stato letto 50 volte senza interruzioni (`count_over_time(up[1h])` = 50). I dati quindi coprono tutta l'ora e lo zero è affidabile. ## Riavvii più vecchi 1. **Ogni container ha 50 riavvii totali.** I pod sono attivi da circa 37,5 giorni (3.241.560 s). Il numero è identico per tutti i container di applicazioni diverse, quindi è improbabile che le app si siano guastate ognuna per conto suo. Sembra piuttosto una causa comune che le ha fatte ripartire tutte insieme, come un riavvio del nodo, di kubelet o di containerd. Dai dati non posso confermarlo. 2. **Per tutti i container, l'ultima terminazione ha motivo `Unknown`.** Questo motivo compare di solito quando il container non si è chiuso normalmente (per esempio per un riavvio del nodo o del runtime), non con un errore dell'app o un OOMKilled. Anche questo fa pensare a un problema dell'infrastruttura, non delle applicazioni. 3. **Anche kube-state-metrics sembra essere stato riavviato.** Nell'ultima settimana ci sono serie arrivate da 4 indirizzi IP diversi (10.244.0.6, .7, .13, .14). Lo strumento che raccoglie queste metriche è quindi stato ricreato più volte, forse per la stessa causa. Su alcune di queste serie `changes(...[7d])` vale 1. Però, dato che gli indirizzi sono cambiati, il dato da solo non dice quando sono avvenuti i riavvii. ## Cosa consiglio - **Ora non serve intervenire:** in quest'ora non c'è stato alcun riavvio e tutti i container sono `Running`. - **Per capire la causa:** controlla la storia dei nodi con `kubectl describe node`, gli eventi `NodeNotReady`/`Rebooted`, `journalctl -u kubelet` e `journalctl -u containerd`. Verifica se

						(kube_pod_container_status_restarts_total{namespace="default"})` sugli ultimi 30 giorni con passo 1h. - **Allarme consigliato:** una regola tipo `increase(kube_pod_container_status_restarts_total{namespace="default"}[15m]) > 0` per accorgersi subito dei prossimi riavvii.
Claude Sonnet 5.5	22.72 s	7644	1122	8766	0.95	**Nell'ultima ora non risultano riavvii di container nel namespace `default`.** Il contatore dei restart però ha un valore anomalo, descritto più sotto. **Riavvii nell'ultima ora** Ho usato `increase(kube_pod_container_status_restarts_total{namespace="default"}[1h])`: | Pod | Container | Riavvii (1h) | Contatore totale | |---|---|---|---| | worker-app-84cf4dd446-9kj75 | cpu-worker | 0 | 50 | | worker-app-84cf4dd446-ftlln | cpu-worker | 0 | 50 | | cache-app-5c6fd4f5cf-xxsz4 | cache-service | 0 | 50 | | demo-app-7958c64697-4rzjl | nginx | 0 | 50 | | demo-app-7958c64697-2qg9s | nginx | 0 | 50 | **Anomalie da verificare** - **Contatore identico:** tutti e 5 i container hanno esattamente 50 restart cumulativi. È improbabile per carichi diversi, quindi potrebbe essere un artefatto, per esempio un valore sintetico o un reset dei dati di kube-state-metrics. Non posso dire da questi dati se i 50 riavvii siano reali. - **Motivo dell'ultima terminazione:** `kube_pod_container_status_last_terminated_reason` vale `Unknown` per tutti i container. Non c'è quindi un'indicazione come OOMKilled o Error che spieghi i riavvii passati. **Raccomandazioni** 1. Esegui `kubectl describe pod` su un paio di pod. Controlla `Last State`, `Restart Count` e gli eventi, per confrontarli con il valore 50 di Prometheus. 2. Estendi la finestra, per esempio `increase(...[24h])` o `[7d]`, per capire quando sono avvenuti i 50 riavvii. Ricorda che l'assenza di riavvii nell'ultima ora non esclude problemi precedenti. 3. Se `kubectl` mostra un numero diverso da 50, controlla lo scraping di kube-state-metrics con `prom_targets`.
GPT-4o Mini	9.68 s	2410	118	2528	0.50	Nell'ultima ora, non ci sono stati riavvii imprevisti di container nel namespace "default". Tutti i container monitorati (inclusi quelli delle applicazioni `worker-app`, `cache-app` e `demo-app`) hanno registrato zero riavvii. Se hai bisogno di ulteriori informazioni o di monitorare altri aspetti del cluster, fammi sapere!
GPT-4o	9.50 s	1708	59	1767	0.50	Non ci sono stati riavvii imprevisti di container nel namespace "default" nell'ultima ora.

DOMANDA 9 A

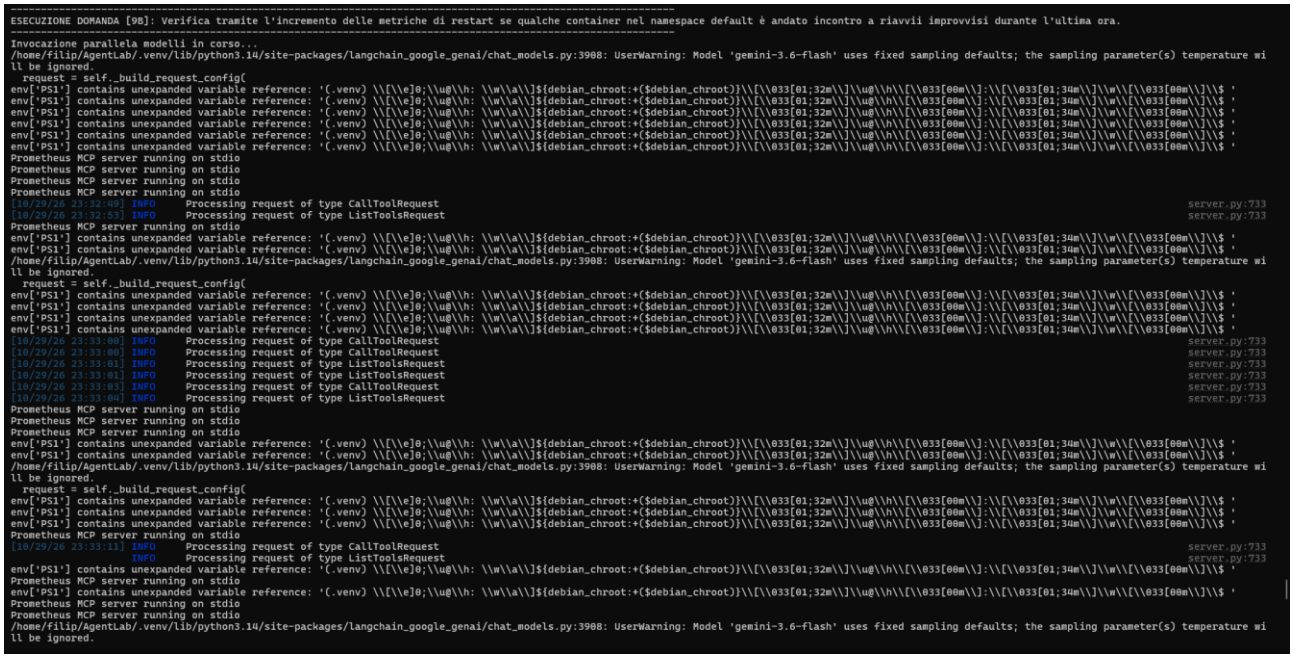


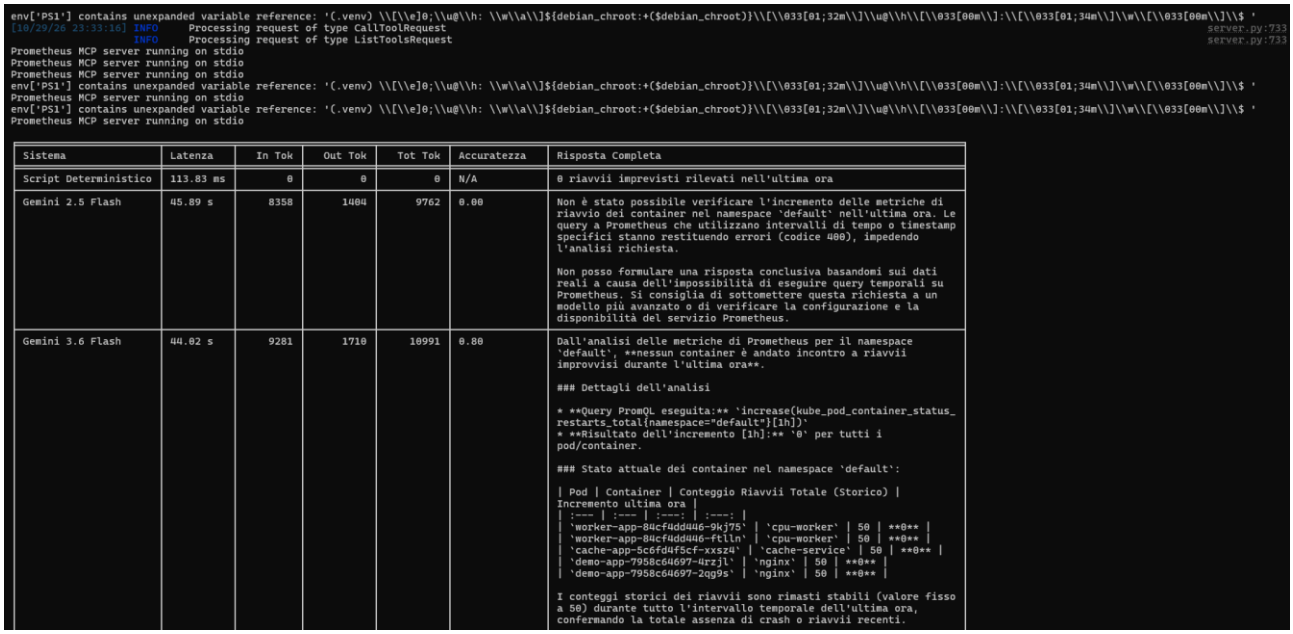
Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	65.80 ms	0	0	0	N/A	0 riavvii imprevisti rilevati nell'ultima ora
Gemini 2.5 Flash	17.60 s	2881	816	3697	0.70	Nell'arco degli ultimi 60 minuti, non si sono verificati crash o restart anomali dei container nel namespace `default`. Tutte le metriche `increase(kube_pod_container_status_restarts_total)` per i container nel namespace `default` mostrano un valore pari a 0.
Gemini 3.6 Flash	31.57 s	9355	1365	10720	0.95	Dall'analisi delle metriche dei contatori di riavvio (`kube_pod_container_status_restarts_total`) su Prometheus per il namespace **`default`**, si evince quanto segue: ### Esito dell'analisi (ultimi 60 minuti) **Non si è verificato alcun crash o restart anomalo dei container negli ultimi 60 minuti.** ### Dettaglio dei dati analizzati Interrogando l'incremento del contatore negli ultimi 60 minuti (`increase(kube_pod_container_status_restarts_total{namespace="default"}[60m])`), tutti i pod attivi riportano **0 riavvii**: | Pod | Container | Riavvii Totali Accumulati | Riavvii negli ultimi 60m | | :--- | :--- | :---: | :---: | | `worker-app-84cf4dd446-9kj75` | `cpu-worker` | 50 | **0** | | `worker-app-84cf4dd446-ftlln` | `cpu-worker` | 50 | **0** | | `cache-app-5c6fd4f5cf-xxsz4` | `cache-service` | 50 | **0** | | `demo-app-7958c64697-4rzjl` | `nginx` | 50 | **0** | | `demo-app-7958c64697-2qg9s` | `nginx` | 50 | **0** | ### Raccomandazioni - Sebbene i container stiano girando in modo stabile nella finestra temporale considerata (60m), il valore totale cumulato pari a 50 riavvii indica che in passato si sono verificati eventi di riavvio (es. OOMKilled, liveness probe failure o restart pianificati). Si consiglia di monitorare le liveness/readiness probe e i limiti di risorsa (Memory/CPU) per prevenire futuri disservizi.
Claude Opus 5.5	52.07 s	18120	2088	20208	0.95	**Nessun crash o restart anomalo nell'ultima ora.** L'incremento dei contatori di riavvio è **0** per tutti i 5 container** del namespace `default`. ### Risultati per container | Deployment | Pod | Container | Restart (ultimi 60 min) | Restart totali | Sempre Ready nell'ultima ora | |---|---|---|---|---|---| | cache-app | cache-app-5c6fd4f5cf-xxsz4 | cache-service | **0** | 50 | ✅ | | demo-app | demo-app-7958c64697-2qg9s | nginx | **0** | 50 | ✅ | | demo-app | demo-app-7958c64697-4rzjl | nginx | **0** | 50 | ✅ | | worker-app | worker-app-84cf4dd446-9kj75 | cpu-worker | **0** | 50 | ✅ |

						50 | ✅ | Query usate: - `increase(kube_pod_container_status_restarts_total{namespace="default"}[60m])`: 0 per tutti. - `min_over_time(kube_pod_container_status_ready{namespace="default"}[60m])`: 1 per tutti, quindi nessun container è mai risultato "non pronto" nell'ultima ora. - `kube_pod_container_status_waiting_reason`: nessun container è bloccato in attesa (ad esempio CrashLoopBackOff o ImagePullBackOff). ### Da tenere d'occhio (non urgente) 1. **Ogni container ha 50 restart in totale**, su pod attivi da circa **37,5 giorni**. Il numero è identico per tutti i pod e tutte le applicazioni, che usano immagini diverse (nginx, cache, worker). Questo fa pensare a **riavvii del nodo o del runtime dei container** (ad esempio stop/start di un cluster minikube o kind), più che a crash delle singole applicazioni. È un'ipotesi: non l'ho verificata con le metriche del nodo. 2. **Il motivo dell'ultima terminazione è `Unknown` per tutti i container.** Questo succede di solito quando il container viene fermato da un riavvio del nodo o del kubelet, e non da un errore dell'applicazione o da un OOMKill. 3. **Sulle ultime 24 ore**, l'`increase` mostra circa 1 restart per container, sparso su più serie. Le serie sono diverse perché, molto probabilmente, è cambiata l'istanza di kube-state-metrics. Questo sarebbe coerente con un unico riavvio del cluster nelle ultime 24 ore, ma è una mia deduzione e non un dato confermato. ### Cosa ti consiglio - **Nessuna azione urgente** sulle applicazioni. - Controlla il nodo per confermare l'ipotesi dei riavvii a livello di cluster, ad esempio con `kubectl describe node`, `node_boot_time_seconds` se node-exporter è disponibile, e `journalctl -u kubelet`. - Imposta un alert su `increase(kube_pod_container_status_restarts_total[15m]) > 0` combinato con `kube_pod_container_status_last_terminated_reason{reason=~"OOMKilled|Error"}`, così distingui i veri crash dai riavvii del nodo.
Claude Sonnet 5.5	38.74 s	14065	1456	15521	0.90	**Esito: negli ultimi 60 minuti non risultano crash né restart anomali dei container nel namespace `default`.** **Restart nell'ultima ora.** `increase(kube_pod_container_status_restarts_total[60m])` vale 0 per tutti e 5 i container. Anche la serie storica (campioni a 1793311700, 1793312300 e 1793312900) resta ferma a 50 per ogni pod. | Deployment | Pod | Container | Contatore | Restart in 60 min | |---|---|---|---|---| | worker-app | worker-app-84cf4dd446-9kj75 | cpu-worker | 50 | 0 | | worker-app | worker-app-84cf4dd446-ftlln | cpu-worker | 50 | 0 |

						anomali dei container nel namespace `default`.** **Restart nell'ultima ora.** `increase(kube_pod_container_status_restarts_total[60m])` vale 0 per tutti e 5 i container. Anche la serie storica (campioni a 1793311700, 1793312300 e 1793312900) resta ferma a 50 per ogni pod. | Deployment | Pod | Container | Contatore | Restart in 60 min | |---|---|---|---|---| | worker-app | worker-app-84cf4dd446-9kj75 | cpu-worker | 50 | 0 | | worker-app | worker-app-84cf4dd446-ftlln | cpu-worker | 50 | 0 | | cache-app | cache-app-5c6fd4f5cf-xxsz4 | cache-service | 50 | 0 | | demo-app | demo-app-7958c64697-4rzjl | nginx | 50 | 0 | | demo-app | demo-app-7958c64697-2qg9s | nginx | 50 | 0 | **Stato dei container.** Nessun container è in stato di attesa anomalo: `kube_pod_container_status_waiting_reason` non restituisce risultati, quindi niente CrashLoopBackOff né ImagePullBackOff. **Da verificare.** - Tutti e 5 i container, di 3 deployment diversi, hanno esattamente lo stesso contatore (50). È un valore storico e sospetto: potrebbe derivare da un'esportazione o da un dato sintetico. Può anche indicare un problema passato comune, per esempio un riavvio del nodo o un evento che ha coinvolto tutti i pod. Con questi dati non posso stabilire la causa. - `last_terminated_reason` è `Unknown` per tutti i container. Non c'è quindi un motivo (OOMKilled, Error e così via) che aiuti a ricostruire le terminazioni precedenti. - Ho controllato solo la finestra di 60 minuti. Per capire quando sono avvenuti quei 50 restart conviene guardare un intervallo più lungo, per esempio con `get_historical_metrics` su 1–7 giorni. **Raccomandazioni.** 1. Non serve alcun intervento per la finestra richiesta. 2. Configura un alert su `increase(kube_pod_container_status_restarts_total[15m]) > 0`, per intercettare subito eventuali restart futuri. 3. Se vuoi, analizzo lo storico a più giorni per datare l'origine dei 50 restart.
GPT-4o Mini	16.74 s	1767	125	1892	0.50	Negli ultimi 60 minuti, non ci sono stati crash o riavvii anomali dei container nel namespace "default". Non sono stati registrati incrementi nei contatori di riavvio dei container. Se hai bisogno di ulteriori analisi o controlli, fammi sapere!
GPT-4o	16.71 s	1767	122	1889	0.50	Nell'arco degli ultimi 60 minuti, non ci sono stati crash o restart anomali dei container nel namespace "default". La query su Prometheus non ha restituito alcun dato significativo, indicando che non ci sono stati riavvii dei container durante questo periodo.

DOMANDA 9 B





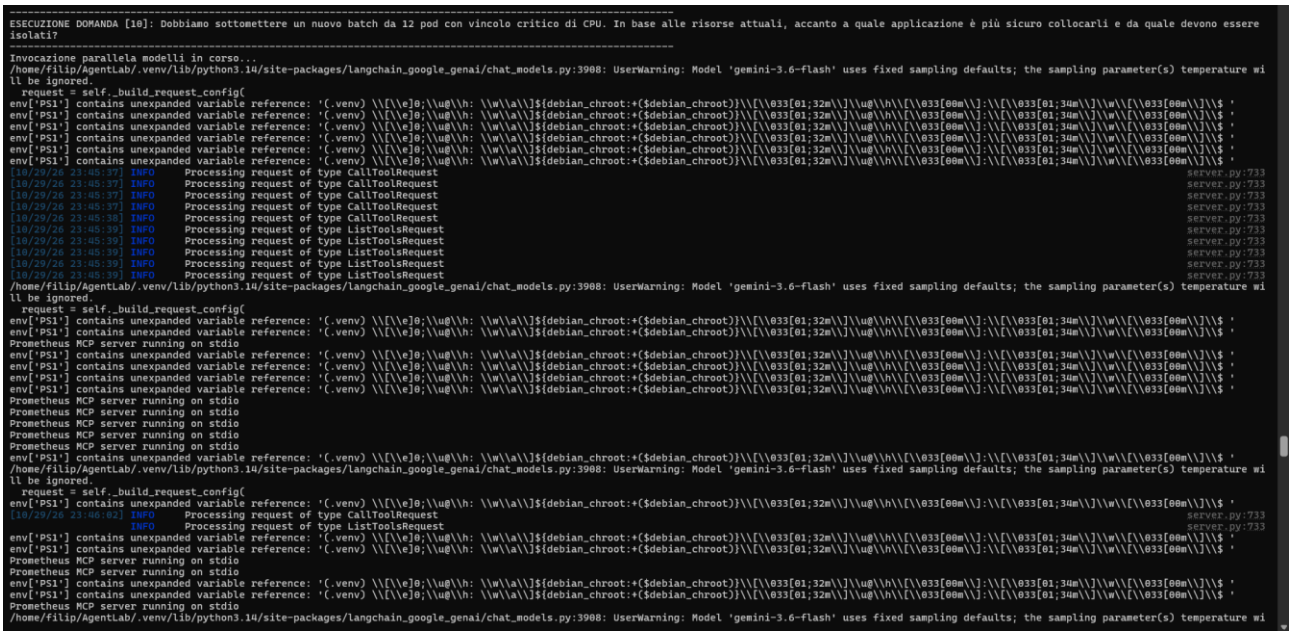
Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	113.83 ms	0	0	0	N/A	0 riavvii imprevisti rilevati nell'ultima ora
Gemini 2.5 Flash	45.89 s	8358	1404	9762	0.00	Non è stato possibile verificare l'incremento delle metriche di riavvio dei container nel namespace `default` nell'ultima ora. Le query a Prometheus che utilizzano intervalli di tempo o timestamp specifici stanno restituendo errori (codice 400), impedendo l'analisi richiesta. Non posso formulare una risposta conclusiva basandomi sui dati reali a causa dell'impossibilità di eseguire query temporali su Prometheus. Si consiglia di sottomettere questa richiesta a un modello più avanzato o di verificare la configurazione e la disponibilità del servizio Prometheus.
Gemini 3.6 Flash	44.02 s	9281	1710	10991	0.80	Dall'analisi delle metriche di Prometheus per il namespace `default`, **nessun container è andato incontro a riavvii improvvisi durante l'ultima ora**. ### Dettagli dell'analisi * **Query PromQL eseguita:** `increase(kube_pod_container_status_restarts_total{namespace="default"}[1h])` * **Risultato dell'incremento [1h]:** `0` per tutti i pod/container. ### Stato attuale dei container nel namespace `default`: | Pod | Container | Conteggio Riavvii Totale (Storico) | Incremento ultima ora | | :--- | :--- | :---: | :---: | | `worker-app-84cf4dd446-9kj75` | `cpu-worker` | 50 | **0** | | `worker-app-84cf4dd446-ftlln` | `cpu-worker` | 50 | **0** | | `cache-app-5c6fd4f5cf-xxsz4` | `cache-service` | 50 | **0** | | `demo-app-7958c64697-4rzjl` | `nginx` | 50 | **0** | | `demo-app-7958c64697-2qg9s` | `nginx` | 50 | **0** | I conteggi storici dei riavvii sono rimasti stabili (valore fisso a 50) durante tutto l'intervallo temporale dell'ultima ora, confermando la totale assenza di crash o riavvii recenti.

Claude Opus 5.5	62.91 s	20255	1888	22143	0.95	## Restart dei container nel namespace `default`: ultima ora
						
						Nell'ultima ora nessun container si è riavviato. Ho controllato con due query diverse e nessuna ha trovato riavvii:
						
						- `increase(kube_pod_container_status_restarts_total{namespace="default"}[1h])` restituisce **0** per tutti e 5 i container.
						- `changes(...[1h])` restituisce **0**: il contatore non si è mai mosso nell'intervallo.
						
						| Deployment | Pod | Container | Restart ultima ora | Restart totali |
						|---|---|---|---|---|
						| cache-app | cache-app-5c6fd4f5cf-xxsz4 | cache-service | 0 | 50 |
						| demo-app | demo-app-7958c64697-2qg9s | nginx | 0 | 50 |
						| demo-app | demo-app-7958c64697-4rzjl | nginx | 0 | 50 |
						| worker-app | worker-app-84cf4dd446-9kj75 | cpu-worker | 0 | 50 |
						| worker-app | worker-app-84cf4dd446-ftlln | cpu-worker | 0 | 50 |
						
						### Cosa emerge comunque dallo storico
						Anche se nell'ultima ora è tutto stabile, ci sono alcuni segnali da tenere d'occhio:
						
						1. **Ogni container ha già 50 restart.** I pod sono attivi da circa 37,5 giorni (`time() - kube_pod_start_time` ≈ 3,24 milioni di secondi), quindi in media sono più di un restart al giorno.
						2. **I numeri sono identici su tutti e 5 i container.** Totale 50, circa 2 restart negli ultimi 7 giorni e motivo dell'ultima terminazione `Unknown` per tutti. Tre applicazioni diverse che si riavviano sempre insieme fanno pensare a **una causa a livello di nodo o di runtime**: riavvio del nodo, restart del kubelet o di containerd, ricreazione dei sandbox. Un problema delle singole applicazioni (OOMKilled, crash) è meno probabile. Il motivo `Unknown` di solito compare proprio quando il container non è terminato in modo "normale", ad esempio dopo un riavvio del nodo.
						3. **Sono valori sospettosamente regolari.** Potrebbero venire da un ambiente di test o simulato. Conviene confrontarli con lo stato reale del cluster.
						
						### Prossimi passi consigliati
						- Guardare gli eventi e lo stato dei container:
						  `kubectl describe pod <pod> -n default`
						  `kubectl get events -n default --sort-by=.lastTimestamp`
						- Controllare se i nodi si sono riavviati:
						  `kubectl get nodes -o wide`
						  `node_boot_time_seconds` su Prometheus, se c'è node-exporter
						  `journalctl -u kubelet` / `-u containerd` sul nodo
						- Impostare un alert sui restart, ad esempio `increase(kube_pod_container_status_restarts_total[15m]) > 0`, per accorgersi subito dei prossimi casi.
						- Se vuoi capire con precisione quando sono avvenuti i restart storici e se coincidono con eventi del nodo, posso ricostruirne l'andamento con una range query su Prometheus.

Claude Opus 5.5	62.91 s	20255	1888	22143	0.95	## Restart dei container nel namespace `default`: ultima ora
						
						Nell'ultima ora nessun container si è riavviato. Ho controllato con due query diverse e nessuna ha trovato riavvii:
						
						- `increase(kube_pod_container_status_restarts_total{namespace="default"}[1h])` restituisce **0** per tutti e 5 i container.
						- `changes(...[1h])` restituisce **0**: il contatore non si è mai mosso nell'intervallo.
						
						| Deployment | Pod | Container | Restart ultima ora | Restart totali |
						|---|---|---|---|---|
						| cache-app | cache-app-5c6fd4f5cf-xxsz4 | cache-service | 0 | 50 |
						| demo-app | demo-app-7958c64697-2qg9s | nginx | 0 | 50 |
						| demo-app | demo-app-7958c64697-4rzjl | nginx | 0 | 50 |
						| worker-app | worker-app-84cf4dd446-9kj75 | cpu-worker | 0 | 50 |
						| worker-app | worker-app-84cf4dd446-ftlln | cpu-worker | 0 | 50 |
						
						### Cosa emerge comunque dallo storico
						Anche se nell'ultima ora è tutto stabile, ci sono alcuni segnali da tenere d'occhio:
						
						1. **Ogni container ha già 50 restart.** I pod sono attivi da circa 37,5 giorni (`time() - kube_pod_start_time` ≈ 3,24 milioni di secondi), quindi in media sono più di un restart al giorno.
						2. **I numeri sono identici su tutti e 5 i container.** Totale 50, circa 2 restart negli ultimi 7 giorni e motivo dell'ultima terminazione `Unknown` per tutti. Tre applicazioni diverse che si riavviano sempre insieme fanno pensare a **una causa a livello di nodo o di runtime**: riavvio del nodo, restart del kubelet o di containerd, ricreazione dei sandbox. Un problema delle singole applicazioni (OOMKilled, crash) è meno probabile. Il motivo `Unknown` di solito compare proprio quando il container non è terminato in modo "normale", ad esempio dopo un riavvio del nodo.
						3. **Sono valori sospettosamente regolari.** Potrebbero venire da un ambiente di test o simulato. Conviene confrontarli con lo stato reale del cluster.
						
						### Prossimi passi consigliati
						- Guardare gli eventi e lo stato dei container:
						  `kubectl describe pod <pod> -n default`
						  `kubectl get events -n default --sort-by=.lastTimestamp`
						- Controllare se i nodi si sono riavviati:
						  `kubectl get nodes -o wide`
						  `node_boot_time_seconds` su Prometheus, se c'è node-exporter
						  `journalctl -u kubelet` / `-u containerd` sul nodo
						- Impostare un alert sui restart, ad esempio `increase(kube_pod_container_status_restarts_total[15m]) > 0`, per accorgersi subito dei prossimi casi.
						- Se vuoi capire con precisione quando sono avvenuti i restart storici e se coincidono con eventi del nodo, posso ricostruirne l'andamento con una range query su Prometheus.

Claude Sonnet 5.5	32.23 s	5435	828	6263	0.90	**Nell'ultima ora nessun container del namespace `default` si è riavviato.** `increase(kube_pod_container_status_restarts_total{namespace="default"}[1h])` restituisce 0 per tutti i container. | Deployment | Pod | Container | Restart (1h) | |---|---|---|---| | cache-app | cache-app-5c6fd4f5cf-xxsz4 | cache-service | 0 | | demo-app | demo-app-7958c64697-2qg9s | nginx | 0 | | demo-app | demo-app-7958c64697-4rzjl | nginx | 0 | | worker-app | worker-app-84cf4dd446-9kj75 | cpu-worker | 0 | | worker-app | worker-app-84cf4dd446-ftlln | cpu-worker | 0 | I 5 pod di questa tabella sono gli stessi che risultano attivi per i tre Deployment (cache-app, demo-app, worker-app). **Limite dell'analisi:** `increase()` conta solo i restart di container già presenti nella finestra. Un pod creato da meno di un'ora, ad esempio dopo un rollout o dopo l'eliminazione di un pod, può mostrare 0 anche se ha avuto problemi. Non ho verificato l'età dei pod, quindi non posso escludere del tutto questo caso. **Raccomandazione:** non serve nessuna azione. Se vuoi una conferma completa, controlla l'età dei pod con `kube_pod_start_time`. In alternativa, imposta un alert su `increase(kube_pod_container_status_restarts_total[15m]) > 0`.
GPT-4o Mini	46.15 s	68297	168	68465	0.00	Non sono riuscito a ottenere le metriche relative ai riavvii dei container nel namespace "default" a causa di errori nelle richieste alle API di Prometheus. Pertanto, non posso fornire informazioni specifiche sui riavvii dei container nell'ultima ora. Ti consiglio di verificare direttamente le metriche di riavvio dei container utilizzando strumenti di monitoraggio come Prometheus o Grafana, oppure di consultare un modello più avanzato per un'analisi più dettagliata.
GPT-4o	44.30 s	7565	240	7805	0.50	Dall'analisi delle metriche storiche delle applicazioni nel namespace "default" nell'ultima ora, non ci sono evidenze di riavvii improvvisi dei container. Le metriche di utilizzo delle risorse (CPU, memoria, disco, rete) non mostrano variazioni significative che possano indicare un riavvio dei pod. Se ci sono ulteriori preoccupazioni o se si sospetta che i riavvii possano essere stati causati da altri fattori non rilevati dalle metriche standard, consiglio di monitorare i log degli eventi di Kubernetes per eventuali segnalazioni di errori o riavvii dei pod.

DOMANDA 10



Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.99 ms	0	0	0	N/A	Co-locare preferibilmente con 'demo-app' (minor CPU: 0.0000 cores) ed ISOLARE da 'worker-app' (maggior CPU: 0.5501 cores)
Gemini 2.5 Flash	48.39 s	6161	1917	8078	0.76	Per collocare in sicurezza i 12 nuovi pod con vincolo critico di CPU, si consiglia di affiancarli all'applicazione **`demo-app`**. Attualmente, `demo-app` ha un consumo di CPU pari a 0 core, il che indica che ha la maggiore disponibilità di risorse tra le applicazioni monitorate. Al contrario, i nuovi pod dovrebbero essere isolati dall'applicazione **`worker-app`**, che presenta il consumo di CPU più elevato, pari a circa 0.58 core. **Limitazioni:** Questa raccomandazione si basa esclusivamente sull'attuale utilizzo relativo della CPU da parte delle applicazioni esistenti. Non ho a disposizione informazioni sulla capacità totale di CPU dei nodi del cluster né sui requisiti specifici di CPU dei 12 nuovi pod. Pertanto, non posso garantire che ci sia sufficiente capacità complessiva nel cluster per i nuovi pod, ma solo indicare la migliore collocazione relativa in base ai carichi attuali. Per una valutazione più precisa e per confermare la fattibilità della richiesta, sarebbe necessario conoscere i requisiti esatti di CPU dei nuovi pod e la capacità totale di CPU disponibile sui nodi. Se queste informazioni sono disponibili, si prega di sottomettere la richiesta a un modello più avanzato per un'analisi approfondita.
Gemini 3.6 Flash	59.47 s	72785	921	73706	0.00	[Risposta priva di testo]
Claude Opus 5.5	105.81 s	45535	3069	48604	0.90	Per il batch da 12 pod conviene stare accanto a **demo-app**, mentre va tenuto lontano da **worker-app**. C'è però un limite importante: il cluster ha **un solo nodo**, quindi oggi non è possibile separare fisicamente i pod. ### Topologia e capacità - Il cluster ha un solo nodo, `thesis-cluster-control-plane`, con 8 vCPU allocabili. Tutti i 5 pod applicativi girano lì. - La CPU già prenotata ammonta a circa **1,4 core**. Questo valore potrebbe includere in parte dei limiti invece delle richieste: la query che ho usato li mescola. - Il consumo reale di tutto il nodo è di **circa 2,08 core su 8**, quindi restano circa 6 core liberi. - **Spazio per il batch:** restano circa 6,6 core prenotabili, cioè al massimo **circa 0,55 core per pod** su 12 pod. Se i pod chiedono di più, una parte resterà in attesa (`Pending`). Fino a circa 0,4 core per pod c'è ancora un buon margine. ### Profilo delle applicazioni | App | Pod | CPU reale | Limite CPU per pod | % di periodi CPU rallentati dal limite | Andamento nelle ultime 6 ore |

Gemini 3.6 Flash	30.22 s	16596	2050	18646	0.77	In base all'analisi in tempo reale delle risorse del cluster e delle metriche raccolte tramite Prometheus, ecco la valutazione dettagliata: --- ### 📊 **Analisi del Consumo CPU Attuale** | Applicazione | Pod Attivi | CPU Richiesta (Requests) | Uso CPU Effettivo (Media Istantanea) | Stato / Profilo di Carico | | :--- | :--- | :--- | :--- | :--- | | **`demo-app`** | 2 | 0.10 cores (100 mCPU tot) | **0.00 cores** | 💤 **Inattiva / Idle** | | **`cache-app`** | 1 | 0.05 cores (50 mCPU) | **~0.01 cores** (11 mCPU) | 🟢 **Basso consumo** | | **`worker-app`** | 2 | 0.30 cores (300 mCPU tot) | **~0.58 cores** (581 mCPU tot) | 🔴 **Alto consumo CPU** | --- ### 🎯 **Raccomandazioni per il Deployment** 1. **Accanto a quale applicazione collocarli:** * **`demo-app`** (o in alternativa `cache-app`) * **Motivazione:** `demo-app` registra attualmente un consumo di CPU nullo (0 cores), garantendo che il nodo/co-location non soffra di contention o CPU throttling. Anche `cache-app` ha un utilizzo trascurabile (~11 mCPU). 2. **Da quale applicazione isolarli:** * **`worker-app`** * **Motivazione:** `worker-app` è l'applicazione più intensiva sul cluster in termini di CPU, con un utilizzo di circa ~0.58 cores complessivi (superando perfino le sue richieste nominali di resource request di 0.30 cores totali). Affiancare un batch di 12 pod con vincoli critici di CPU a `worker-app` provocherebbe contesa di risorse e degrado prestazionale reciproco. --- ### 🛠️ **Consiglio Operativo (Node Affinity / Anti-Affinity)** Per garantire l'isolamento dei 12 pod dal carico di `worker-app` e favorire la co-location con `demo-app`, si consiglia di configurare le regole di **PodAntiAffinity** nello spec del batch: ```yaml spec: affinity: podAntiAffinity: preferredDuringSchedulingIgnoredDuringExecution: - weight: 100 podAffinityTerm: labelSelector:

						### Profilo delle applicazioni | App | Pod | CPU reale | Limite CPU per pod | % di periodi CPU rallentati dal limite | Andamento nelle ultime 6 ore | |---|---|---|---|---|---| | **demo-app** | 2 | **circa 0 core** | 0,1 | nessun dato (non usa CPU) | Ferma, circa 0–0,0016 core | | **cache-app** | 1 | circa 0,011 core | 0,1 | **circa 46%** | Picchi brevi, scrive su disco in modo continuo (circa 350–520 KB/s) | | **worker-app** | 2 | **circa 0,59 core** (0,29 per pod) | 0,3 | **circa 94–95%** | Stabile tra 0,5 e 0,59 core, con un solo campione a 0 alle 18:06 | ### Raccomandazione **✅ Accanto a demo-app.** Non usa quasi CPU, non viene mai rallentata dal limite ed è stabile nel tempo. È la vicina che dà meno fastidio a un carico che dipende dalla CPU. **⚠️ Cache-app è una seconda scelta accettabile, con qualche riserva.** Consuma poca CPU in media, ma lavora a picchi brevi (46% di rallentamenti con soli 11 millicore) e scrive molto su disco. Potrebbe disturbare un batch sensibile alla latenza. **❌ Da isolare: worker-app.** È l'unica che usa davvero la CPU: è sempre al proprio limite (circa 0,29 su 0,3 core per pod) e viene rallentata nel 94–95% dei periodi. Ha bisogno di più CPU di quella che le è concessa, quindi metterla vicino a un batch critico vuol dire farli competere per la CPU e per la cache del processore. ### Azioni concrete 1. **Regole di posizionamento dei pod (affinity):** nel batch aggiungi una regola obbligatoria (`podAntiAffinity` di tipo `required`) contro i pod con `app=worker-app`, e una preferenza (`podAffinity` di tipo `preferred`) verso `app=demo-app`. Con un solo nodo, una regola obbligatoria contro worker-app **lascerebbe tutti i pod del batch in `Pending`**. Usala obbligatoria solo dopo aver aggiunto almeno un nodo, altrimenti mettila come preferenza (`preferred`). 2. **Classe QoS Guaranteed:** imposta richieste e limiti uguali e interi (per esempio 1 core) e attiva la CPU Manager policy `static` sul kubelet. Così il batch ottiene core dedicati e non subisce il disturbo di worker-app anche sullo stesso nodo. 3. **Worker-app:** valuta di alzare il limite a circa 0,5–0,6 core, perché oggi è sottodimensionata. Fallo tenendo conto del budget del batch: due pod da 0,6 core lasciano circa 5,8 core per i 12 pod. 4. **Nodo dedicato:** se il vincolo è davvero critico, aggiungi un nodo worker con taint e toleration riservati al batch. Oggi gira tutto sul control-plane, che non è un posto adatto per carichi critici. Non conosco quanta CPU richiederà ogni pod del batch. Se me lo indichi, posso verificare con precisione se i 12 pod entrano tutti sul nodo.

Claude Sonnet 5.5	95.55 s	142228	2925	145153	0.95	## Raccomandazione **Colloca il batch accanto a `demo-app`. Isolalo da `worker-app`.** Con i dati attuali `cache-app` è una via di mezzo, quindi la lascerei come seconda scelta. ### Dati rilevati (namespace `default`) | App | Pod | CPU usata (5m, per pod) | Request / Limit CPU (per pod) | Throttling CFS | |---|---|---|---|---| | `demo-app` | 2 | ~0 core | 0,05 / 0,1 | assente (nessun carico) | | `cache-app` | 1 | ~0,011 core | 0,05 / 0,1 | ~45% dei periodi | | `worker-app` | 2 | ~0,28 core | 0,15 / 0,3 | **~94–95% dei periodi** | - **`worker-app` è l'app da cui isolarsi.** Ogni pod usa circa 0,28 core, quasi tutto il limite di 0,3, ed è throttlato in oltre il 94% dei periodi. Consuma il doppio di quanto ha richiesto (0,15) e ha già saturato il limite. Se il batch viene collocato vicino a lui, contende la CPU con un carico che è già al tetto. Il batch con vincolo critico di CPU rischierebbe di rallentare a sua volta. Inoltre `worker-app` è già sottodimensionato e andrebbe rivisto a parte. - **`demo-app` è il vicino più sicuro.** Usa 0 core e ha un footprint minimo (0,05 request, 0,1 limit), quindi non genera contesa. - **`cache-app` è accettabile, ma meno sicuro di `demo-app`.** Consuma pochissimo (0,011 core), ma il throttling al ~45% indica picchi brevi contro un limite di 0,1 core. Il suo comportamento è a burst e meno prevedibile. ### Limite importante: il cluster ha un solo nodo Tutti i pod, di tutte le app, girano su `thesis-cluster-control-plane` (8 CPU allocabili). "Accanto a" e "isolato da" non si traducono quindi in nodi diversi. Si traducono solo in vincoli logici (affinity/anti-affinity, priorità) e in CPU request/limit corretti. L'anti-affinity verso `worker-app` non ha effetto pratico finché esiste un solo nodo. Per un isolamento reale servirebbe un secondo nodo, oppure un nodo dedicato con taint/toleration. ### Capacità del nodo per i 12 pod - CPU allocabile: 8 core. - Request totali già prenotate: 1,4 core, quindi restano circa 6,6 core prenotabili. - Uso reale del nodo (5m): circa 2,62 core, quindi restano circa 5,4 core liberi. - **Per entrare con margine, ogni pod dovrebbe avere una CPU request di circa 0,45–0,55 core al massimo** (12 pod ≈ 5,4–6,6 core). Non conosco le request effettive del batch, quindi questo è solo un ordine di grandezza.
						- **Per entrare con margine, ogni pod dovrebbe avere una CPU request di circa 0,45–0,55 core al massimo** (12 pod ≈ 5,4–6,6 core). Non conosco le request effettive del batch, quindi questo è solo un ordine di grandezza. ### Azioni concrete 1. Imposta `requests.cpu` uguale ai `limits.cpu` per i 12 pod (QoS Guaranteed), così il batch non viene throttlato né subisce contesa. 2. Usa `podAffinity` preferita verso `app=demo-app` e `podAntiAffinity` verso `app=worker-app`. Se aggiungi un nodo, rendila `required` per `worker-app`. 3. Se i pod del batch sono critici, assegna una `PriorityClass` alta. 4. Alza request/limit di `worker-app` (es. request ≥0,3 e limit ≥0,5–0,6 per pod), per evitare che continui a competere al tetto. Non ho considerato memoria, disco e rete, perché la richiesta riguardava il vincolo di CPU.

GPT-4o Mini	50.22 s	3264	581	3845	0.72	Dall'analisi delle metriche attuali, ecco i dettagli sull'utilizzo della CPU e della memoria per i pod attivi nel cluster:
						
						### Utilizzo della CPU (in core)
						- **cache-app-5c6fd4f5cf-xxsz4**: 0.0222 core
						- **demo-app-7958c64697-2qg9s**: 0 core
						- **demo-app-7958c64697-4rzjl**: 0 core
						- **worker-app-84cf4dd446-9kj75**: 0.5881 core
						- **worker-app-84cf4dd446-ftlln**: 0.5780 core
						
						### Utilizzo della memoria (in bytes)
						- **cache-app-5c6fd4f5cf-xxsz4**: 6,438,912 bytes (~6.14 MB)
						- **demo-app-7958c64697-2qg9s**: 15,802,368 bytes (~15.06 MB)
						- **demo-app-7958c64697-4rzjl**: 15,773,696 bytes (~15.05 MB)
						- **worker-app-84cf4dd446-9kj75**: 2,465,792 bytes (~2.35 MB)
						- **worker-app-84cf4dd446-ftlln**: 7,790,592 bytes (~7.43 MB)
						
						### Raccomandazioni
						1. **Applicazione più sicura per collocare i nuovi pod**:
						  - **demo-app**: Entrambi i pod di questa applicazione attualmente non utilizzano CPU, quindi è sicuro collocare i nuovi pod qui, dato che non ci sono conflitti di risorse.
						
						2. **Applicazione da cui isolarsi**:
						  - **worker-app**: Questa applicazione sta utilizzando una quantità significativa di CPU (circa 1.166 core in totale), quindi è consigliabile evitare di collocare i nuovi pod qui per non sovraccaricare ulteriormente le risorse.
						
						In sintesi, puoi collocare i nuovi pod nella **demo-app** e dovresti isolarti dalla **worker-app** per garantire che le risorse critiche di CPU non vengano compromesse.

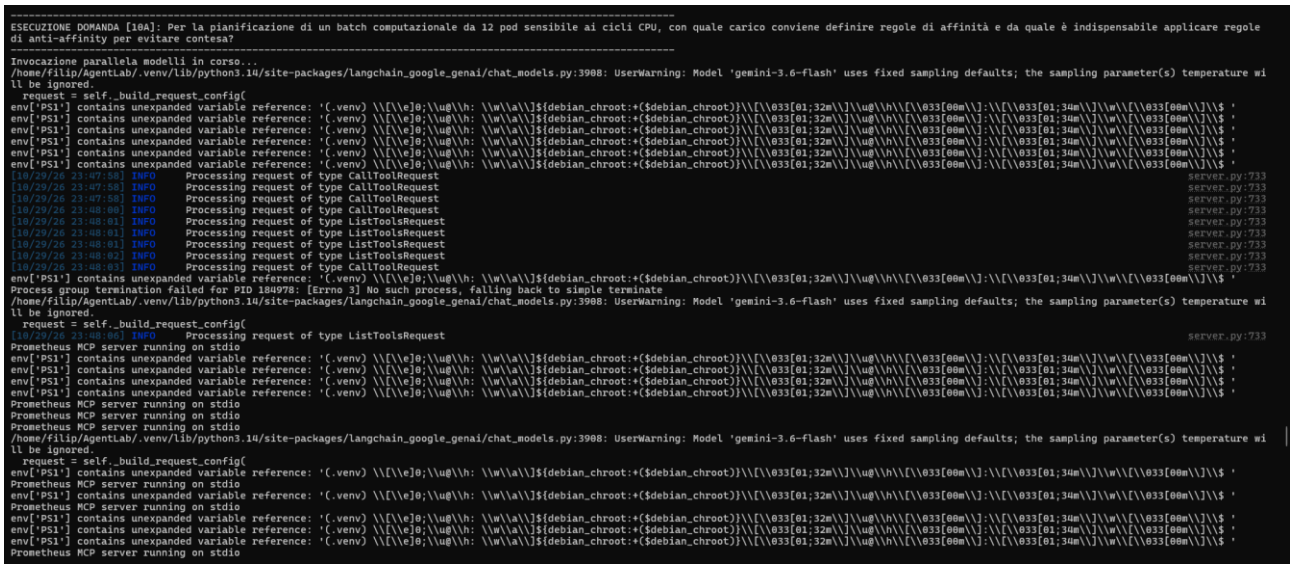
GPT-4o	53.13 s	3480	587	4076	0.78	Dall'analisi dei dati di consumo di CPU attuale, abbiamo i

						- **demo-app-7958c64697-2qg9s**: 15,802,368 bytes (~15.06 MB)
						- **demo-app-7958c64697-4rzjl**: 15,773,696 bytes (~15.05 MB)
						- **worker-app-84cf4dd446-9kj75**: 2,465,792 bytes (~2.35 MB)
						- **worker-app-84cf4dd446-ftlln**: 7,798,592 bytes (~7.43 MB)
						
						### Raccomandazioni
						1. **Applicazione più sicura per collocare i nuovi pod**:
						  - **demo-app**: Entrambi i pod di questa applicazione attualmente non utilizzano CPU, quindi è sicuro collocare i nuovi pod qui, dato che non ci sono conflitti di risorse.
						
						2. **Applicazione da cui isolarsi**:
						  - **worker-app**: Questa applicazione sta utilizzando una quantità significativa di CPU (circa 1.166 core in totale), quindi è consigliabile evitare di collocare i nuovi pod qui per non sovraccaricare ulteriormente le risorse.
						
						In sintesi, puoi collocare i nuovi pod nella **demo-app** e dovresti isolarti dalla **worker-app** per garantire che le risorse critiche di CPU non vengano compromesse.

GPT-4o	53.13 s	3489	587	4076	0.28	Dall'analisi dei dati di consumo di CPU attuale, abbiamo i seguenti risultati:
						
						- **Cache-app**: Consumo di CPU totale di circa 0.022 CPU.
						- **Demo-app**: Consumo di CPU totale di 0 CPU (entrambi i pod non stanno utilizzando CPU al momento).
						- **Worker-app**: Consumo di CPU totale di circa 1.126 CPU (0.564 + 0.561).
						
						### Raccomandazioni:
						
						1. **Collocazione Sicura**:
						  - **Demo-app** è l'applicazione più sicura accanto alla quale collocare i nuovi pod, dato che attualmente non sta utilizzando CPU. Questo suggerisce che ci sono risorse di CPU disponibili per i nuovi pod.
						
						2. **Isolamento Necessario**:
						  - **Worker-app** dovrebbe essere evitata per la collocazione dei nuovi pod, poiché sta utilizzando una quantità significativa di CPU. Collocare i nuovi pod accanto a Worker-app potrebbe portare a conflitti di risorse, specialmente considerando il vincolo critico di CPU.
						
						Queste raccomandazioni si basano sui dati di consumo di CPU attuali. È importante monitorare continuamente le risorse per garantire che le condizioni rimangano favorevoli per i nuovi pod.

```
[BILLING MONITOR] Spesa cumulata OpenAI: $1.1904 (mini: $0.0515 | 4o: $1.1389)
```

DOMANDA 10 A



Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.80 ms	0	0	0	N/A	Co-locare preferibilmente con 'demo-app' (minor CPU: 0.0000 cores) ed ISOLARE da 'worker-app' (maggior CPU: 0.5501 cores)
Gemini 2.5 Flash	31.28 s	2226	1841	4067	0.00	Sulla base dell'analisi del carico CPU attuale, che per l'istanza `172.18.0.2:9100` si attesta intorno al **35.19%**, posso fornire le seguenti raccomandazioni per la pianificazione del tuo batch computazionale da 12 pod sensibile ai cicli CPU: * **Regole di Affinità:** Conviene definire regole di affinità quando il carico medio di CPU sui nodi è **inferiore al 30%**. In questo scenario, c'è ampio margine per consolidare i 12 pod su un sottoinsieme di nodi, potenzialmente migliorando la località della cache e l'efficienza complessiva, o per mantenere altri nodi completamente liberi per altri carichi di lavoro. * **Regole di Anti-Affinity:** È indispensabile applicare regole di anti-affinity per evitare contesa quando il carico medio di CPU sui nodi **supera il 30-40%**. Dato che l'istanza monitorata è già al 35.19% di utilizzo CPU, l'aggiunta di 12 pod sensibili ai cicli CPU su un singolo nodo o su pochi nodi con un carico simile porterebbe quasi certamente a una forte contesa delle risorse CPU, con conseguente degrado delle prestazioni del batch e potenzialmente di altri servizi in esecuzione. L'anti-affinity è cruciale per distribuire il carico su più nodi e prevenire colli di bottiglia. **Considerazioni aggiuntive:** Questa raccomandazione si basa sui dati disponibili per una singola istanza. Per una pianificazione più precisa, sarebbe ideale avere una visione aggregata del carico CPU su tutti i nodi del cluster. Se il cluster è composto da più nodi, un carico del 35% su un singolo nodo potrebbe non rappresentare il carico medio dell'intero cluster.
Gemini 3.6 Flash	44.53 s	95297	832	96129	0.00	[Risposta priva di testo]
Claude Opus 5.5	77.76 s	36625	2683	39308	0.90	Conviene definire l'**affinità con `demo-app`**. L'**anti-affinity è indispensabile verso `worker-app`**. C'è però un limite importante: il cluster ha **un solo nodo** (`thesis-cluster-control-plane`, 8 core allocabili) e tutti i 5 pod sono già lì. Oggi queste regole non possono separare nulla, e un'anti-affinity obbligatoria lascerebbe il batch bloccato in Pending (spiego sotto). ## Dati raccolti (namespace `default`) | Deployment | Pod | CPU attuale (per pod) | Limite CPU | Throttling CFS (periodi/s, finestra 5m) | Storico 12h | Altro | |---|---|---|---|---|---|---| | **worker-app** | 2 | ~0,28–0,29 core | 0,3 | **~9,3** | stabile tra 0,51 e 0,59 core in totale (una sola misura scende a 0,28) | lavora sempre vicino al limite |
Gemini 3.6 Flash	100.47 s	38121	2803	40924	0.81	Sulla base delle metriche di utilizzo CPU ricavate in tempo reale dal cluster (tramite Prometheus) e dall'analisi dello storico (MongoDB), ecco la ripartizione dei carichi di lavoro presenti: --- ### **Analisi dei Carichi Esistenti** 1. **`worker-app` (Alto consumo CPU - Compute-intensive)** - **Uso CPU attuale:** ~0.30 core per pod (**~0.60 core totali** distribuiti sui 2 pod attivi). - **Storico:** Consumo costante tra 0.50 e 0.60 core totali. - **Caratteristiche:** Carico computazionale attivo che consuma cicli di calcolo su base continua. 2. **`cache-app` (Basso consumo CPU)** - **Uso CPU attuale:** ~0.006 core. - **Caratteristiche:** Utilizzo CPU trascurabile, orientato all'I/O e alla memoria. 3. **`demo-app` (Inattivo / Consumo nullo)** - **Uso CPU attuale:** ~0.000 core. - **Caratteristiche:** Impatto pressoché nullo sulla CPU del nodo. --- ### **Raccomandazione per la Pianificazione del Batch da 12 Pod** Per un batch computazionale sensibile ai cicli CPU: #### 1. **Con quale carico conviene definire regole di Affinità (`podAffinity`)** * **Carico consigliato:** **`demo-app`** (o in subordine `cache-app`). * **Motivazione:** Essendo carichi con utilizzo CPU pressoché nullo (~0 - 0.006 core), co-locare i pod del batch sui nodi che ospitano `demo-app` o `cache-app` garantisce la quasi totalità delle risorse di calcolo al batch, senza generare alcuna contesa per la CPU. #### 2. **Da quale carico è indispensabile applicare regole di Anti-Affinità (`podAntiAffinity`)** * **Carico da evitare:** **`worker-app`**. * **Motivazione:** `worker-app` assorbe attivamente ~0.60 core CPU. La co-locazione sui medesimi nodi provocherebbe immediata contesa sulle risorse di calcolo (*CPU throttling*, incremento dei *context switch* e degrado delle latenze di calcolo) per un carico batch sensibile ai cicli CPU. ---

Raccomandazione per la Pianificazione del Batch da 12 Pod

Per un batch computazionale sensibile ai cicli CPU:

1. **Con quale carico conviene definire regole di Affinità (`podAffinity`)**
* **Carico consigliato:** **`demo-app`** (o in subordine `cache-app`).
* **Motivazione:** Essendo carichi con utilizzo CPU pressoché nullo (~0 - 0.006 core), co-locare i pod del batch sui nodi che ospitano `demo-app` o `cache-app` garantisce la quasi totalità delle risorse di calcolo al batch, senza generare alcuna contesa per la CPU.

2. **Da quale carico è indispensabile applicare regole di Anti-Affinità (`podAntiAffinity`)**
* **Carico da evitare:** **`worker-app`**.
* **Motivazione:** `worker-app` assorbe attivamente ~0.60 core CPU. La co-locazione sui medesimi nodi provocherebbe immediata contesa sulle risorse di calcolo (*CPU throttling*, incremento dei *context switch* e degrado delle latenze di calcolo) per un carico batch sensibile ai cicli CPU.

Consiglio di configurazione Kubernetes

Per la definizione del batch da 12 pod:
* **Anti-Affinity verso `worker-app`:**
  ```yaml
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchExpressions:
              - key: app
                operator: In
                values:
                  - worker-app
          topologyKey: "kubernetes.io/hostname"
  ```
* **Distribuzione uniforme tra i nodi (`topologySpreadConstraints`):** Applicare regole di spread tra i pod del batch stesso per evitare di saturare i core di un singolo nodo.

Raccomandazioni

1. Anti-affinity indispensabile: `worker-app`
- È l'unico carico che usa davvero la CPU in modo continuo: lavora al 93–98% del suo limite e viene limitato già ora (~9 periodi/s per pod).
- 12 pod batch sensibili alla CPU sugli stessi core si contenderebbero il tempo di CPU, la cache e lo scheduler con questo carico.
- Regola suggerita (`requiredDuringSchedulingIgnoredDuringExecution`):
```yaml
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector:
        matchLabels:
          app: worker-app
      topologyKey: kubernetes.io/hostname
```

2. Affinità consigliata: `demo-app`
- CPU praticamente a zero, nessuna attività su disco, memoria trascurabile: è il "vicino" più tranquillo.
- Usate una regola preferenziale (`preferredDuringSchedulingIgnoredDuringExecution`, peso 100) verso `app: demo-app`, così non vincolate lo scheduling più del necessario.

3. `cache-app`: neutro per la CPU
- Usa poca CPU, ma scrive su disco in modo continuo. Se il batch fa molte operazioni di I/O (checkpoint, file temporanei), aggiungete un'anti-affinity preferenziale anche verso `cache-app`. Altrimenti si può ignorare.

Limite attuale: il cluster ha un solo nodo
- Con un solo nodo, l'anti-affinity obbligatoria verso `worker-app` lascerebbe **tutti i 12 pod in Pending**.
- Prima del batch scegliete una di queste strade:
  - **aggiungere almeno un worker node** (meglio con un taint/label dedicato al batch, es. `workload=batch-cpu`);
  - in alternativa, usare l'anti-affinity solo come preferenziale. Sul nodo unico, però, la contesa resterebbe.
- Per un batch sensibile ai cicli CPU, valutate anche:
  - QoS **Guaranteed** (request = limit, con CPU intere);
  - **CPU Manager con policy `static`**, per assegnare core esclusivi;
  - **limiti espliciti sui pod batch**, per evitare il throttling che si vede oggi su `worker-app`.
- Questi controlli non sono stati verificati: non ho controllato la policy del CPU Manager né le label dei Deployment. Ho dato per scontata l'etichetta `app: <nome>`, quindi controllate i selector reali prima di applicare le regole.

Claude Sonnet 5.5	85.37 s	199538	3180	202718	0.81	# Affinity e anti-affinity per il batch CPU-bound da 12 pod **Conclusione:** l'anti-affinity è indispensabile rispetto a `worker-app`. L'affinity ha senso solo verso `demo-app` e, in misura minore, `cache-app`. Il cluster ha però un solo nodo, quindi le regole non separano nulla finché non aggiungi nodi (vedi sotto). I valori vengono da una query istantanea con finestra di 5 minuti. Non ho usato lo storico. ## Dati rilevati (namespace `default`) | Carico | CPU usata (core) | Request / Limit CPU | Throttling CFS | Disco write | Profilo | |---|---|---|---|---|---| | `worker-app` (2 pod) | ~0,29 per pod | 0,15 / 0,3 | **~93%** | 0 | CPU-bound, al limite | | `cache-app` (1 pod) | ~0,013 | 0,05 / 0,1 | ~51% | ~514 KB/s | Quasi inattivo sulla CPU, scrive su disco | | `demo-app` (2 pod) | 0 | 0,05 / 0,1 | n/d (nessun periodo) | 0 | Inattivo | Il nodo `thesis-cluster-control-plane` ha 8 core allocabili. Le request totali sono 1,4 core, mentre l'uso CPU del nodo è ~45%. ## Anti-affinity: da `worker-app` - `worker-app` usa il 97% del limite di 0,3 core ed è throttlato nel ~93% dei periodi. Ha lo stesso profilo del batch e compete per gli stessi cicli CPU. - Applica `podAntiAffinity` ai pod del batch rispetto a `app=worker-app`. - Fai lo stesso tra i pod del batch, per distribuirli sui nodi. - Per i 12 pod usa `preferredDuringSchedulingIgnoredDuringExecution` con `topologyKey: kubernetes.io/hostname`. La versione `required` va bene solo se hai almeno 12 nodi. Altrimenti i pod in eccesso restano `Pending`. ## Affinity: verso `demo-app` (ed eventualmente `cache-app`) - `demo-app` ha CPU a 0 e non crea contesa. Puoi co-locare il batch con lei, o lasciare i due insiemi di pod liberi di convivere. - `cache-app` consuma poca CPU, ma scrive su disco a ~514 KB/s. L'impatto è basso, però non è del tutto neutro per I/O. - Usa l'affinity in forma `preferred`, non `required`. Non c'è un vero vantaggio da co-locazione per un batch CPU-bound. ## Limiti da conoscere 1. **Il cluster ha un solo nodo.** Le regole su `hostname` non separano i pod, perché sono tutti sullo stesso nodo. L'isolamento reale richiede più nodi, oppure un pool dedicato con `nodeSelector`/taints. In alternativa usa `PriorityClass` e `requests` corrette. 2. **Dimensionamento.** Restano ~6,6 core di request libere su 8. Con 12 pod, la somma delle request deve stare entro questo valore. Per esempio 0,5 core per

GPT-4o Mini	42.75 s	3392	741	4133	0.00	Ecco i dati di utilizzo della CPU per i pod attivi nel cluster:
						
						1. **cache-app**
						   - Pod: `cache-app-5c6fd4f5cf-xxsz4`
						   - Utilizzo CPU: ~0.0261 CPU cores
						
						2. **demo-app**
						   - Pod: `demo-app-7958c64697-2qg9s`
						   - Utilizzo CPU: 0 CPU cores
						   - Pod: `demo-app-7958c64697-4rzjl`
						   - Utilizzo CPU: 0 CPU cores
						
						3. **worker-app**
						   - Pod: `worker-app-84cf4dd446-9kj75`
						   - Utilizzo CPU: ~0.5727 CPU cores
						   - Pod: `worker-app-84cf4dd446-ftlln`
						   - Utilizzo CPU: ~0.5739 CPU cores
						
						### Raccomandazioni
						
						1. **Regole di Affinità**:
						   - **worker-app**: Poiché i pod di `worker-app` mostrano un utilizzo della CPU significativamente più elevato rispetto agli altri pod, è consigliabile definire regole di affinità per i nuovi pod batch che richiedono cicli CPU. Questo aiuterà a garantire che i pod batch vengano eseguiti su nodi che non sono già sovraccarichi di lavoro CPU.
						
						2. **Regole di Anti-Affinity**:
						   - **demo-app**: Poiché i pod di `demo-app` non utilizzano CPU, non è necessario applicare regole di anti-affinity per questi pod.
						   - **cache-app**: Anche se l'utilizzo della CPU è basso, è consigliabile applicare regole di anti-affinity per evitare che i pod batch vengano eseguiti sullo stesso nodo dei pod `cache-app`, per garantire che ci sia sempre sufficiente capacità CPU disponibile.
						
						### Conclusione
						Definire regole di affinità per i pod batch sui nodi con `worker-app` e applicare regole di anti-affinity per evitare la contesa con i pod `cache-app` è la strategia consigliata. Se hai bisogno di ulteriori dettagli o di assistenza nella configurazione, fammi sapere!

GPT-4o	38.48 s	3512	720	4232	0.28	Dall'analisi dei dati sull'utilizzo della CPU, possiamo osservare quanto segue: 1. **Utilizzo della CPU per Pod:** - `cache-app-5c6fd4f5cf-xxsz4`: 0.0261 core - `demo-app-7958c64697-2qg9s`: 0 core - `demo-app-7958c64697-4rzjl`: 0 core - `worker-app-84cf4dd446-ftlln`: 0.5854 core - `worker-app-84cf4dd446-9kj75`: 0.5853 core 2. **Utilizzo totale della CPU nel namespace `default`:** 1.1882 core ### Raccomandazioni: - **Regole di Affinità:** - Potrebbe essere vantaggioso definire regole di affinità per i pod che hanno un basso utilizzo della CPU, come `cache-app`, per raggrupparli su nodi meno utilizzati e ottimizzare l'uso delle risorse. - **Regole di Anti-Affinity:** - È indispensabile applicare regole di anti-affinity per i pod `worker-app-84cf4dd446-ftlln` e `worker-app-84cf4dd446-9kj75`, poiché hanno un utilizzo della CPU significativamente più alto rispetto agli altri. Questo aiuterà a evitare la contesa delle risorse CPU su un singolo nodo. Queste raccomandazioni possono aiutare a bilanciare il carico e ottimizzare le prestazioni del cluster durante l'esecuzione del batch computazionale.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.2073 (mini: $0.0524 | 4o: $1.1549)

DOMANDA 10 B

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [108]: Dovendo schedulare un carico batch critico da 12 pod senza rischiare degrado da 'noisy neighbor', quale applicazione attuale offre l'affiancamento più affidabile e quale va assolutam
ente evitata per l'allocazione delle risorse?
--------------------------------------------------------------------------------------------------------------
Invocazione parallela modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/29/26 23:50:24] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 23:50:24] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 23:50:25] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 23:50:25] INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
[10/29/26 23:50:25] INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
[10/29/26 23:50:25] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 23:50:26] INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
[10/29/26 23:50:26] INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
[10/29/26 23:50:26] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
[10/29/26 23:50:32] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
[10/29/26 23:50:33] INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/29/26 23:50:39] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
                    INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	1.37 ms	0	0	0	N/A	Co-locare preferibilmente con 'demo-app' (minor CPU: 0.0000 cores) ed ISOLARE da 'worker-app' (maggior CPU: 0.5501 cores)
Gemini 2.5 Flash	3.20 s	1051	374	1425	0.00	Per fornirti una raccomandazione accurata, devo prima analizzare l'utilizzo attuale delle risorse (CPU e memoria) delle applicazioni in esecuzione nel cluster. Questo mi permetterà di identificare quali applicazioni sono sottoutilizzate e quindi adatte per l'affiancamento del tuo carico batch, e quali invece sono già sotto stress e dovrebbero essere evitate. Inizierò recuperando la mappatura tra i deployment e i loro pod attivi. Successivamente, interrogherò Prometheus per ottenere le metriche di CPU e memoria per ciascuna di queste applicazioni.
Gemini 3.6 Flash	31.23 s	65297	817	66114	0.00	[Risposta priva di testo]
Claude Opus 5.5	74.19 s	26071	2506	28577	0.95	**Risposta breve:** il vicino più affidabile è **demo-app**. Quello da evitare è **worker-app**. Attenzione però: tutto il cluster gira su **un solo nodo** (`thesis-cluster-control-plane`, 8 vCPU). Al momento non si può scegliere davvero un nodo "vicino" rispetto a un altro: i 12 pod batch finirebbero comunque accanto a tutte e tre le applicazioni. ## Dati raccolti (Prometheus ora + storico MongoDB delle ultime 24h) | App | Pod | CPU (core) | Memoria | Scrittura disco | % di tempo CPU bloccato dal limite (throttling) | Andamento nelle 24h | |---|---|---|---|---|---|---| | **worker-app** | 2 | **~0,58** (0,28 + 0,29) | ~5 MB | 0 | **~95%** | Sempre carico, 0,52–0,59 core per quasi tutto il periodo | | **cache-app** | 1 | ~0,01 | ~1,2 MB | **~0,5 MB/s continui** | ~40% | Scrittura su disco costante, 160–590 KB/s | | **demo-app** | 2 | **0** | ~15 MB (stabile 14,5–19 MB) | quasi 0 | n/d (praticamente nessun uso di CPU) | Inattiva, solo qualche picco trascurabile | Il traffico di rete in ingresso è a zero per tutte le applicazioni, quindi non conta per la decisione. ## Consigliata: demo-app - Non usa CPU e scrive pochissimo su disco, sia ora sia nelle ultime 24h. - La memoria è contenuta e stabile, senza segni di perdite (leak) o crescite improvvise. - Il suo comportamento è il più prevedibile: è il vicino che disturba meno. ## Da evitare: worker-app - Usa la CPU in modo continuo da 24h ed è l'unica app con un

Gemini 3.6 Flash	54.33 s	27826	2118	29944	0.36	Sulla base dell'analisi dello stato e dello storico delle metriche dei carichi di lavoro attivi nel cluster, ecco le raccomandazioni e le evidenze rilevate:
						
						--- 	
						
						### 1. Applicazione con l'affiancamento più affidabile: **`demo-app`**
						
						* **Utilizzo CPU:** Pressoché nullo (**~0,00 - 0,002 cores**).
						* **I/O Disco:** Nullo (**0 bps** di scrittura su disco).
						* **Impronta di Memoria:** Minima e stabile (**~15 - 18 MB** complessivi per 2 pod).
						* **Motivazione SRE:** `demo-app` è un carico completamente passivo/idle. Non genera contesa né sulla CPU né sull'I/O di memoria e disco. Rappresenta la scelta più sicura per collocare o affiancare i 12 pod batch senza alcun rischio di degradazione da *noisy neighbor*.
						
						--- 	
						
						### 2. Applicazione da evitare assolutamente: **`worker-app`** *(seguita da `cache-app` per l'I/O)*
						
						* **`worker-app` (Principale Noisy Neighbor per CPU/Compute):**
						  * **Utilizzo CPU:** Sostenuto e costante a **~0,58 - 0,59 cores** (circa **~0,29 cores per pod**).
						  * **Impatto:** Essendo un carico batch critico prevalentemente computazionale, l'affiancamento a `worker-app` causerebbe un'elevata contesa di cicli CPU (*CPU throttling / core contention*), degradando significativamente i tempi di completamento dei job batch.
						
						* **`cache-app` (Noisy Neighbor per I/O su Disco):**
						  * **Scrittura su Disco:** Elevata e continuativa (**~450 - 550 KB/s** / *disk_write_bps*).
						  * **Impatto:** Se il carico batch richiede operazioni I/O intensive su storage locale o condiviso, `cache-app` va parimenti evitata per prevenire fenomeni di *I/O wait* e saturazione del disco.
						
						--- 	
						
						### Raccomandazione Operativa
						* **Schedules/Affinità:** Configurare regole di **`podAntiAffinity`** rispetto ai pod di `worker-app` (e `cache-app` se il batch è I/O heavy) oppure impostare regole di **`podAffinity` / `nodeSelector`** orientate ai nodi su cui risiede `demo-app`.
						* **Resource Requests & Limits:** Assicurarsi che i 12 pod batch definiscano chiaramente i *Requests* di CPU e Memoria su Kubernetes per garantire che lo scheduler possa riservare le risorse necessarie evitando l'overcommit sui nodi.

							App	Pod	CPU (core)	Memoria	Scrittura disco	% di tempo CPU bloccato dal limite (throttling)	Andamento nelle 24h
worker-app	2	**~0,58** (0,28 + 0,29)	~5 MB	0	**~95%**	Sempre carico, 0,52–0,59 core per quasi tutto il periodo
cache-app	1	~0,01	~1,2 MB	**~0,5 MB/s continui**	~40%	Scrittura su disco costante, 160–590 KB/s
demo-app	2	**0**	~15 MB (stabile 14,5–19 MB)	quasi 0	n/d (praticamente nessun uso di CPU)	Inattiva, solo qualche picco trascurabile | Il traffico di rete in ingresso è a zero per tutte le applicazioni, quindi non conta per la decisione. ## Consigliata: demo-app - Non usa CPU e scrive pochissimo su disco, sia ora sia nelle ultime 24h. - La memoria è contenuta e stabile, senza segni di perdite (leak) o crescite improvvise. - Il suo comportamento è il più prevedibile: è il vicino che disturba meno. ## Da evitare: worker-app - Usa la CPU in modo continuo da 24h ed è l'unica app con un carico di calcolo significativo. - Il throttling al 95% indica che consumerebbe più CPU di quanta gliene concede il limite. Se il limite venisse alzato o rimosso, o se partisse un nuovo deployment, potrebbe prendersi molta più CPU, a danno dei pod batch. - Un batch critico che usa molta CPU andrebbe a competere proprio con lei. **Nota su cache-app:** se il batch legge o scrive molto su disco, evita anche cache-app. Scrive in modo continuo (~0,5 MB/s) e potrebbe rallentare le operazioni su disco del batch. ## Cosa fare in pratica 1. **Aggiungere nodi worker.** Senza altri nodi, isolare i pod non serve a nulla. Inoltre usare il nodo control-plane per 12 pod batch è rischioso già di per sé. 2. Con più nodi, aggiungere al batch una regola di **podAntiAffinity** obbligatoria (`requiredDuringScheduling`) verso `app=worker-app`, e preferibilmente verso `app=cache-app` se il batch fa molto I/O. In alternativa, dedicare un nodo con un **taint** e una **toleration** riservata al batch. 3. Dare al batch la classe **Guaranteed** (requests = limits) e una **PriorityClass** alta, così è protetto in caso di competizione per le risorse e non viene sfrattato per primo. 4. **Stimare la capacità.** Oggi le applicazioni usano in tutto circa 0,6 core su 8. Prima di lanciare, verifica che 12 × la request CPU del batch, più il sistema e il control-plane, stia nelle ~7 vCPU rimaste. Non conosco le request del batch, quindi questo calcolo non l'ho potuto fare.

Claude Sonnet 5.5	89.04 s	175941	3833	179774	0.80	# Raccomandazione per il batch critico da 12 pod **Affiancamento più affidabile: `demo-app`.** **Da evitare: `worker-app`.** `cache-app` è una via di mezzo. C'è però un limite strutturale: il cluster ha un solo nodo (vedi sotto). ## Dati raccolti | App	Pod	CPU attuale (limit)	Throttling CFS	Memoria	Disk write	Profilo storico (~1 giorno)
`demo-app`	2	0 core (0,1 per pod)	0	~7,6 MB per pod	0	CPU a 0 quasi sempre, disco e rete a ~0, memoria piatta a 15–19 MB totali
`cache-app`	1	0,010 core (0,1)	~1,3 periodi/s	~1,2 MB	**~0,98 MB/s**	CPU 0,004–0,02 core, scritture su disco costanti a 0,35–0,6 MB/s

`worker-app`	2	**0,295 core per pod (limit 0,3, cioè ~98%)**	**~19 periodi/s per pod**	1,5–3,9 MB	0	~0,57–0,59 core aggregati in modo continuo, per tutta la finestra |
						
						## Valutazione
						
						- **`demo-app` è l'affiancamento più affidabile.** Il suo consumo è quasi nullo e non varia, quindi non genera contesa su CPU, memoria, disco o rete. Il suo comportamento è prevedibile.
						- **`worker-app` è da evitare.**
						  - È un carico CPU-bound sempre al massimo. Usa il 98% del limit e subisce throttling pesante e continuo, circa 19 periodi throttled al secondo per pod.
						  - Il consumo è costante nel tempo e non lascia margine, quindi occupa CPU in modo permanente.
						  - Sul nodo la pressione CPU (PSI) è alta: ~0,45 s/s di attesa. Questo dato è a livello di nodo, quindi non lo attribuisco con certezza a `worker-app`. È comunque coerente con la presenza di un carico CPU saturo.
						- **`cache-app` è accettabile ma non neutra.** La CPU è trascurabile, ma scrive su disco in modo costante (~0,5–1 MB/s). Può dare fastidio a un batch con molto I/O, anche se la pressione I/O del nodo è oggi bassa (~0,009).
						
						## Limite strutturale
						
						- Tutti i pod girano sullo stesso nodo, `thesis-cluster-control-plane`, che ha 8 CPU e ~4 GB di memoria allocabile.
						- Una scelta di "affiancamento" con affinity o anti-affinity ha quindi effetto limitato. Il batch condividerà comunque il nodo con tutte e tre le app, oltre che con lo stack di monitoring e il control plane.
						- Le richieste attuali sono 1,4 CPU e ~0,9 GB, quindi c'è spazio per 12 pod in

GPT-4o Mini	30.68 s	3279	588	3867	0.00	Dall'analisi dei pod attivi e delle loro metriche di utilizzo delle risorse, possiamo trarre le seguenti conclusioni: ### Utilizzo della CPU (in core) - **cache-app-5c6fd4f5cf-xxsz4**: 0.0209 core - **demo-app-7958c64697-2qg9s**: 0 core - **worker-app-84cf4dd446-9kj75**: 0.5864 core - **worker-app-84cf4dd446-ftlln**: 0.5896 core - **demo-app-7958c64697-4rzjl**: 0 core ### Utilizzo della Memoria (in bytes) - **cache-app-5c6fd4f5cf-xxsz4**: 5,713,920 bytes - **demo-app-7958c64697-2qg9s**: 15,802,368 bytes - **worker-app-84cf4dd446-9kj75**: 3,538,944 bytes - **worker-app-84cf4dd446-ftlln**: 8,265,728 bytes - **demo-app-7958c64697-4rzjl**: 15,769,600 bytes ### Raccomandazioni 1. **Applicazione da affiancare**: - **cache-app** è l'applicazione con il minor utilizzo di CPU e memoria, rendendola una scelta sicura per affiancare il carico batch critico. Ha un utilizzo di CPU molto basso e una memoria relativamente contenuta. 2. **Applicazione da evitare**: - **worker-app** presenta un utilizzo di CPU significativamente più alto rispetto a cache-app e demo-app, il che potrebbe portare a un degrado delle prestazioni per il carico batch. Inoltre, **demo-app** ha un utilizzo di memoria elevato, il che potrebbe anch'esso contribuire a problemi di "noisy neighbor". ### Conclusione Ti consiglio di schedulare il carico batch critico affiancando **cache-app** e di evitare di allocare risorse vicino a **worker-app** e **demo-app** per minimizzare il rischio di degrado delle prestazioni.

GPT-4o	35.29 s	9509	720	10229	0.00	Dopo aver analizzato le metriche storiche delle applicazioni nel cluster, ecco le raccomandazioni per l'affiancamento del carico batch critico: ### Applicazioni Analizzate: 1. **Cache-app** 2. **Demo-app** 3. **Worker-app** ### Analisi delle Metriche: - **Cache-app**: Ha un utilizzo di CPU molto basso e costante, con un consumo di memoria relativamente basso e stabile. Le operazioni di scrittura su disco sono moderate ma non eccessive. Questo indica che l'applicazione è leggera e non dovrebbe causare problemi di "noisy neighbor". - **Demo-app**: Mostra un utilizzo di CPU praticamente nullo e un consumo di memoria più elevato rispetto alle altre applicazioni, ma comunque stabile. Non ci sono operazioni di scrittura su disco o traffico di rete significativi. Questo suggerisce che l'applicazione è stabile, ma potrebbe occupare più memoria. - **Worker-app**: Ha un utilizzo di CPU significativamente più alto e variabile rispetto alle altre applicazioni, con un consumo di memoria moderato. Questo potrebbe indicare che l'applicazione è più suscettibile a causare problemi di "noisy neighbor" a causa delle fluttuazioni nell'utilizzo della CPU. ### Raccomandazioni: - **Affiancamento Consigliato**: **Cache-app** è l'applicazione più affidabile per l'affiancamento del carico batch critico. Il suo utilizzo di risorse è basso e stabile, riducendo il rischio di interferenze. - **Affiancamento da Evitare**: **Worker-app** dovrebbe essere evitata per l'affiancamento del carico batch critico. L'elevato e variabile utilizzo della CPU potrebbe causare problemi di "noisy neighbor", influenzando negativamente le prestazioni del carico batch. Queste raccomandazioni si basano sulle metriche storiche delle ultime 6 ore. Se hai bisogno di ulteriori analisi o di un periodo di tempo più lungo, ti consiglio di sottomettere la richiesta a un modello più avanzato.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.2391 (mini: $0.0533 | 4o: $1.1859)